



INVESTIGACIÓN ____

PROFUNDIZACIÓN __X__

Diseño de una plataforma para la ejecución de modelos de predicción sobre el mercado de divisas mediante machine learning

Autor

Kevin Steven Gordillo Orjuela

Director

Paulo Alonso Gaona García. PhD.

Universidad Distrital Francisco José De Caldas
Maestría en Ciencias de la Información y las Comunicaciones
Énfasis en Ingeniería de software
Bogotá, Colombia
Abril 2023

CONTENIDO

CONTENIDO	2
RESUMEN	10
PALABRAS CLAVE	10
1. INTRODUCCIÓN	11
1.1 PROBLEMA DE INVESTIGACIÓN	12
1.1.1 PLANTEAMIENTO DEL PROBLEMA	12
1.1.2 FORMULACIÓN DEL PROBLEMA	13
1.1.3 SISTEMATIZACIÓN DEL PROBLEMA	13
1.2 OBJETIVOS	14
1.2.1 OBJETIVO GENERAL	14
1.2.2 OBJETIVOS ESPECÍFICOS	14
1.3 JUSTIFICACIÓN	16
1.3.1 JUSTIFICACIÓN METODOLÓGICA	16
1.3.2 JUSTIFICACIÓN PRÁCTICA	16
2. MARCO DE REFERENCIA	17
2.1 MARCO TEÓRICO	17
2.1.1 ESTADO DEL ARTE	17
2.1.2 REVISIÓN DE ALGORITMOS A TRABAJAR	20
2.1.2.1 SVR-Wavelet Adaptive Model for Forecasting Financial Time Series	20
2.1.2.2 Bayesian Compressed Vector Autoregression for Financial Time-series Analysis and Forecasting	23
2.1.2.3 A CNN-LSTM model for gold price time-series forecasting	27
2.1.2.4 Stock Price Forecasting via Sentiment Analysis on Twitter	32
2.1.2.5 Price Forecasting for Agricultural Products Based on BP and RBF Neural Network	34
2.2 MARCO CONCEPTUAL	38
3. METODOLOGÍA	39
FASE I	39
FASE II	40
FASE III	41
FASE IV	41
4. COMPARATIVA DE RENDIMIENTO DE ALGORITMOS SELECCIONADOS	43
4.1 SELECCIÓN DE MÉTRICAS DE COMPARACIÓN	43
4.1.1 MAE	44
4.1.2 RMSE	44
4.1.3 MAPE	45
4.2 CASO DE ESTUDIO APLICADO Y RESULTADOS DE LA COMPARACIÓN	47
4.2.1 RENDIMIENTO: SVR-WAVELET ADAPTIVE MODEL	48
USD/EUR	49
USD/JPY	49
USD/COP	50

EUR/COP	51
TABLA DE RESUMEN DE RESULTADOS	52
4.2.2 RENDIMIENTO: BCVAR MODEL	53
Resultados para un día a futuro	53
Resultados para cuatro días a futuro	54
Resultados para doce días a futuro	55
4.2.3 RENDIMIENTO: CNN-LSTM MODEL	56
USD/EUR	56
USD/JPY	57
USD/COP	58
EUR/COP	59
4.2.4 RENDIMIENTO: ANÁLISIS DE SENTIMIENTOS - TWITTER	59
USD/EUR	60
USD/JPY	61
USD/COP	62
EUR/COP	63
4.2.5 RENDIMIENTO: RED NEURONAL - RBF/BP	64
USD/EUR	64
USD/JPY	65
USD/COP	66
EUR/COP	67
5. ESPECIFICACIÓN A ALTO NIVEL DEL SISTEMA	68
5.1 MODELO CONCEPTUAL DEL SISTEMA	68
5.2 MODELO DE PROCESOS	70
5.3 CARACTERIZACIÓN DE USUARIOS DEL SISTEMA	72
5.4 HISTORIAS DE USUARIO	73
5.4.1 Épica principal	74
5.4.2 Historias de usuario - Desarrollador	74
Tabla de historias - Desarrollador	77
Tabla de criterios de aceptación - Desarrollador	78
5.4.3 Historias de usuario - Usuario regular	79
Tabla de historias - Usuario regular	81
Tabla de criterios de aceptación - Usuario regular	81
5.4.4 Historias de usuario - Administrador	82
Tabla de historias - Administrador	85
Tabla de criterios de aceptación - Administrador	86
5.5 CASOS DE USO	89
5.5.1 CU01 - Vinculación de código del algoritmos a la plataforma	89
5.5.2 CU02 - Visualización de catálogo de algoritmos en la plataforma	91
5.5.3 CU03 - Ejecución de algoritmos	92
5.5.4 CU04 - Visualización de resultados	92
5.5.5 CU05 - Administración de usuarios y roles	94
5.5.6 CU06 - Administración de catálogos	96
5.5.7 CU07 - Administración de fuentes de entrada de datos	96

5.5.8 CU08 - Gestión de entornos de ejecución	97
5.5.9 CU09 - Gestión de notificaciones	99
5.5.10 CU10 - Gestión de proveedores de repositorios	100
5.5.11 CU11 - Gestión de proveedores de análisis de código	101
5.6 ARQUITECTURA DE LA APLICACIÓN	103
5.6.1 Autenticación	103
5.6.2 Servicio de composición de IGU	103
5.6.3 Microservicios Internos	104
5.6.3.1 Microservicios de vinculación y ejecución de código:	104
5.6.3.2 Microservicios de gestión de la plataforma:	105
5.6.3.3 Microservicios de resultados:	106
5.6.4 Tópicos y suscriptores	108
5.7 DISEÑO DE LOS COMPONENTES DE LA APLICACIÓN	112
5.7.1 Microservicio de detección de vulnerabilidades en el código	112
5.7.1.1 Modelo estructural	112
5.7.1.2 Modelo de datos	114
5.7.2 Microservicio de generación de entornos de ejecución	115
5.7.2.1 Modelo estructural	115
5.7.2.2 Modelo de datos	118
5.7.3 Microservicio de gestión de catálogo	119
5.7.3.1 Modelo estructural	119
5.7.3.2 Modelo de datos	120
5.7.4 Microservicio de vinculación de código	122
5.7.4.1 Modelo estructural	122
5.7.4.2 Modelo de datos	123
5.7.5 Microservicio de gestión de parámetros API	124
5.7.5.1 Modelo estructural	124
5.7.5.2 Modelo de datos	126
5.7.6 Microservicio de gestión de repositorios	127
5.7.6.1 Modelo estructural	127
5.7.6.2 Modelo de datos	128
5.7.7 Microservicio de gestión de notificaciones	129
5.7.7.1 Modelo estructural	129
5.7.7.2 Modelo de datos	131
5.7.8 Microservicio de gestión de usuarios	131
5.7.8.1 Modelo estructural	131
5.7.8.2 Modelo de datos	134
5.7.9 Microservicio de gestión de resultados	134
5.7.9.1 Modelo estructural	134
5.7.9.2 Modelo de datos	136
5.7.10 Microservicio de métricas	137
5.7.10.1 Modelo estructural	137
5.7.10.2 Modelo de datos	139
5.8 ESQUEMAS DE PÁGINAS WEB	140

6. RESULTADOS - PROTOTIPO	144
7. CONCLUSIONES	153
8. TRABAJOS FUTUROS	156
REFERENCIAS	157

LISTADO DE FIGURAS

Figura 1. Curva o hiperplano utilizado en SVR para describir la tendencia de los datos.....	21
Figura 2. Descomposición de los datos en componentes con diferentes escalas y frecuencias.....	22
Figura 3. Descripción del modelo SVR-Wavelet.....	23
Figura 4. Descripción del modelo VAR usando como ejemplo celulares y audífonos para un periodo t.....	24
Figura 5. Arquitectura típica de una red neuronal convolucional (CNN).....	28
Figura 6. Unidad de una neurona LSTM.....	29
Figura 7. Modelo número dos propuesto por los autores con dos capas de convolución, una capa de agrupamiento, una capa LSTM y una capa densa completamente conectada como salida.....	30
Figura 8. Modelo de análisis de sentimientos y predicción usando SVM.....	33
Figura 9. arquitectura y funcionamiento interno de una RBFNN.....	34
Figura 10. Gráfico de comparación entre los valores reales y los valores estimados para USD/EUR utilizando el algoritmo de SVR+Wavelets. En el eje Y se encuentra el valor de la moneda en una escala de 0 a 1, mientras que en el eje X se representa la fecha en la que se registró dicho valor.....	49
Figura 11. Gráfico de valor real vs valor estimado para USD/JPY usando algoritmo de SVR+Wavelets.....	50
Figura 12. Gráfico de valor real vs valor estimado para USD/COP usando algoritmo de SVR+Wavelets.....	51
Figura 13. Gráfico de valor real vs valor estimado para EUR/COP usando algoritmo de SVR+Wavelets.....	52
Figura 14. Gráfico de valor real vs valor estimado para USD/EUR, USD/JPY, USD/COP y EUR/COP usando algoritmo de BCVAR con h=4 (días a futuro).....	54
Figura 15. Gráfico de valor real vs valor estimado para USD/EUR, USD/JPY, USD/COP y EUR/COP usando algoritmo de BCVAR con h=12 (días a futuro).....	55
Figura 16. Gráfico de valor real vs valor estimado para USD/EUR usando algoritmo de CNN+LSTM.....	56
Figura 18. Gráfico de valor real vs valor estimado para USD/COP usando algoritmo de CNN+LSTM.....	58
Figura 19. Gráfico de valor real vs valor estimado para EUR/COP usando algoritmo de CNN+LSTM.....	59
Figura 20. Gráfico de valor real vs valor estimado para USD/EUR usando algoritmo de análisis de sentimientos con información de Twitter.....	60
Figura 21. Gráfico de valor real vs valor estimado para USD/JPY usando algoritmo de análisis de sentimientos con información de Twitter.....	61

Figura 22. Gráfico de valor real vs valor estimado para USD/COP usando algoritmo de análisis de sentimientos con información de Twitter.....	62
Figura 23. Gráfico de valor real vs valor estimado para EUR/COP usando algoritmo de análisis de sentimientos con información de Twitter.....	63
Figura 24. Gráfico de valor real vs valor estimado para USD/EUR usando algoritmo de red neuronal con RBF + BP.....	64
Figura 25. Gráfico de valor real vs valor estimado para USD/JPY usando algoritmo de red neuronal con RBF + BP.....	65
Figura 26. Gráfico de valor real vs valor estimado para USD/COP usando algoritmo de red neuronal con RBF + BP.....	66
Figura 27. Gráfico de valor real vs valor estimado para EUR/COP usando algoritmo de red neuronal con RBF + BP.....	67
Figura 28. Modelo conceptual del proceso que sigue un desarrollador para vincular un nuevo algoritmo, de predicción con ML, a la plataforma.....	69
Figura 29. Modelo conceptual del proceso que sigue un usuario regular para usar y analizar los modelos de predicción con ML dentro la plataforma.....	69
Figura 30. Modelo de proceso de negocio para la vinculación de un nuevo algoritmo de ML en la plataforma.....	70
Figura 31. Modelo de proceso de negocio para uso de los algoritmos de ML dentro de la plataforma.....	71
Figura 32. Modelo de épicas e historias de usuario usando notación BABOK para el rol de desarrollador.....	75
Figura 33. Modelo de historias de usuario y criterios de aceptación usando notación BABOK para el rol de desarrollador.....	76
Figura 34. Modelo de épicas e historias de usuario usando notación BABOK para el rol de usuario regular.....	80
Figura 35. Modelo de historias de usuario y criterios de aceptación usando notación BABOK para el rol de usuario regular.....	80
Figura 36. Modelo de historias de usuario y criterios de aceptación usando notación BABOK para el rol de administrador.....	83
Figura 37. Modelo de historias de usuario y criterios de aceptación usando notación BABOK para el rol de usuario regular.....	84
Figura 38. Diagrama de casos de uso CU01 - Vinculación de código del algoritmos de ML a la plataforma, desde la perspectiva del Desarrollador del algoritmo.....	90
Figura 39. Diagrama de casos de uso CU02 - Visualización de catálogo de algoritmos de ML en la plataforma, desde la perspectiva del usuario regular.....	91
Figura 40. Diagrama de casos de uso CU03 - Ejecución de algoritmos de ML seleccionados en el catálogo, desde la perspectiva del usuario regular.....	93
Figura 41. Diagrama de casos de uso CU04 - Visualización de resultados, desde la perspectiva del usuario regular.....	94
Figura 42. Diagrama de casos de uso CU05 - Gestión de usuarios y roles, desde la perspectiva del usuario administrador.....	95
Figura 43. Diagrama de casos de uso CU06 - administración de catálogos de algoritmos, desde la perspectiva del usuario administrador.....	96
Figura 44. Diagrama de casos de uso CU07 - administración de fuentes de entrada de datos, desde la perspectiva del usuario administrador.....	98
Figura 45. Diagrama de casos de uso CU08 - Gestión de entornos de ejecución, desde la perspectiva del usuario administrador.....	99
Figura 46. Diagrama de casos de uso CU09 - Gestión de notificaciones, desde la	

perspectiva del usuario regular.....	100
Figura 47. Diagrama de casos de uso CU010- Gestión de proveedores de repositorios, desde la perspectiva del usuario administrador.....	101
Figura 48. Diagrama de casos de uso CU011- Gestión de proveedores de análisis de código, desde la perspectiva del usuario administrador.....	102
Figura 49. Arquitectura de la aplicación utilizando Bus de eventos y microservicios.....	107
Figura 50. Diagrama de clases para el microservicio de análisis de vulnerabilidades en el código.....	113
Figura 51. Modelo de datos para base de datos documental del componente de análisis de código.....	114
Figura 52. Diagrama de clases para el microservicio de generación de entornos de ejecución.....	117
Figura 53. Modelo de datos para base de datos documental del componente de gestor de entornos de ejecución.....	118
Figura 54. Diagrama de clases para el microservicio de gestión de catálogo.....	120
Figura 55. Modelo de datos para base de datos documental del componente de gestión de catálogo.....	121
Figura 56. Diagrama de clases para el microservicio de gestión de vinculación de algoritmos	122
Figura 57. Modelo de datos para base de datos documental del componente de vinculación de código.....	123
Figura 58. Diagrama de clases para el microservicio de gestión de parámetros de la API a vincular.....	125
Figura 59. Modelo de datos para base de datos documental del componente de gestión de parámetros de la API de ML.....	126
Figura 60. Diagrama de clases para el microservicio de gestión de repositorios.....	127
Figura 61. Modelo de datos para base de datos documental del componente de gestión de repositorios.....	128
Figura 62. Diagrama de clases para el microservicio de gestión de repositorios.....	130
Figura 63. Modelo de datos para base de datos documental del componente de gestión de notificaciones.....	131
Figura 64. Diagrama de clases para el microservicio de gestión de usuarios.....	133
Figura 65. Modelo de datos para base de datos documental del componente de gestión de usuarios.....	134
Figura 66. Diagrama de clases para el microservicio de gestión de resultados.....	135
Figura 67. Modelo de datos para base de datos documental del componente de gestión de resultados.....	136
Figura 68. Diagrama de clases para el microservicio de métricas.....	138
Figura 69. Modelo de datos para base de datos documental del componente de cálculo de métricas.....	139
Figura 70. Esquema web de página principal. Se muestran los resultados principales arrojados por los algoritmos.....	140
Figura 71. Esquema web de página para la vinculación de algoritmos. El primer panel representa el espacio para incluir el enlace al repositorio. El segundo panel muestra el estado de la vinculación.....	141
Figura 72. Esquema web de página que muestra el catálogo general del sistema. Permite agregar algoritmos al catálogo personal.....	141
Figura 73. Esquema web de página web que permite interactuar con los algoritmos	

añadidos al catálogo personal. Permite realizar el llamado al método de entrenamiento y a los demás métodos que ofrece el algoritmo de ML seleccionado.....	142
Figura 74. Esquema web de página web que permite realizar las configuraciones dentro de la plataforma. Los paneles se activan y desactivan dependiendo del rol y permisos del usuario en el sistema.....	143
Figura 75. Arquitectura simplificada del prototipo implementado utilizando los servicios de AWS: Beanstalk, SNS, DynamoDB y API gateway. Se utilizan los lenguajes de programación Java y Javascript.....	147
Figura 76. Pantalla de vinculación de algoritmos a la plataforma. El usuario ingresa la información del algoritmo a vincular: enlace al repositorio, nombre del modelo, descripción y versión. Luego, al hacer clic en “Agregar” se dispara la validación del modelo.....	148
Figura 77. Pantalla del catálogo general de algoritmos vinculados a la plataforma. El usuario puede ver y escoger los modelos a usar.....	149
Figura 78. Pantalla del catálogo personal donde se puede hacer uso del algoritmo agregado, se tienen las opciones de iniciar el entrenamiento del algoritmos y de realizar una predicción una vez se ha entrenado.....	150
Figura 79. Pantalla principal que muestra la predicción del precio de cierta divisa para un día seleccionado y los resultados del entrenamiento para cada uno de los algoritmos utilizados en el catálogo personal.....	151
Figura 80. Pantalla principal que muestra las métricas de comparación: MAE, MAPE y RMSE entre los diferentes algoritmos entrenados.....	152

RESUMEN

El objetivo de este proyecto es presentar una propuesta de diseño para crear una plataforma tecnológica que permita integrar varios algoritmos de Machine Learning (ML). Para su desarrollo se tomará como base una revisión de literatura para identificar algoritmos más comunes abordados en este campo durante los últimos años. La propuesta se encuentra dividida en varias secciones. En la primera, se exponen el problema de investigación, los objetivos y la justificación. En las siguientes secciones, se presenta el marco de referencia o estado del arte y la metodología de investigación. En la sección cuatro, se realiza una comparativa de rendimiento entre los cinco algoritmos seleccionados para este caso de estudio. En la sección cinco, se desarrolla el diseño de la plataforma, incluyendo especificaciones del sistema y los componentes. Finalmente, en las secciones seis y siete se presentan las conclusiones y los trabajos futuros que podrían derivarse de este trabajo.

PALABRAS CLAVE

Machine Learning (ML), FOREX, predicción de mercados, redes neuronales convolucionales (CNN), Long-short-Term Memory neural network (LSTM-NN), Bayesian Compressed VAR (BCVAR), transformada de Wavelet, análisis de sentimientos, twitter, Radial Basis Function NN.

1. INTRODUCCIÓN

Actualmente se han planteado una serie de modelos y algoritmos que buscan predecir el valor de diferentes monedas en el mercado de divisas, también conocido como "Forex". Con la creciente importancia de la inteligencia artificial, se han desarrollado modelos de predicción basados en "machine learning" que se dividen en siete categorías principales: métodos de regresión, técnicas de optimización, máquinas de vectores de soporte (SVM), redes neuronales, teoría del caos, métodos basados en patrones y otros métodos que incluyen procesamiento de lenguaje natural (Islam et al., 2020).

El número de artículos científicos publicados sobre la predicción de precios en el mercado de divisas es muy alto. Según el estudio de Islam et al. realizado en 2020, en el periodo de 2017 a 2019 se publicaron alrededor de 79 artículos sobre este tema. Sin embargo, sólo 39 de estos artículos utilizan técnicas de "machine learning". Aunque las técnicas utilizadas en estos 39 artículos son similares, cada autor tiene su propio método para aplicarlas y algunos añaden variaciones para mejorar las predicciones de acuerdo a criterios y condiciones de ciertos mercados. El objetivo de este documento es proponer el diseño de una plataforma de software que permita la ejecución de varios modelos basados en "machine learning", para predecir los precios del mercado de divisas, con el propósito de identificar elementos que faciliten su implementación dentro de diversos escenarios y facilitar el proceso de despliegue y medición de estos modelos.

El proyecto se encuentra dividido en cuatro fases, cada una asociada con diversas actividades relacionadas con el análisis, caracterización de algoritmos basados en machine learning, así como las diversas fases definidas para el análisis, diseño y desarrollo de software para el planteamiento de la plataforma. En la primera fase, se llevará a cabo un proceso de revisión de literatura orientada a identificar modelos de machine learning para predecir precios en el mercado de divisas. En la segunda fase, se recopilarán los requisitos del negocio y se definirá el alcance de la plataforma de conformidad con criterios identificados por algoritmos seleccionados. En la tercera fase, se diseñará la solución y se definirá su arquitectura de conformidad con las especificaciones de software requeridas para tal fin. Por último, en la cuarta fase, se diseñará y creará un prototipo que permita validar el diseño propuesto.

1.1 PROBLEMA DE INVESTIGACIÓN

1.1.1 PLANTEAMIENTO DEL PROBLEMA

En los últimos años, la inteligencia artificial, y en particular el campo del "machine learning", ha dado lugar a una gran cantidad de investigaciones sobre la predicción de precios en mercados financieros, tales como los de hidrocarburos, metales preciosos, divisas, entre otros.

Algunas propuestas para predecir los precios en el mercado de divisas o "FOREX" mediante técnicas de machine learning son modelos híbridos que combinan transformaciones matemáticas con técnicas de aprendizaje supervisado, como las máquinas de vectores de soporte o SVM. Un ejemplo es el estudio realizado por M.S. Raimundo et al. en 2018, en el que se utilizaron modelos de Wavelet y SVM para predecir el precio de conversión de varias monedas, incluyendo el franco suizo (CHF) a yen japonés (JPY) y a euro (EUR). Los criterios utilizados para evaluar la precisión de la predicción fueron el margen de error (ME), el error cuadrático medio (MSE) y el error absoluto medio (MAE).

Otras propuestas para la predicción de precios en el mercado de divisas se enfocan en mejorar los algoritmos de machine learning existentes mediante el uso de técnicas de optimización, como Jaya presentado por Das et al. (2020). Algunas investigaciones también utilizan algoritmos genéticos, como la optimización por enjambre de partículas (PSO). Por ejemplo, en el estudio realizado por Pradeep Kumar et al. (2017), se usó el algoritmo PSO para entrenar una red neuronal de regresión cuantílica, permitiendo predecir el valor del mercado de divisas para USD/INR, EUR/USD y USD/JPY. La investigación arrojó buenos resultados basados en el ME y el MSE, aunque solo para ciertas divisas y no para todas las utilizadas en el estudio.

Aunque se han realizado numerosos estudios sobre este tema, cada uno con resultados prometedores, se ha investigado poco en el desarrollo de una plataforma que permita recopilar varios de estos modelos y ejecutarlos en un ambiente que permita su comparación estadística. Algunos trabajos presentados por Zhao, S. et al. (2018) o Borgli et al. (2019) se centran en plataformas que permiten el intercambio y la ejecución de modelos de ML de manera general, pero no en un contexto específico como el mercado FOREX.

Una plataforma que permita la comparación de técnicas de ML sería de gran utilidad para la elección de modelos específicos que optimicen la predicción del valor de ciertas monedas y, a su vez, permita la combinación de algunos de estos modelos para obtener resultados aún más precisos.

1.1.2 FORMULACIÓN DEL PROBLEMA

Dada la carencia de plataformas que permiten la ejecución y comparación de múltiples algoritmos, nace la siguiente pregunta de investigación:

¿Cómo diseñar una plataforma que permita la ejecución de múltiples algoritmos de machine learning para la predicción de precios sobre el mercado de divisas?

1.1.3 SISTEMATIZACIÓN DEL PROBLEMA

Ya que se trata de una plataforma de software, se deben considerar los requerimientos y entradas del sistema. ¿Cuáles son las variables y requerimientos de software a considerar para desarrollar una plataforma que permita la ejecución de diferentes modelos de machine learning para la predicción de valores sobre el mercado de divisas?

Cada modelo de machine learning presenta diferentes métricas para establecer su nivel de precisión respecto a la predicción a realizar. ¿Cuáles son las métricas comunes que se deben tener en cuenta para hacer una comparación entre los diferentes modelos?

Cada modelo de machine learning tiene una arquitectura y características propias. Por ende, se deben tener en cuenta diversas restricciones tecnológicas. ¿Qué restricciones, a nivel tecnológico, se deben establecer para integrar en una sola plataforma algunos de los múltiples modelos de machine learning existentes para la predicción del precio de divisas en el mercado?

La plataforma busca proporcionar métricas para proponer un modelo a aplicar a una moneda determinada. ¿Cómo se puede probar que la plataforma diseñada es capaz de proporcionar la información necesaria para comparar diferentes modelos de aprendizaje automático para la predicción del precio de divisas en el mercado?

1.2 OBJETIVOS

1.2.1 OBJETIVO GENERAL

Diseñar una plataforma de software que permita la ejecución y comparación de modelos de predicción de precios del mercado de divisas basados en una selección de cinco técnicas de machine learning.

1.2.2 OBJETIVOS ESPECÍFICOS

- Identificar algoritmos basados en técnicas de machine learning que permitan ajustarse a procesos de predicción del mercado de divisas.
- Definir las métricas en común para los diferentes modelos de predicción que se incorporarán en la plataforma.
- Realizar el diseño de la plataforma que soporte la ejecución de los algoritmos de predicción, a través de un caso de estudio sobre un sector específico del mercado de divisas.
- Implementar un prototipo del sistema que compare los diferentes modelos implementados en la plataforma usando las métricas definidas.

1.3 JUSTIFICACIÓN

1.3.1 JUSTIFICACIÓN METODOLÓGICA

La presente propuesta se orienta a desarrollar el diseño de software de una plataforma que permita la integración de múltiples modelos de predicción de precios del mercado de divisas. Por lo tanto, se propone una nueva forma de abordar este problema de predicción de divisas: incorporar varios algoritmos reportados en la literatura dentro de un sistema que permita la comparación de su desempeño. Los resultados de esta investigación podrán ser usados a futuro en el desarrollo de la plataforma y, a su vez, en el procedimiento para la incorporación de nuevos modelos de predicción.

En este caso de estudio en particular, se ha optado por seleccionar cinco técnicas de Machine Learning, siguiendo el enfoque adoptado en otros estudios como los de Bansal et al. (2022) y Biswas et al. (2021). Estos estudios han demostrado que cinco técnicas son suficientes para llevar a cabo un caso de estudio de manera efectiva y obtener resultados significativos.

1.3.2 JUSTIFICACIÓN PRÁCTICA

Como se planteó anteriormente, abundan las investigaciones sobre modelos de predicción de precios de divisas usando técnicas de machine learning, aunque no se encuentran plataformas que permitan su comparación.

Con el fin de comparar y permitir la selección del modelo que mejor se acomode a la predicción de cierta moneda, se propone una plataforma que posibilite la ejecución de varios de estos algoritmos de predicción y proporcione ciertas métricas útiles que concedan información relevante para la toma de decisiones en cuanto a la compra de determinada divisa.

2. MARCO DE REFERENCIA

2.1 MARCO TEÓRICO

2.1.1 ESTADO DEL ARTE

El mercado de divisas, conocido también como "FOREX", es un mecanismo transaccional en el cual los interesados pueden adquirir unidades de una moneda para comprar una cantidad proporcional en otra. Existen dos tipos de análisis sobre este mercado: el análisis fundamental y el análisis técnico. En este trabajo, nos enfocaremos en el análisis técnico, el cual, en la mayoría de los casos, se basa en gráficos estadísticos y técnicas de machine learning (Sidehabi et al., 2016).

La tarea principal del machine learning es enseñar a una computadora las habilidades necesarias para clasificar y agrupar elementos de un conjunto de datos y encontrar patrones (Thi Thu y Xuan, 2018). En el caso específico del mercado de divisas, también conocido como FOREX, la mayoría de las técnicas de machine learning se enfocan en realizar predicciones en un periodo corto de tiempo, como estimar el valor de una moneda en los próximos treinta minutos. Esto permite a los negociantes tomar decisiones más informadas respecto a las divisas que compran o venden.

Como se mencionó anteriormente, en la revisión del estado del arte realizada por Islam et al. (2020), se identificaron siete categorías principales en las que se pueden clasificar los actuales avances en ML aplicados a la predicción en el mercado de divisas. Todos estos trabajos tienen en común el uso de medidas estadísticas para establecer la precisión de sus predicciones. Aunque las medidas utilizadas pueden variar según el enfoque del modelo, generalmente se emplean las siguientes medidas estadísticas: MAE (Mean Absolute Error) o Error Absoluto Medio, MSE (Mean Square Error) o Error Cuadrático Medio, RMSE (Root-mean-square Error) o Raíz Cuadrada del Error Cuadrático Medio, y MAPE (Mean Absolute Percentage Error) o Error Porcentual Absoluto Medio. Todas estas medidas buscan mostrar qué tan lejos está la predicción del valor real en el mercado.

Dentro de las propuestas que se encuentran en la literatura sobre la predicción de precios en FOREX usando ML, se encuentra la tesis de maestría presentada por Wazir Mohammadi en 2019, de la Universidad de NEU de Ciencias Aplicadas en Nicosia (Mohammadi, 2019). En dicho estudio, Mohammadi se centró en la posibilidad de realizar la predicción basado en el análisis de datos históricos del mercado FOREX usando cuatro modelos de ML: propagación hacia atrás (Backpropagation), función de base radial (Radial Basis Function), memoria a corto plazo (Long short-term memory) y máquinas de vectores de soporte (Support Vector Regression). Su trabajo se desarrolló en el lenguaje de programación "Python" y se centró en tres pares de monedas: EUR/USD, USD/JPY y USD/TRY. Como conclusiones, establece que aunque la información histórica es útil para realizar predicciones, no es suficiente para establecer un modelo preciso debido a la volatilidad del mercado. Además, expone que el algoritmo SVR tiene el mejor rendimiento y que el menos eficaz es el método de propagación hacia atrás.

Algunos modelos se enfocan en mejorar o combinar técnicas de ML con procesamiento de lenguaje natural. Por ejemplo, el trabajo realizado por Semironi et al. (2020) combina el uso de información textual de noticias del calendario económico con indicadores e información histórica del mercado de divisas para entrenar modelos de ML como "Extreme Gradient Boosting" (XGB), "Random Forest" (RF) y SVM. Al final del estudio, se obtuvo una precisión en las medidas estadísticas por encima del 60%, y en algunos casos del 64%, para predicciones en los siguientes treinta minutos. Según los autores, esto demuestra que la minería de texto combinada con técnicas de ML puede mejorar la precisión de predicción en FOREX.

La predicción del valor de algunas monedas con alta volatilidad, como Bitcoin, ha sido objeto de investigación por parte de autores como (Chen et al., 2021). En su estudio, se utilizaron múltiples modelos de ML para determinar el mejor enfoque. La investigación se dividió en dos fases. En la primera fase, se entrenó una red neuronal y se utilizó el método de "Random Forest" (RF) con factores tecnológicos como tamaño promedio del bloque en la red Blockchain de Bitcoin, tasa de Hash, ganancia por minado, volumen de transacciones, capitalización de mercado, entre otros. También se consideraron factores económicos como indicadores macroeconómicos, como el precio del petróleo o el oro, y el valor de cambio de algunas monedas en relación al dólar, como el euro y el yen japonés. En la segunda fase, se utilizó la técnica LSTM teniendo en cuenta el valor histórico y previo de la moneda. Los resultados mostraron que la predicción del precio de criptomonedas, como Bitcoin, es más precisa cuando se tienen en cuenta los factores tecnológicos y económicos del momento.

Aunque existen muchos modelos y estudios sobre la predicción del mercado de divisas, hay pocas iniciativas que se enfoquen en recopilar, ejecutar y comparar dichos modelos en una plataforma de software. Una de estas iniciativas es "Saga", una plataforma de código abierto presentada por Borgli et al. en 2019. Saga permite compartir modelos de ML, técnicas de entrenamiento para redes neuronales y conjuntos de datos. Además, la plataforma ofrece la posibilidad de mezclar varios modelos e incluir extensiones para mejorar el entrenamiento de redes neuronales. Esta iniciativa promueve la colaboración y el intercambio de conocimientos en el campo de la predicción del mercado de divisas.

La plataforma NAVER Smart Machine Learning (NSML), presentada por (Sung N. et al., 2017), tiene como objetivo ayudar a los investigadores a enfocarse en el modelado de sistemas de ML, en lugar de realizar tareas de bajo nivel como el monitoreo del estado de entrenamiento de redes neuronales o la asignación de unidades de procesamiento gráfico. El sistema consta de tres partes principales: un planificador de tareas, que asigna de manera eficiente la memoria para ejecutar las tareas de ML; un sistema de contenedores que proporciona un entorno de ejecución y almacenamiento para los modelos que se ejecutan en el sistema; y una interfaz gráfica que actúa como un mediador entre los resultados generados por el sistema y los flujos de entrada proporcionados por los usuarios.

Según (Shuai et al., 2018), los modelos de aprendizaje automático suelen ser desarrollados para tareas específicas utilizando diferentes frameworks, lo que dificulta su reutilización y mejora. Para abordar esta problemática, presentan la plataforma de código abierto llamada "Acumos", que permite empaquetar modelos de ML en contenedores de microservicios y compartirlos a través del catálogo de la plataforma. Aunque esta plataforma facilita la reutilización de modelos al tratarlos como "cajas negras" y enfocarse en la comunicación, no permite la optimización de modelos y deja esta tarea a cargo de los desarrolladores.

2.1.2 REVISIÓN DE ALGORITMOS A TRABAJAR

A continuación se detallan teóricamente los cinco algoritmos seleccionados para la implementación del prototipo de la plataforma. Estos algoritmos fueron elegidos debido a su nivel de detalle en la descripción de la técnica utilizada para la predicción. Esto es importante ya que se busca replicar los resultados de cada uno de estos algoritmos e implementar su lógica en el código de software.

2.1.2.1 SVR-Wavelet Adaptive Model for Forecasting Financial Time Series

Este trabajo fue realizado por Raimundo et al. (2018). El estudio se enfoca en el uso de vectores de soporte de regresión (SVR) y la transformada de Wavelet para realizar predicciones en series financieras en el tiempo, específicamente en el mercado de divisas (FOREX).

Para proporcionar un contexto a la idea presentada por los autores, es importante explicar la idea principal detrás de SVR.

Los SVR son un modelo matemático utilizado en Machine Learning que permite realizar análisis de regresión y clasificación de datos. En este algoritmo, se busca construir una curva o hiperplano que se ajuste a la tendencia de los datos de entrada, y a partir de ella se crean dos bandas o vectores de soporte que cubren la mayor cantidad de datos posible. Cualquier valor que no pueda ser encerrado dentro de estas bandas se considera un error. El objetivo es encontrar la mejor curva o hiperplano que minimice el error del modelo, lo que se logra mediante la selección de parámetros óptimos durante el entrenamiento. La Figura 1 muestra un ejemplo visual de este proceso.

La formulación matemática del modelo se expone a continuación:

El SVR utilizando SVM, como lo plantea (M.S, Raimundo et al., 2018), se puede expresar de la siguiente manera:

$$\gamma_t = \omega_t \Phi(x_t) + b$$

Donde:

γ_t : es el valor predicho dado un vector de la serie de tiempo

ω_t : es el vector de pesos, está encargado de darle la forma al "vector principal"

x_t : es el vector de datos de entrada (de la serie de tiempo)

b : es el sesgo (bias), también se puede interpretar como la distancia del hiperplano al origen

Φ : función encargada de transformar los datos a una dimensión más grande: $R^D \rightarrow R^M$; $M > D$

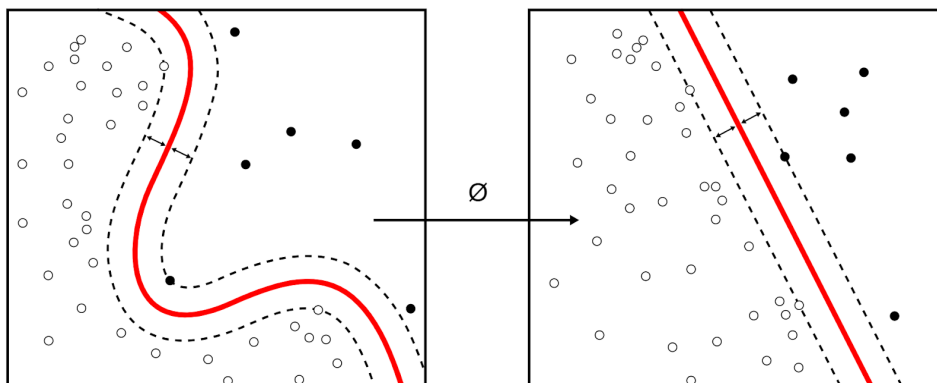


Figura 1. Curva o hiperplano utilizado en SVR para describir la tendencia de los datos.

Fuente: By Original: Alisneaky Vector: Zirguezi - Own work based on: Kernel Machine.png, CC BY-SA 4.0, <https://commons.wikimedia.org/w/index.php?curid=47868867>

Por otro lado, la transformada de Wavelet es una técnica que busca descomponer una señal o un vector de datos en diferentes componentes con distintas escalas y frecuencias. Esto permite analizar cada uno de estos elementos por separado, encontrar patrones y eliminar posibles ruidos en la señal o en los datos. Por ejemplo, como se puede observar en la figura 2, los datos de una serie de tiempo para el valor de una moneda son descompuestos en subcomponentes que tienen diferentes frecuencias y magnitudes. La última componente de esta serie de tiempo, por ejemplo, es una función más limpia y con un menor número de fluctuaciones, lo que nos indica que puede ser una señal útil para el análisis.

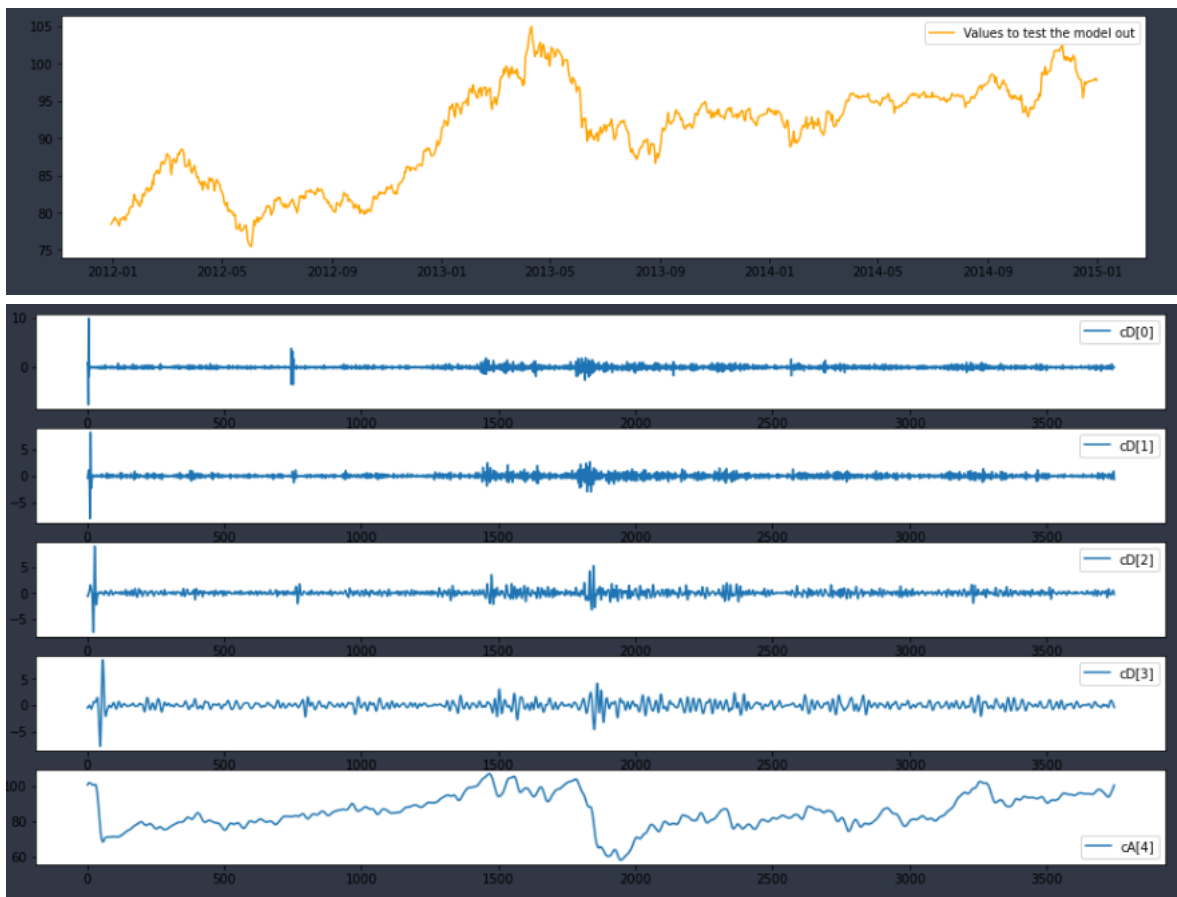


Figura 2. Descomposición de los datos en componentes con diferentes escalas y frecuencias.

Fuente: Autoría propia

El objetivo de este modelo es combinar ambas técnicas para mejorar la precisión de la predicción sobre una serie temporal. El proceso es relativamente simple: primero se descompone la serie temporal original utilizando la transformada de Wavelet. Luego, se utiliza el algoritmo SVR para calcular la predicción sobre el componente con menos ruido.

Finalmente, se recombinan las capas y se obtiene la predicción para "n" pasos en el futuro. Esto se ilustra de manera más clara en la figura 3.

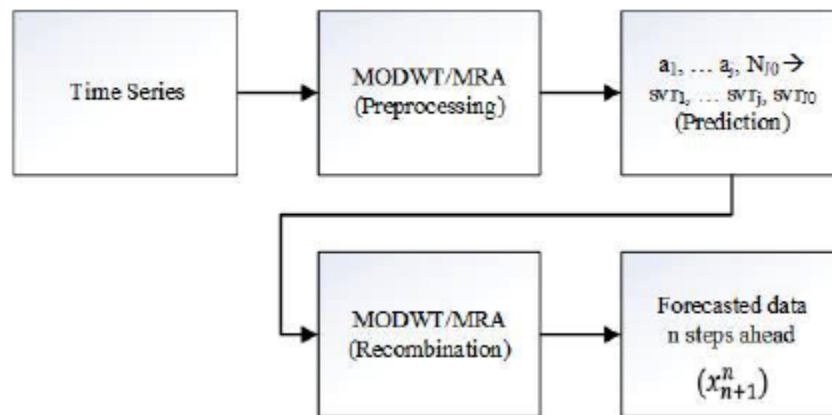


Figura 3. Descripción del modelo SVR-Wavelet.

Fuente: Raimundo, M.S.; Okamoto, J. (2018, Marzo). SVR-wavelet adaptive model for forecasting financial time series. In Proceedings of the 2018 International Conference on Information and Computer Technologies (ICICT), DeKalb, IL, USA, 23–25; pp. 111–114.

Este modelo, según los autores, ha demostrado tener un mejor rendimiento que los modelos ARIMA y ARFIMA en términos de predicción del precio del dólar australiano frente al yen japonés. Para comparar los modelos, se utilizaron varias métricas, incluyendo ME (Error Medio), RMSE (Raíz del Error Cuadrático Medio), MAE (Error Absoluto Medio), MPE (Error Porcentual Medio) y MAPE (Error Porcentual Absoluto Medio).

Adicionalmente, para el proceso de entrenamiento se utiliza el 70% de los datos de entrada y se deja el 30% restante para el cálculo del error.

2.1.2.2 Bayesian Compressed Vector Autoregression for Financial Time-series Analysis and Forecasting

Este trabajo publicado por T. Paponpat et al. (2019) tiene sus orígenes en el trabajo previo publicado por G. Koop, D. Korobilis y D. Petternuzzo (2019), en el que se combina el modelo vectorial autorregresivo (VAR) con el modelo de compresión Bayesiana (BC). A continuación se explica a grandes rasgos cómo funciona este modelo matemático.

El modelo VAR (Vector AutoRegressive) es utilizado para analizar la relación entre varias variables, de tal manera que si una variable es afectada por un evento, esto afectará a todo el modelo. Por ejemplo, si se tienen dos variables como la inflación y el desempleo, cualquier cambio en una de estas variables tendría un impacto en la otra.

En nuestro día a día podemos encontrar muchos ejemplos de modelos VAR. Por ejemplo, imaginemos que somos vendedores de productos electrónicos como teléfonos móviles y auriculares. Cada mes, queremos estimar cuántos auriculares podemos vender en función de la cantidad de teléfonos móviles que se vendieron el mes anterior, y viceversa. La figura 4 ilustra cómo funciona el modelo VAR(1), que predice el número de ventas para un solo período de tiempo en el futuro. En términos matemáticos, esta situación se puede representar como: la cantidad de teléfonos móviles vendidos está determinada por la cantidad de teléfonos móviles y auriculares vendidos en el período anterior, y viceversa.

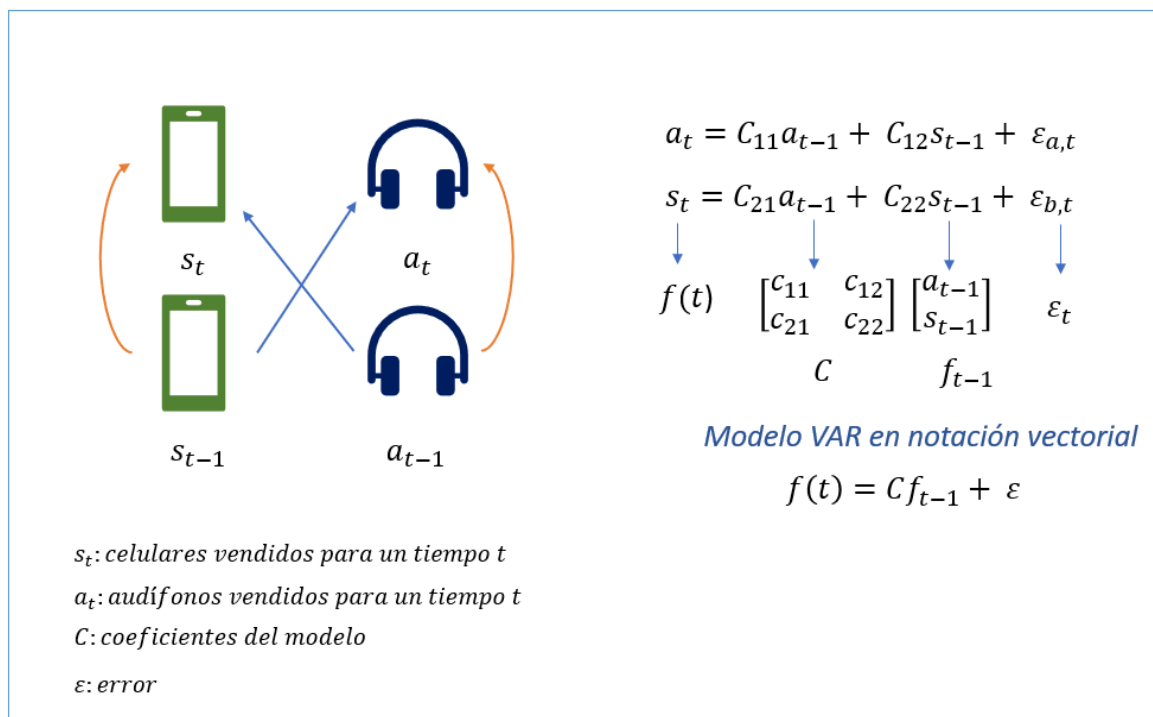


Figura 4. Descripción del modelo VAR usando como ejemplo celulares y audífonos para un periodo t .

Fuente: Autoría propia

Los modelos VAR son comúnmente utilizados para la predicción en mercados financieros. No obstante, cuando el número de variables se incrementa, la carga computacional también aumenta. Este problema, conocido como "problema de dimensionalidad", se presenta cuando el número de predictores en las ecuaciones del modelo VAR es mayor que el número de observaciones. En otras palabras, se habla de "métodos con alta dimensionalidad" para referirse a los modelos que enfrentan este problema. (T. Paponpat et al., 2019) lo explican detalladamente.

Hay muchos estudios que buscan reducir la complejidad computacional de los modelos VAR mediante la reducción de su dimensionalidad. El enfoque del artículo mencionado anteriormente utiliza la regresión Bayesiana para comprimir aleatoriamente los predictores del modelo VAR a través del uso de una matriz especial. Esto se conoce como el modelo BCVAR (Bayesian Compressed VAR). A continuación, se explica de manera general cómo funciona este modelo.

El modelo VAR, que se muestra en la figura 4, se puede extender a “p” periodos de tiempo, de manera tal que la formulación matemática se da como:

$$Y_t = BY_{t-p} + \varepsilon_t$$

Donde “B” es la matriz de coeficientes que se quieren estimar para obtener una mejor predicción. Y_{t-p} es la matriz de predictores o mediciones previas con “p” periodos en el pasado. Como se mencionó anteriormente, a medida que el número de predictores en el modelo VAR crece, también lo hace el costo computacional. Es por esto que se propone agregar una matriz de proyección generada de manera aleatoria que permite comprimir la matriz de predictores. Esto significa que se seleccionan solo algunas variables y no la totalidad de las variables del modelo.

El modelo dada una matriz de compresión Φ se puede escribir como:

$$Y_t = B^c(\Phi Y_{t-1} + \varepsilon_t)$$

Donde, B^c es la matriz de coeficientes comprimida y Φ es una matriz aleatoria de proyección que busca comprimir la cantidad de estimadores del modelo inicial. Sin embargo, una sola iteración no es suficiente para tener un modelo adecuado de predicción. Para esto, se debe iterar sobre el modelo múltiples veces; lo que implica generar múltiples matrices de compresión.

El modelo completo de BCVAR es expuesto por los autores (G. Koop, D. Korobilis, D. Petternuzzo, 2019). Dentro del modelo, se iteran varios modelos con diferentes matrices de compresión para buscar cuál de ellos obtiene mejores resultados. En la teoría Bayesiana, existe un enfoque llamado promedio de modelo Bayesiano (BMA, por sus siglas en inglés). Este enfoque busca obtener la mejor predicción dada una combinación de “R” modelos o proyecciones para “h” períodos de tiempo a futuro. Para el modelo BCVAR, tal como lo establecen los autores, la predicción utilizando BMA se realiza de la siguiente forma:

$$p(Y_{t+h}|D^t) = \sum_{r=1}^R \omega_r p(Y_{t+h}|M_r, D^t),$$

Donde:

D^t : es el conjunto de datos o información disponible para un tiempo t

ω_r : es el peso del modelo "r" (M_r). Está dado por: $\omega_r = \exp(-0.5\Psi_r) / \sum_{r=1}^R \exp(-0.5\Psi_r)$

Ψ_r : es el resultado de la resta del Criterio de Información Bayesiana (BIC, por la siglas en inglés) del modelo "r" y el mínimo BIC entre todos los modelos evaluados:

$$\Psi_r = BIC_r - BIC_{min}$$

$$BIC = k_i \times \ln(T) \times \ln\left(\frac{SSE}{T}\right),$$

k : predictores, T : Observaciones de los datos, SSE : suma de los errores cuadráticos (SE)

Este modelo es el usado en el artículo de: T. Paponpat et al. (2019), para realizar predicciones sobre múltiples pares de divisas cómo: AUD-CAD, AUD - JPY, EUR-JPY, EUR-AUD, entre otros. (AUD: Australian Dollar. CAD: Canadian Dollar. JPY: Japanese Yen. EUR: Euro).

Para el entrenamiento del modelo se utilizó el 75% de los datos y el 25% restante para calcular el error. Para obtener la precisión del modelo sólo se utilizó la métrica de: MSFE (Mean Square Forecasting Error).

2.1.2.3 A CNN–LSTM model for gold price time-series forecasting

El estudio llevado a cabo por (Livieris et al., 2020) utiliza redes neuronales para predecir el precio del oro en el mercado bursátil. Concretamente, se combinan dos modelos de aprendizaje automático: redes neuronales convolucionales (CNN) y redes neuronales recurrentes de memoria a corto-largo plazo (LSTM).

La principal ventaja de las CNN es su capacidad para extraer características de los datos. A menudo se utilizan en el procesamiento de imágenes, ya que su arquitectura en capas permite descomponer los datos de la imagen en componentes más pequeños y analizarlos utilizando filtros.

Para comprender mejor el funcionamiento de las CNN, en la figura 5 se ilustra la arquitectura básica de este tipo de redes neuronales. Estas redes funcionan esencialmente mediante dos tipos de capas: capas de convolución y capas de agrupación.

Podemos entender la convolución como un proceso en el cual se aplican filtros al conjunto de datos original con el fin de extraer o resaltar ciertas características de la entrada. Por ejemplo, en el procesamiento de imágenes, se busca extraer las líneas o ejes principales que, posteriormente, en la capa de agrupación, serán interpretados por la red para realizar una estimación o clasificación. Las convoluciones se realizan utilizando filtros o "kernels", los cuales son matrices de tamaño $m \times m$ que se desplazan por todo el conjunto de datos para obtener las matrices de características correspondientes.

Por otro lado, las capas de agrupación tienen como función reducir el tamaño de la imagen o datos de entrada y resaltar las características más importantes. Su objetivo es detectar patrones o elementos simples que se forman a partir de las matrices de características obtenidas en las capas de convolución.

Por lo general, este tipo de redes se diseñan con múltiples capas de convolución y agrupación, y al final de la red se coloca una capa de salida completamente conectada que permite realizar la clasificación.

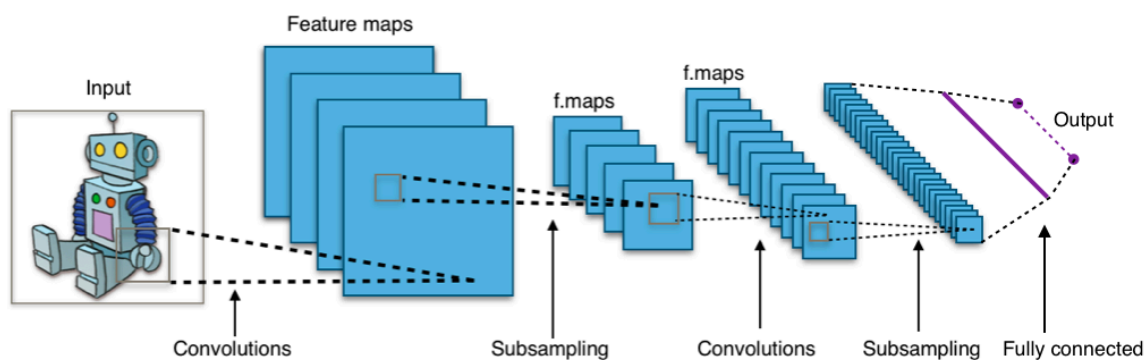


Figura 5. Arquitectura típica de una red neuronal convolucional (CNN).

Fuente: By Aphex34 - Own work, CC BY-SA 4.0,
<https://commons.wikimedia.org/w/index.php?curid=45679374>

En cuanto a las Redes Neuronales LSTM, pertenecen a la categoría de redes recurrentes y son especialmente útiles para el procesamiento de secuencias de datos. Sus dos principales características son: primero, que permiten capturar y retener información importante de la secuencia a través de la memoria de la red, y segundo, que tienen la

capacidad de seleccionar qué información retener y cuál desechar mediante puertas o "gates".

A diferencia de las redes CNN, las redes recurrentes permiten procesar secuencias de datos como videos, texto escrito, voz, series de tiempo, entre otros, ya que estos datos tienen sentido cuando se analizan en conjunto. Por ejemplo, al analizar un texto, no es suficiente interpretar palabra por palabra o letra por letra, sino que es necesario analizar cómo se concatenan estas palabras/letras para dar sentido a una oración o frase.

Para analizar la correlación entre los datos, las redes neuronales recurrentes no solo utilizan la activación de la neurona actual, sino que también utilizan la activación de la iteración anterior. En otras palabras, dichas redes implementan una especie de "memoria" que les permite analizar los resultados de una activación previa.

Como se muestra en la Figura 6, las neuronas de una red LSTM están compuestas por tres compuertas que controlan el flujo de información hacia y desde la celda de estado "C", que puede entenderse como una banda que almacena la información relevante de la red. Estas compuertas actúan como válvulas que permiten modificar la celda de estado, ya sea para olvidar, agregar o dar una salida. La primera compuerta es la de olvido o "Forget gate", que permite eliminar información innecesaria de la celda de estado. La segunda es la compuerta de entrada o "Input gate", que permite agregar información nueva a la celda de estado. Por último, la compuerta de salida o "Output gate" nos permite generar la salida de la neurona LSTM.

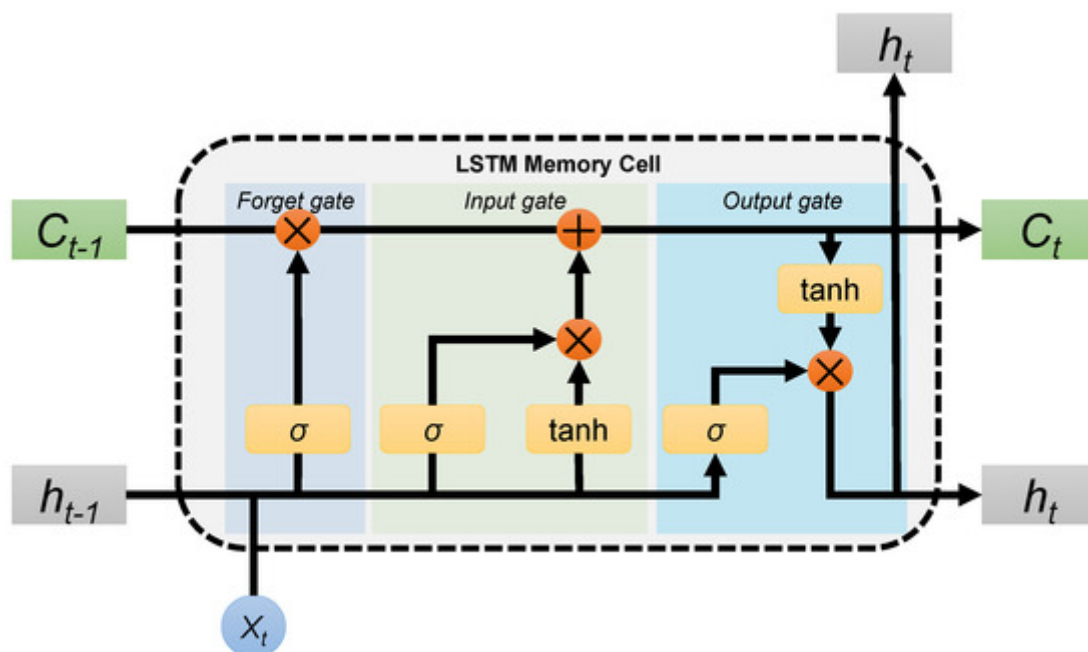


Figura 6. Unidad de una neurona LSTM.

Fuente: Fan, H.; Jiang, M.; Xu, L.; Zhu, H.; Cheng, J.; Jiang, J. Comparison of Long Short Term Memory Networks and the Hydrological Model in Runoff Simulation. *Water* **2020**, 12, 175.

Como lo plantean los autores (Livieris et al., 2020), la motivación de unir ambos modelos de redes neuronales es combinar de manera eficiente las bondades de ambos modelos. En palabras de los mismos:

“Las capas convolucionales están caracterizadas por su habilidad de extraer conocimiento relevante y aprender la representación interna de los datos de series de tiempo, mientras que las redes LSTM son eficientes identificando dependencias de corto y largo plazo. La idea principal de nuestro modelo propuesto es combinar de manera eficiente las ventajas de estas dos técnicas de aprendizaje profundo.”¹ (Livieris et al., 2020, p. 17353).

A grandes rasgos, la funcionalidad de este modelo consiste en utilizar una red convolucional para extraer los componentes principales de la serie de tiempo, en este caso la del precio del oro. Posteriormente, se utiliza una red LSTM para procesar dicha información filtrada y realizar una predicción, teniendo como salida una capa densa de "n" neuronas. En la figura 7 se muestra la arquitectura de este modelo.

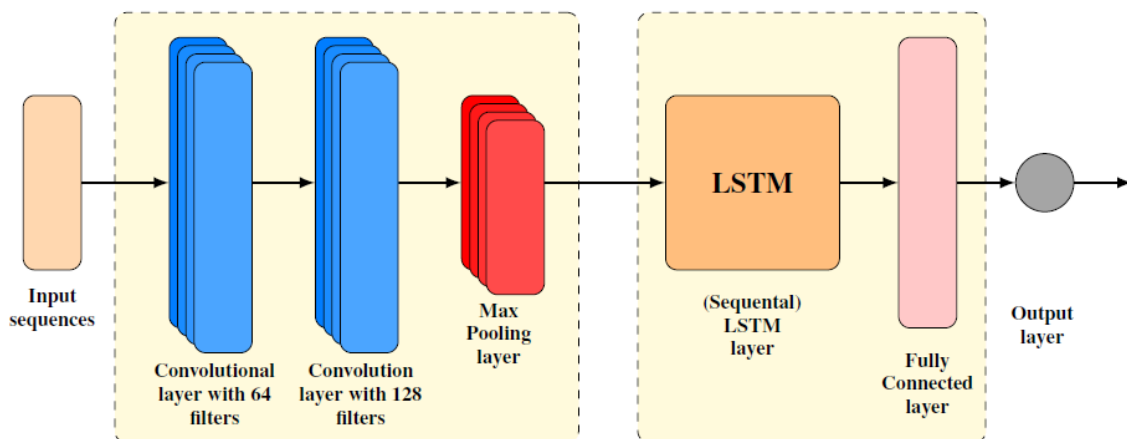


Figura 7. Modelo número dos propuesto por los autores con dos capas de convolución, una capa de agrupamiento, una capa LSTM y una capa densa completamente conectada como salida.

Fuente: Livieris, I.E., Pintelas, E. & Pintelas, P. A CNN–LSTM model for gold price time-series forecasting. *Neural Comput & Applic* 32, 17351–17360 (2020).

¹ Traducción realizada por el autor

El conjunto de datos utilizado para este trabajo fue el precio del oro entre enero de 2014 y abril de 2018. Los resultados se compararon con otros modelos, como SVR y LSTM, utilizando las siguientes métricas: MAE (Mean Absolute Error), RMSE (Root Mean Square Error), ACC (Accuracy), AUC (Area Under Curve), SEN (Sensitivity) y SPE (Specificity).

2.1.2.4 Stock Price Forecasting via Sentiment Analysis on Twitter

En este trabajo (Kordonis et al., 2016), se recopila un gran número de tweets de diferentes fechas relacionados con el precio de una determinada acción del mercado de valores. Luego, mediante técnicas de análisis de sentimientos y el uso de SVM (Support Vector Machine), se realiza una predicción del precio de la acción que se está analizando.

Como se muestra en la figura 8, el modelo utiliza dos APIs (Interfaz de Programación de Aplicaciones). La primera permite descargar tweets utilizando palabras clave, un intervalo de tiempo definido y el idioma en el que se escribió el mensaje. La segunda permite descargar desde Yahoo Finance los datos de la serie de tiempo para cierta acción del mercado o el valor de una divisa en tiempo real para su análisis.

Por otro lado, el módulo de SVM/Naive Bayes permite procesar palabra por palabra los mensajes de los tweets capturados anteriormente.

El módulo funciona de manera relativamente sencilla: primero, se realiza una limpieza del mensaje para eliminar caracteres especiales e información no relevante, como los "emojis". Luego, se puntúa cada mensaje en función de la emotividad que este expresa, utilizando palabras clave. Por ejemplo, si un mensaje contiene palabras como "mal", "odio" o "asco", se considera que tiene una puntuación de emotividad negativa. Por otro lado, si un mensaje utiliza palabras como "amor", "oportunidad", "excelente" o "bueno", se considera que tiene una emotividad positiva. Los conectores, como "para", "de", "el", "la", entre otros, son omitidos.

Al final, cada mensaje es ponderado y etiquetado como: positivo, negativo o neutro. Esta información se pone en una matriz, junto con la fecha del "tweet" y pasa a la siguiente fase que es la de procesamiento y análisis.

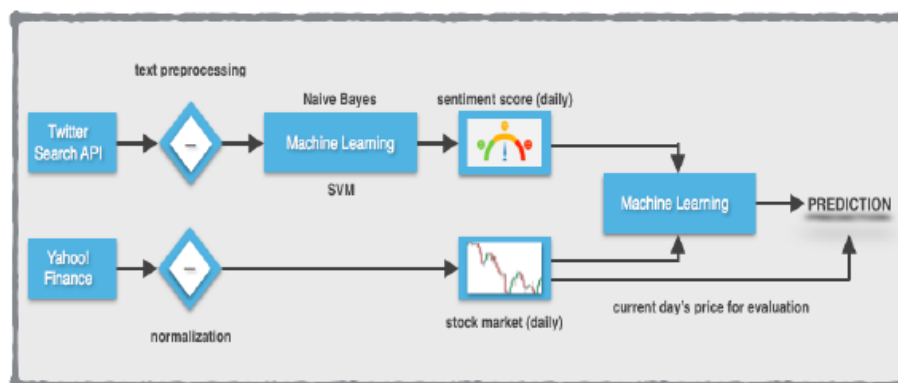


Figura 8. Modelo de análisis de sentimientos y predicción usando SVM

Fuente: John Kordonis, Symeon Symeonidis, and Avi Arampatzis. 2016. *Stock Price Forecasting via Sentiment Analysis on Twitter*. In *Proceedings of the 20th Pan-Hellenic Conference on Informatics (PCI '16)*. Association for Computing Machinery, New York, NY, USA, Article 36, 1–6.

Cuando se tiene la matriz de puntuación de sentimientos y la matriz de la serie de tiempo, se procede a unir ambas en una única matriz de características. Esta matriz de características se construye con los siguientes parámetros para cada uno de los días anteriores a la predicción: porcentaje de sentimientos positivos, porcentaje de sentimientos negativos, porcentaje de sentimientos neutros, precio de cierre de la acción/divisa, HLPCT (High-Low percentage), PCTchange (Percentage Change), volumen de la acción/Divisa.

El HLPCT y el PCTchange se calculan, usando la información de la serie de tiempo, con las siguientes fórmulas:

$$HLPCT = \frac{High - Low}{Low}$$

$$PCTchange = \frac{Close - Open}{Open}$$

Donde las variables High, Low, Open y Close, se refieren al precio más alto, más bajo, precio de apertura y precio de cierre de la acción/divisa a analizar, respectivamente.

Una vez construída la matriz de datos, se procede a realizar la predicción utilizando SVM, los resultados arrojados para la predicción de diferentes acciones fueron contrastados usando la medida ACC (Accuracy), la cual arrojó un porcentaje del 87%.

2.1.2.5 Price Forecasting for Agricultural Products Based on BP and RBF Neural Network

En este trabajo realizado por Yu, Shouhua y Ou, Jingying (2009) se hace uso de las redes neuronales basadas en funciones radiales (RBF) y la técnica de retropropagación (Backpropagation) para predecir el valor de ciertos productos agrícolas en el mercado chino entre enero y diciembre de 2011.

A diferencia del perceptrón de múltiples capas, que es como funcionan la mayoría de redes neuronales, las NN basadas en RBF (RBFNN) permiten realizar una clasificación de elementos utilizando lo que se denomina distancia euclidiana o distancia entre dos puntos. Tomando como referencia la figura 9, cuando llegan los datos de entrenamiento, cada neurona de procesamiento guarda una referencia o prototipo con el cual se pueda comparar más adelante. Luego, cuando llegan los datos a ser clasificados, se realiza una

comparación usando la distancia euclidiana entre el dato de entrada y el que se guardó previamente. Con esto, la red neuronal es capaz de clasificar si el dato de entrada está más cercano a una categoría o a otra, es decir, determina a cuál de los datos de entrenamiento está más cerca y así lleva a cabo la clasificación.

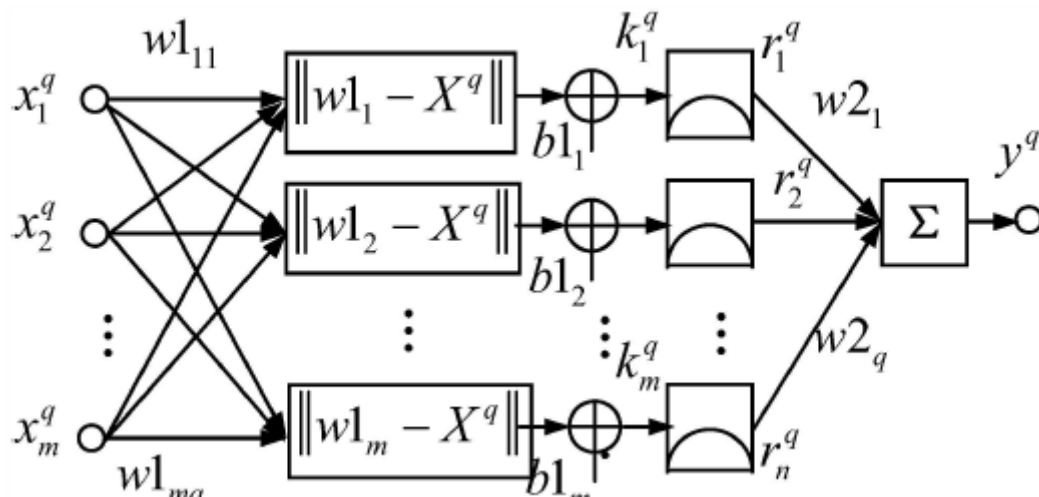


Figura 9. arquitectura y funcionamiento interno de una RBFNN

Fuente: Yu, Shouhua & Ou, Jingying. (2009). Forecasting Model of Agricultural Products Prices in Wholesale Markets Based on Combined BP Neural Network -Time Series Model. *International Conference on Information Management, Innovation Management and Industrial Engineering*. 1. 558-561. 10.1109/ICIII.2009.140.

El trabajo de cada neurona en esta red consiste en comparar el valor de entrada con el vector que se guardó previamente en la fase de entrenamiento y, según su cercanía, devolver un valor de salida entre 0 y 1. Cuanto más cercano sea el valor a 1, mayor será la similitud con la categoría de los datos de entrenamiento. A esta respuesta también se le denomina "salida de la neurona" o "respuesta de la función de activación".

Como se observa en la figura 9, este tipo de redes generalmente sólo consisten en una capa de procesamiento ($b_1, b_2, \dots, b_n$) y una capa de salida ($r_1, r_2, \dots, r_n$). Esta capa de salida está formada por neuronas que representan las diferentes categorías en las que queremos clasificar los datos. Su trabajo es computar y ordenar las salidas de la capa de procesamiento utilizando los pesos que arroja dicha capa.

Como se mencionó anteriormente, la función de activación de cada neurona busca calcular la similitud entre los datos de entrenamiento y los datos reales. Al ser una función radial, se hace uso de una función gaussiana como la que se muestra a continuación.

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Donde:

x: es el vector de entrada

μ : es la media

σ : es la desviación estándar

Dado que parte del proceso de las redes neuronales consiste en aprender los valores de los pesos para que la función de salida se ajuste a los valores esperados, el término de $\frac{1}{\sigma\sqrt{2\pi}}$ puede ser eliminado, ya que este principalmente controla la altura de la función gaussiana. Por otro lado, el parámetro $\frac{1}{2\sigma^2}$, controla la amplitud de la función, se puede reemplazar con un parámetro β que será optimizado por la misma red neuronal. De manera simple la ecuación gaussiana anterior, como función de activación de la red neuronal, se puede escribir de la siguiente manera.

$$\varphi(x) = e^{-\beta ||x-\mu||^2}$$

La función de coste es una medida que indica la discrepancia entre los valores predichos por el modelo y los valores reales de los datos de entrenamiento. Durante la fase de entrenamiento, el objetivo es minimizar esta función de coste ajustando los pesos y los sesgos de las neuronas. Al minimizar la función de coste, se mejora la precisión del modelo en la tarea para la que se ha entrenado la red neuronal.

En cuanto a la técnica de BP, su objetivo es minimizar la función de coste utilizando derivadas parciales para ajustar los valores de pesos y sesgos. La función de coste representa el error entre el valor real y el valor que se predice y se busca minimizarla lo máximo posible durante la fase de entrenamiento de una red neuronal. El MSE es una función de coste comúnmente utilizada en redes neuronales.

El gradiente descendente es una técnica utilizada para minimizar la función de coste, haciendo uso de derivadas parciales. De manera iterativa, se busca encontrar el mínimo global de la función de coste. Durante la propagación hacia atrás, el valor de la función de error que arroja cada neurona se ingresa de nuevo a la capa de entrada en cada iteración,

de tal manera que se pueda calcular la porción de error que cada neurona aporta. Después de varias iteraciones, se puede minimizar la función global del error y ajustar los valores de los pesos y sesgos.

Basados en estas dos técnicas, los autores (Yu, Shouhua & Ou, Jingying, 2009) realizaron una predicción del precio de diez productos agrícolas en el mercado chino: azúcar, carne de res, carne de cerdo, carne de cordero, huevos, arroz, aceite vegetal, aceite de soya, harina y aceite de cacahuete. La arquitectura de la red utilizada se muestra en la figura 9, y el conjunto de datos utilizado contenía los precios de cierre de los productos obtenidos del banco de agricultura de China. Cada vector de datos de entrenamiento que se ingresó a la red contenía el valor de tres semanas en el pasado.

La medida utilizada para comparar sus resultados fue el MAPE (Mean Absolute Percentage Error). Se realizó una comparativa entre el uso de RBF NN y BP + RBF NN. Como conclusión, los autores destacan que la precisión que puede alcanzar cada una de estas redes es del 98.7% y 99.6%, respectivamente. Se infiere que el método de BP mejora significativamente la eficacia del algoritmo en términos porcentuales.

2.2 MARCO CONCEPTUAL

El Machine Learning (ML) o aprendizaje automático busca que las máquinas aprendan mediante diferentes técnicas desarrolladas con este propósito. Según IBM, el "Machine learning" es una forma de la IA que permite a un sistema aprender de los datos en lugar de aprender mediante la programación explícita. Sin embargo, el machine learning no es un proceso sencillo. Conforme el algoritmo ingiere datos de entrenamiento, es posible producir modelos más precisos basados en esos datos. Es decir, el activo más importante en el aprendizaje automático es la fuente de datos con la que alimentamos el modelo.

Por otro lado, el mercado de divisas, también llamado "FOREX", es un espacio mundial descentralizado donde compradores de todo el mundo pueden negociar con divisas de diferentes países y la tasa de cambio es el principal indicador de negociación. Podemos definir la tasa de cambio como "el precio de una moneda expresado en términos de otra moneda". Es un indicador clave en la economía de cualquier país y es una medida de la salud económica de ese país. Las tasas de conversión tienen un gran impacto en la relación comercial entre países, lo que a su vez afecta el nivel de vida de la población en general (Pradeepkumar y Ravi, 2018).

En secciones anteriores se ha discutido sobre algunos métodos para la predicción del valor de una moneda en el mercado de divisas mediante el uso de modelos de aprendizaje automático. Como se mencionó previamente, estas técnicas suelen utilizar medidas estadísticas como el MAE, RMSE, MSE y MAPE para evaluar la precisión de los modelos.

El error absoluto (AE) se define como el valor absoluto de la diferencia entre dos variables continuas, y se utiliza para medir qué tan alejado está una medición del valor real. Por otro lado, el MAE (Error Absoluto Medio) es el valor promedio del error absoluto sobre una serie de muestras o valores medidos. Este indicador busca dar una medida de precisión en la técnica de medición o predicción utilizada, ya que indica cuánto se desvía en promedio la predicción del valor real..

En ML, el MSE es una medida que mide el promedio de los errores al cuadrado entre los valores predichos y los valores reales. Es decir, mide qué tan alejados están los valores predichos de los valores reales, en promedio. Por otro lado, el RMSE es simplemente la raíz cuadrada del MSE y proporciona una medida del error típico de las predicciones en la misma unidad que la variable de destino. Ambas medidas son útiles para evaluar la

precisión de un modelo en la predicción del valor de conversión de una moneda. (Vernier, 2018)

El MAPE se define como "El error porcentual absoluto medio (MAPE) es el promedio de los errores porcentuales absolutos de las predicciones en relación con los valores reales. El error porcentual absoluto se define como la diferencia absoluta entre el valor real y el valor predicho, dividido por el valor real, y se expresa como un porcentaje. El MAPE es una medida comúnmente utilizada para evaluar la precisión de los modelos de predicción en diferentes industrias y campos." (Encyclopedia of Production and Manufacturing Management [Springer], 2000).

3. METODOLOGÍA

Para alcanzar los objetivos propuestos, se proponen cuatro fases en este proyecto, una por cada objetivo específico. Cada fase ha de contribuir al objetivo general del proyecto.

FASE I

Esta primera fase tiene como objetivo seleccionar los algoritmos más utilizados y con mejores resultados basados en una revisión bibliográfica del estado del arte sobre modelos para la predicción de precios en el mercado de divisas. Esto permitirá establecer los insumos para el diseño de la aplicación, los valores de entrada y salida de los modelos, así como identificar las medidas estadísticas necesarias para comparar el desempeño de estos algoritmos.

Las actividades a desarrollar en esta fase son las siguientes:

1. Revisión bibliográfica del estado del arte sobre algoritmos de ML para la predicción en el mercado de divisas.
2. Selección y prueba de concepto de cinco de estos algoritmos de ML.
3. Según la literatura revisada (Raimundo et al., 2018; Paponpat et al., 2019; Livieris et al., 2020; Kordonis et al., 2016; Yu & Ou, 2009), los autores seleccionan las divisas para analizar de manera aleatoria, con el único criterio de que los datos estén en forma de serie de tiempo. Por lo tanto, en esta etapa inicial se realizarán comparaciones entre el rendimiento de estos algoritmos utilizando como caso de

estudio los pares de divisas USD/EUR, USD/JPY, USD/COP y EUR/COP. Estas divisas fueron seleccionadas de conformidad con los principales comportamientos de monedas que son de interés para el mercado colombiano, como lo plantean, Bittencourt, M. et al. (2021), Villamizar Villegas, M. (2016), Amaya, C. S. H., & López, A. E. G. (2015)., así como al histórico de precios disponible en forma de serie de tiempo.

FASE II

La segunda fase del proyecto tiene como objetivo establecer las métricas y criterios que se utilizarán para comparar el rendimiento de los diferentes modelos de ML para la predicción en el mercado de divisas. Esto implica definir las interfaces de salida de cada algoritmo y los mecanismos de almacenamiento y cálculo de errores en las predicciones, en línea con la literatura revisada en la sección 2.1. Dado que cada modelo tiene un comportamiento diferente, será necesario realizar un diseño preliminar del mecanismo de comunicación entre la aplicación y los modelos.

Las actividades a desarrollar en esta fase son:

1. Especificar a alto nivel el funcionamiento de la plataforma.
2. Definición de historias de usuario o requerimientos conductores.
3. Caracterización de actores del sistema.

FASE III

La tercera fase del proyecto se centrará en el diseño de la plataforma de software que permitirá la ejecución y comparación de los algoritmos seleccionados a nivel estadístico. En esta fase, se llevará a cabo el modelado de casos de uso, la definición de la arquitectura a utilizar, la selección de los componentes de software necesarios, la definición de las interfaces de comunicación, el diseño de la interfaz gráfica de usuario y la creación del mecanismo para el soporte de la ejecución de los algoritmos, los cuales podrán ser desarrollados utilizando múltiples frameworks.

Las actividades que se desarrollarán en esta etapa son:

1. Modelado de casos de uso basado en los requerimientos o historias de usuario para el desarrollo del sistema.

2. Establecer la arquitectura del sistema a utilizar basado en una revisión de trabajos previos y arquitecturas de referencia para este tipo de sistemas.
3. Definir los componentes del sistema y sus interfaces de comunicación.
4. Diseño de los módulos del sistema.
5. Diseñar las interfaces gráficas de usuario de la plataforma

FASE IV

La cuarta y última fase del proyecto tiene como objetivo la implementación de un prototipo de la solución diseñada que permita la ejecución de los cinco algoritmos seleccionados y su comparación mediante las métricas definidas en la fase anterior. Para el desarrollo de software, se utilizará la metodología Kanban debido a sus características de flexibilidad y adaptabilidad a los cambios. El prototipo será probado con datos históricos y se evaluará su desempeño en términos de precisión en la predicción de precios en el mercado de divisas.

Las actividades a desarrollar en esta fase son:

1. Establecer las tareas a ejecutar en un tablero Kanban, especificando prioridades y tiempo de ejecución.
2. Desarrollo del prototipo que permita la posterior verificación del desempeño de los algoritmos ejecutados.

4. COMPARATIVA DE RENDIMIENTO DE ALGORITMOS SELECCIONADOS

De acuerdo con el estudio realizado por (Botchkarev, Alexei, 2019), las métricas de rendimiento o mediciones de error, son utilizadas para comparar las predicciones realizadas por el modelo entrenado con los datos reales u observados.

Dada la naturaleza de los algoritmos de Machine Learning (ML) utilizados para predicción, es necesario realizar la comparación del rendimiento mediante la medición del error. A continuación, se presentan las métricas seleccionadas.

4.1 SELECCIÓN DE MÉTRICAS DE COMPARACIÓN

Basándonos en la revisión de algoritmos presentada en la sección 2.1.2 y tomando en cuenta el marco conceptual, se han seleccionado tres métricas principales para la comparación de los cinco algoritmos seleccionados: el MAE (Mean Absolute Error), el RMSE (Root Mean Square Error) y el MAPE (Mean Absolute Percentage Error).

Se decidió utilizar estas tres medidas porque, según la revisión de literatura descrita en la sección 2.1.2, son las más utilizadas para presentar los resultados y permiten determinar el grado de error de cada algoritmo en la predicción. Además, autores como Alexei Botchkarev (2019) destacan que estas métricas han sido las más populares en los últimos 25 años. En la siguiente tabla se presenta un resumen de las encuestas realizadas a múltiples investigadores sobre el uso de estas métricas.

Table 1. Top three metrics identified in the surveys, percentage

Metrics	C&A, 1982	M&K, 1995	M et al, 2006	F&G, 2007
Mean square error (MSE) or Root MSE (RMSE)	34	10	6	9
Mean absolute error (MAE)	18	25	20	36
Mean absolute percentage error (MAPE)	15	52	45	44

Tabla tomada de: Alexei Botchkarev. 2019. *Performance Metrics (Error Measures) in Machine Learning Regression, Forecasting and Prognostics: Properties and Typology.* *interdisciplinary Journal of Information, Knowledge, and Management*, 14, 45-79

A continuación, se presenta la formulación matemática de cada una de estas medidas.

4.1.1 MAE

Como se mencionó anteriormente, el MAE busca dar una medida de qué tan alejado está el dato calculado del dato observado. En otras palabras, qué tan alejada estaría la predicción de la realidad. Su fórmula está dada por:

$$MAE = \sum_{i=1}^n \frac{|y_i - x_i|}{n}$$

Donde: y_i , es el valor de la predicción y x_i es el valor real u observado.

4.1.2 RMSE

La Raíz del Error Cuadrático Medio (RMSE, por sus siglas en inglés), al igual que el MAE, permite medir qué tan alejados están los valores predichos de los valores observados o reales. La principal diferencia entre estas dos métricas es que el RMSE nos muestra la diferencia al cuadrado entre el valor estimado y el valor observado.

En palabras simples, el RMSE nos indica qué tan acertado es un modelo representando el fenómeno de la realidad. Su formulación matemática nos dice qué tan desviados están los datos que se predicen de los datos reales.

$$RMSE = \sqrt{\sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{n}}$$

Donde n es el número de observaciones. $\hat{y}_i$, es el valor predicho en el instante i . y_i , es el valor real en el instante i .

El MSE por sí sólo penaliza los errores que son más grandes en el modelo. Por otro lado, el RMSE, al tomar la raíz cuadrada del MSE, permite tener una mejor perspectiva en términos de puntaje cuando existen algunos errores muy grandes pero, que a nivel general no deberían impactar el rendimiento del modelo.

4.1.3 MAPE

El Error Porcentual Absoluto Medio (MAPE, por sus siglas en inglés), es una medida que nos permite expresar en términos porcentuales el error absoluto. Nos da una estimación de la precisión del método de predicción que estamos utilizando.

Es más intuitivo de entender que el error absoluto ya que el error es expresado en términos porcentuales para una serie de tiempo dada. Por ejemplo, si tenemos un valor del MAPE del 5%, se interpreta como que la diferencia entre el valor que se predice y el valor real es aproximadamente del 5%. Su formulación es simple y se expresa a continuación.

$$MAPE = \frac{100\%}{n} \sum_{t=1}^n \left| \frac{A_t - F_t}{A_t} \right|$$

Donde: n es el número de mediciones. A_t , es real. F_t , es el valor predicho por el algoritmo.

4.2 CASO DE ESTUDIO APLICADO Y RESULTADOS DE LA COMPARACIÓN

En los artículos revisados en la sección 2.1.2, se presentan casos de estudio que se enfocan en mejorar la precisión de las técnicas de aprendizaje automático para predecir los precios de las divisas y/o acciones en el mercado financiero. La evaluación del rendimiento de cada algoritmo propuesto por los autores se realiza mediante la medición de errores utilizando diversas métricas, entre las cuales destacan el MAE, RMSE y MAPE, principalmente. La sección 4.1 describe en detalle estas métricas.

Dado que todos los métodos presentados en los artículos mencionados en la sección 2 utilizan series de tiempo como entrada para los algoritmos de aprendizaje automático, se seleccionaron series de tiempo del mercado FOREX para comparar la precisión de las predicciones de cada método mediante la medición del error, utilizando las métricas mencionadas anteriormente.

Siguiendo la primera fase de la metodología planteada, se replicaron los resultados de los cinco algoritmos seleccionados en la sección 2.1.2 utilizando el lenguaje de programación Python; específicamente, utilizando Jupyter Notebooks como herramienta.

Una vez obtenidos los algoritmos, se procedió a comparar su rendimiento utilizando las tres métricas mencionadas anteriormente (MAE, RMSE y MAPE), las cuales, como señala Botchkarev (2019), permiten comprender qué tan distantes están las estimaciones de los valores observados, y han sido las medidas estadísticas más utilizadas desde 1982. A continuación, se presentan los resultados de cada algoritmo para las siguientes parejas de divisas: USD/EUR, USD/JPY, USD/COP y EUR/COP. Los datos evaluados corresponden al periodo desde el 1 de enero de 2022 hasta el 1 de agosto del mismo año (01/01/2022 - 01/08/2022).

El porcentaje de datos de entrenamiento fue del 75%, dejando como datos de prueba el 25% restante. Es decir, se utilizó la serie de tiempo desde el 01/01/2022 hasta el 08/06/2022 como información de entrenamiento para cada modelo, y se utilizó la serie de tiempo restante, desde el 09/06/2022 hasta el 01/08/2022, como datos de prueba para medir la precisión de cada algoritmo, de conformidad con los conjuntos de datos expuestos en <https://finance.yahoo.com/> una ejemplificación de estos datos se muestra en la siguiente tabla para la conversión de Euro (EUR) a peso colombiano (COP). En ella se muestran los precios de apertura, precio más alto, más bajo y precio de cierre.

	Open	High	Low	Close	Adj Close	Volume
Date						
2022-01-03	4557.200195	4571.700195	4557.200195	4557.200195	4557.200195	0
2022-01-04	4571.700195	4571.700195	4539.399902	4571.700195	4571.700195	0
2022-01-05	4539.399902	4568.399902	4539.399902	4539.399902	4539.399902	0
2022-01-06	4568.399902	4568.399902	4504.100098	4568.399902	4568.399902	0
2022-01-07	4504.100098	4510.799805	4504.100098	4504.100098	4504.100098	0
...	...	...	...	...	...	...
2022-07-26	4519.600098	4519.600098	4485.399902	4519.600098	4519.600098	0
2022-07-27	4485.399902	4485.399902	4462.200195	4485.399902	4485.399902	0
2022-07-28	4462.200195	4462.200195	4434.200195	4462.200195	4462.200195	0
2022-07-29	4434.200195	4434.200195	4426.399902	4434.200195	4434.200195	0
2022-08-01	4426.399902	4426.399902	4345.000000	4426.399902	4426.399902	0
151 rows x 6 columns						

La siguiente sección presentará los resultados de cada algoritmo de conformidad con el caso de estudio definido para cada par de monedas durante el periodo establecido anteriormente. Al final del capítulo se presentará una tabla comparativa que resume el rendimiento de cada algoritmo.

4.2.1 RENDIMIENTO: SVR-WAVELET ADAPTIVE MODEL

Este algoritmo, como se describió anteriormente, pretende descomponer la serie de tiempo en componentes más pequeños para remover el ruido y, luego, tomar la componente más limpia y realizar la predicción usando SVR.

USD/EUR

El valor promedio del periodo seleccionado de los datos de entrenamiento, para la conversión de dólares americanos a euros, fue de 0.924 EUR por cada USD.

Realizando la predicción, utilizando los datos de prueba, se encuentran las siguientes mediciones.

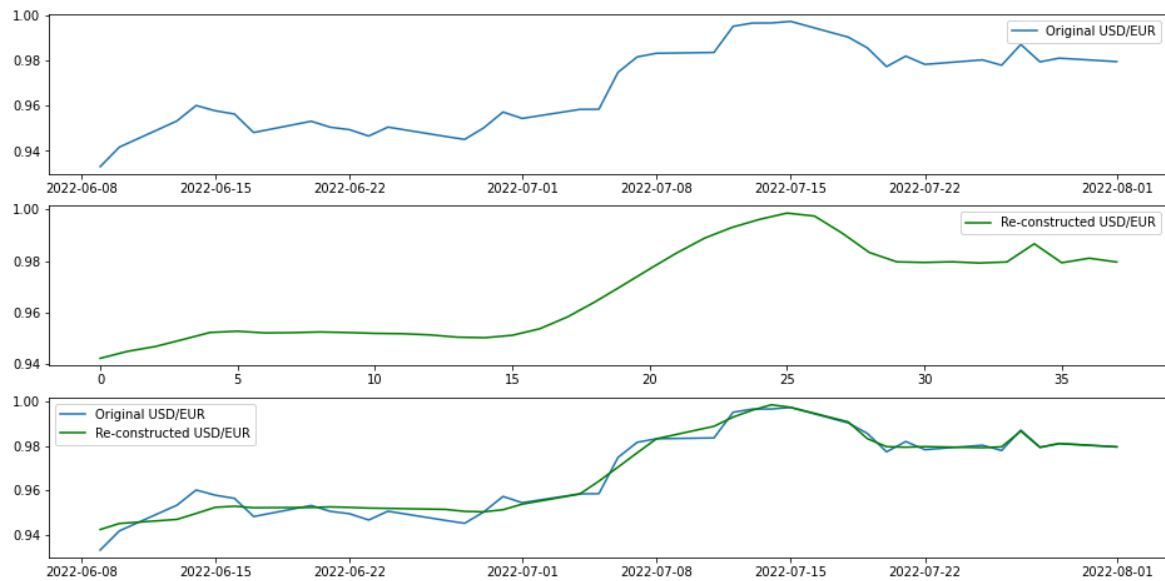


Figura 10. Gráfico de comparación entre los valores reales y los valores estimados para USD/EUR utilizando el algoritmo de SVR+Wavelets. En el eje Y se encuentra el valor de la moneda en una escala de 0 a 1, mientras que en el eje X se representa la fecha en la que se registró dicho valor.

MAE: 0.0029

RMSE: 0.0039

MAPE: 0.30%

USD/JPY

El valor promedio del periodo seleccionado para los datos de entrenamiento, para la conversión de dólares americanos a Yenes japoneses, fue de 125.00 YEN por cada 1 USD.

Realizando la predicción, utilizando los datos de prueba, se encuentran las siguientes mediciones.

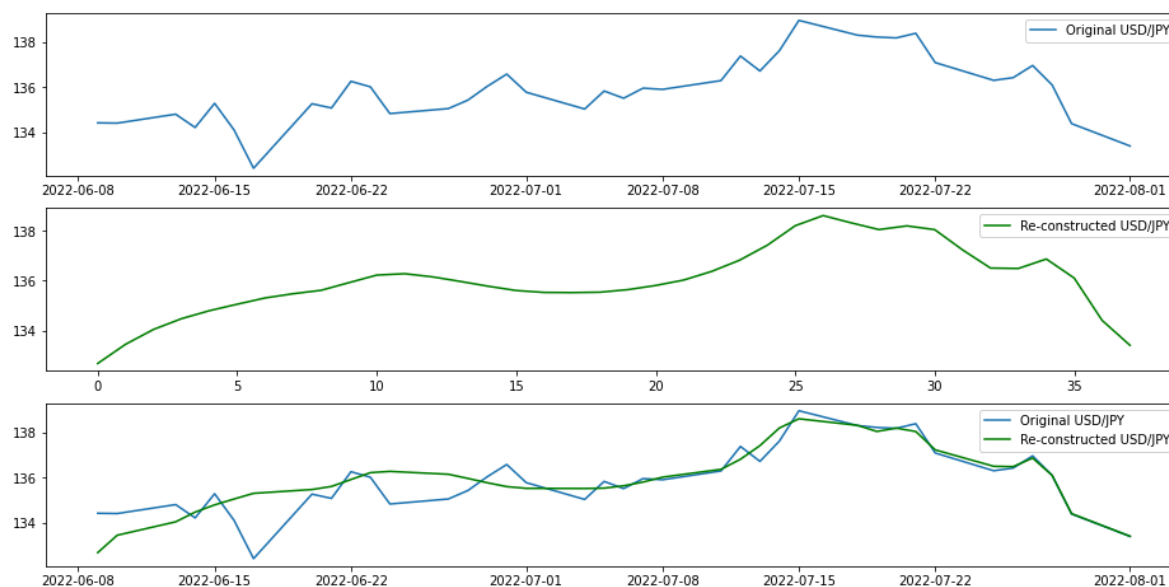


Figura 11. Gráfico de valor real vs valor estimado para USD/JPY usando algoritmo de SVR+Wavelets.

MAE: 0.485

RMSE: 0.746

MAPE: 0.35%

USD/COP

El valor promedio del periodo seleccionado para los datos de entrenamiento, para la conversión de dólares americanos a pesos colombianos, fue de 3902.41 COP por cada USD.

Realizando la predicción, utilizando los datos de prueba, se encuentran las siguientes mediciones.

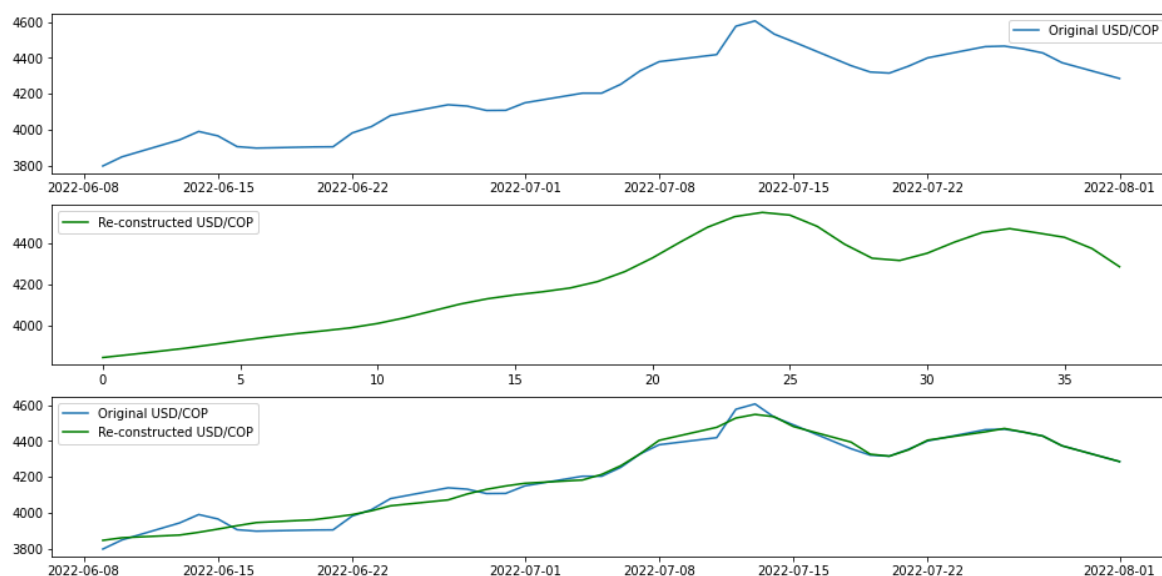


Figura 12. Gráfico de valor real vs valor estimado para USD/COP usando algoritmo de SVR+Wavelets.

MAE: 26.77

RMSE:37.17

MAPE: 0.65%

EUR/COP

El valor promedio del periodo seleccionado para los datos de entrenamiento, para la conversión de euros a pesos colombianos, fue de 4258.21 COP por cada EUR.

Realizando la predicción, utilizando los datos de prueba, se encuentran las siguientes mediciones.

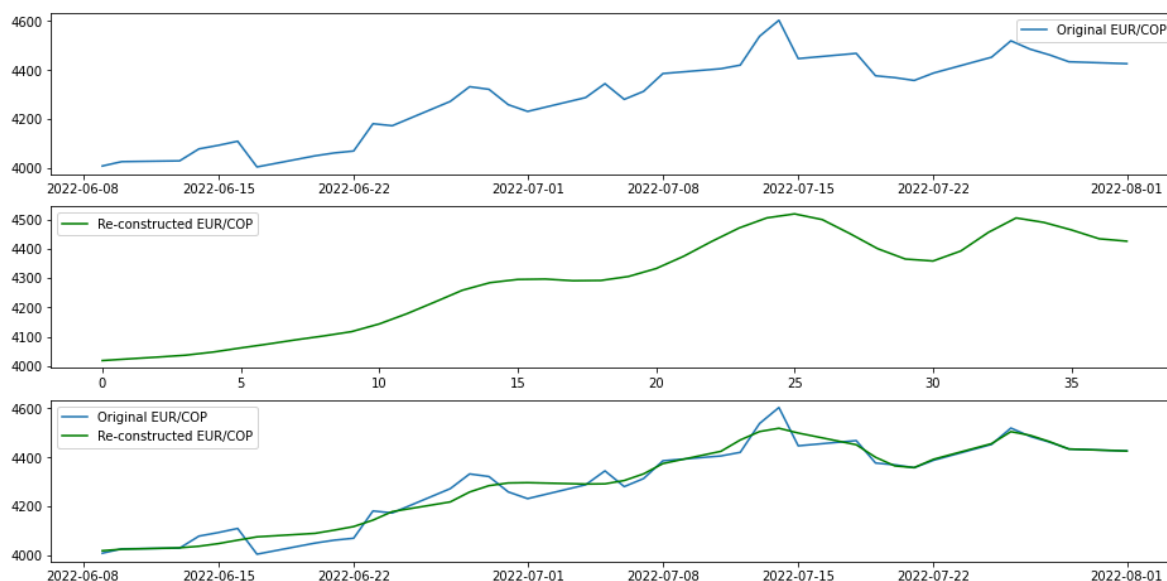


Figura 13. Gráfico de valor real vs valor estimado para EUR/COP usando algoritmo de SVR+Wavelets.

MAE: 28.39

RMSE:37.07

MAPE: 0.67%

TABLA DE RESUMEN DE RESULTADOS

TABLA COMPARATIVA SVR-WAVELET				
	USD/EUR	USD/JPY	USD/COP	EUR/COP
MAE	0.0029	0.4850	26.7700	28.3900
RMSE	0.0039	0.7460	37.1700	37.0700
MAPE	0.30%	0.35%	0.65%	0.67%

Como se puede observar en la tabla anterior, este algoritmo presentó el menor porcentaje de error (MAPE) al realizar la predicción de la tasa de cambio de dólares estadounidenses (USD) a euros (EUR). En cuanto al peor resultado, se presentó al realizar la predicción de la tasa de cambio de euros (EUR) a pesos colombianos (COP) con un error del 0.67%.

4.2.2 RENDIMIENTO: BCFAR MODEL

Este algoritmo, descrito anteriormente, actúa cuando hay muchas variables dentro del sistema a predecir y se hace computacionalmente difícil encontrar la relación entre ellas. La lógica detrás del algoritmo es generar matrices de compresión de manera aleatoria y luego aplicar el modelo VAR para predecir el valor de la moneda "h" días a futuro, utilizando dichas matrices junto con las variables de entrada.

En este caso particular, para predecir el valor de USD/EUR, USD/JPY, USD/COP y EUR/COP, se incluyeron otras monedas en la matriz de entrada con el fin de determinar su posible influencia en el valor de la moneda objetivo. Estas monedas adicionales son USD/CAD, USD/AUD, EUR/AUD, EUR/CAD, CAD/JPY, EUR/JPY y EUR/MXN, que corresponden al dólar canadiense, el dólar australiano, el euro australiano, el euro canadiense, el dólar canadiense frente al yen japonés, el euro frente al yen japonés y el euro frente al peso mexicano, respectivamente.

La predicción se hizo con base a criterios definidos por $h=1, 4$ y 12 días a futuro, esto basados en criterios definidos en modelo por T. Paponpat et al. (2019).

Resultados para un día a futuro

Realizando la estimación con 1 día a futuro ($h=1$), se encuentran los siguientes resultados:

TABLA COMPARATIVA BCFVAR: $h=1$				
	USD/EUR	USD/JPY	USD/COP	EUR/COP
MAE	0.0007	1.8674	82.0599	11.0077
RMSE	0.0007	1.8674	82.0599	11.0077
MAPE	0.07%	1.40%	1.91%	0.25%

Resultados para cuatro días a futuro

Realizando la estimación con 4 días a futuro ($h=4$), se encuentran los siguientes resultados:

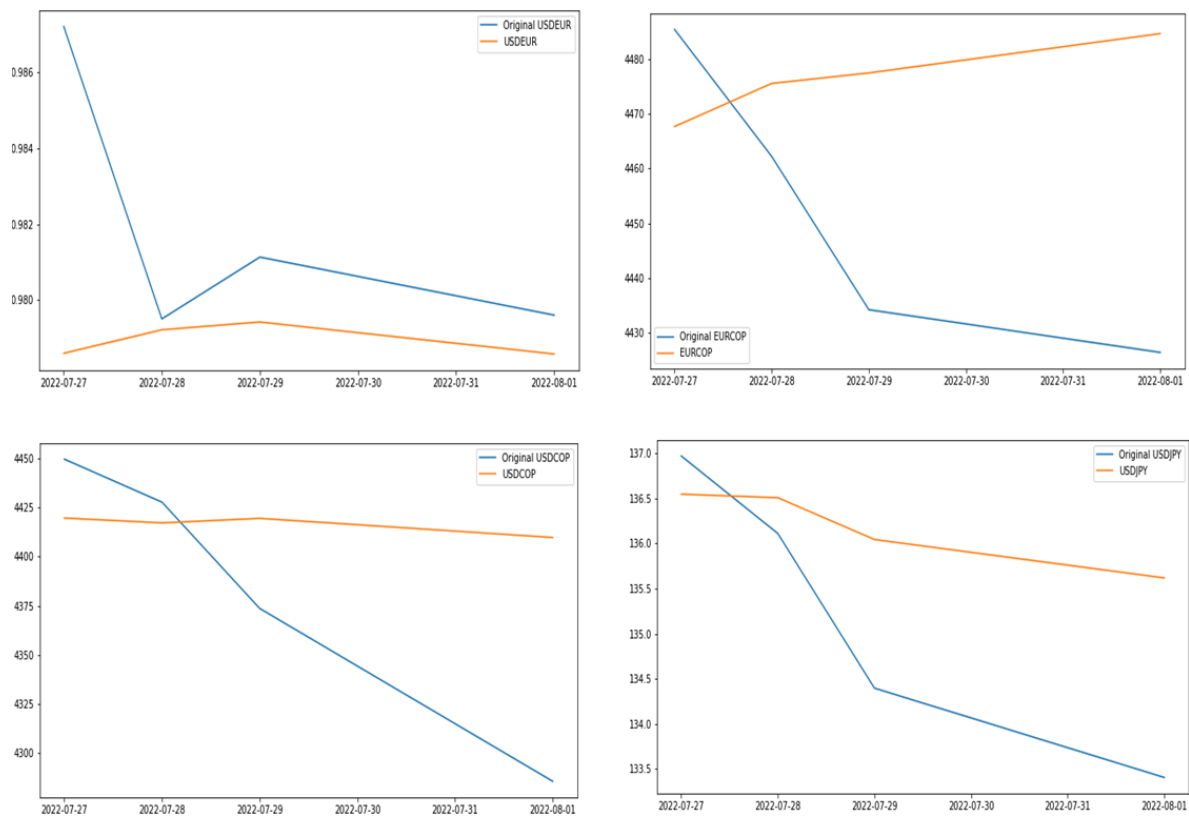


Figura 14. Gráfico de valor real vs valor estimado para USD/EUR, USD/JPY, USD/COP y EUR/COP usando algoritmo de BCVAR con $h=4$ (días a futuro).

TABLA COMPARATIVA BCVAR: $h=4$				
	USD/EUR	USD/JPY	USD/COP	EUR/COP
MAE	0.0029	1.1696	52.8267	33.1416
RMSE	0.0044	1.4089	68.2996	37.9313
MAPE	0.30%	0.87%	1.22%	0.75%

Resultados para doce días a futuro

Realizando la estimación con 12 días a futuro ($h=12$), se encuentran los siguientes resultados:

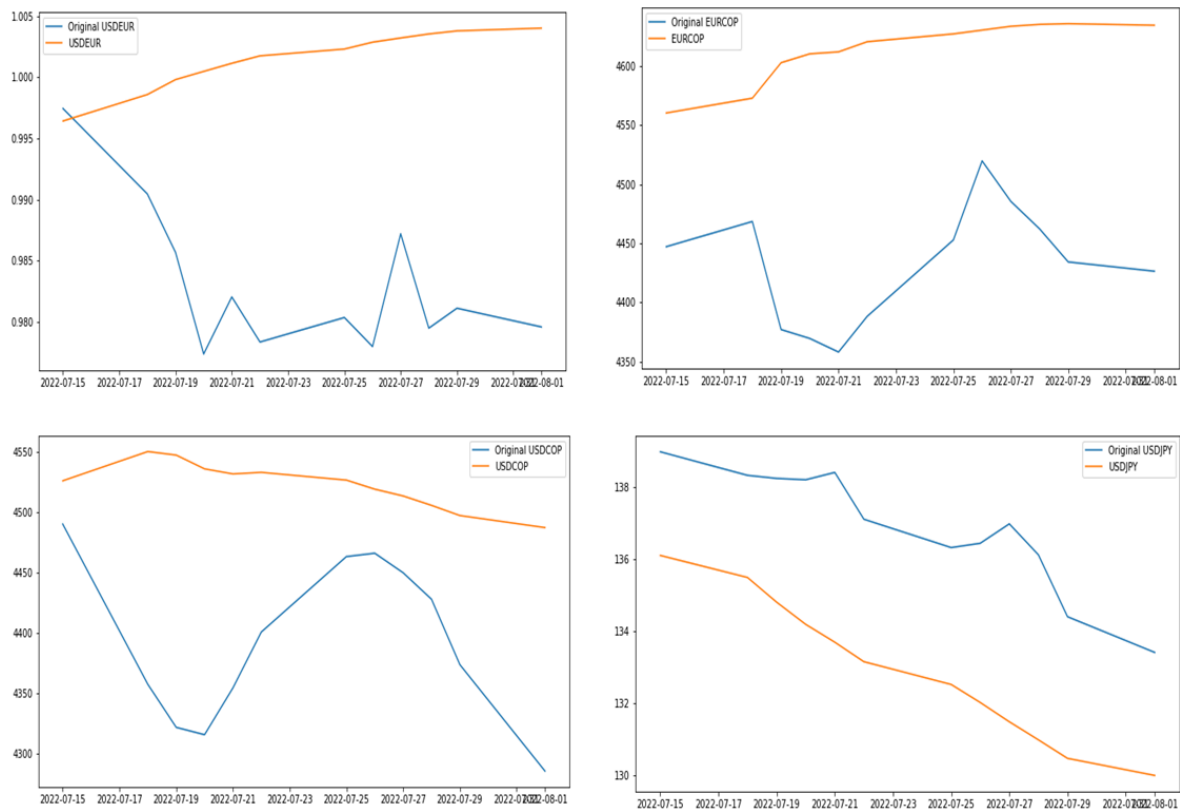


Figura 15. Gráfico de valor real vs valor estimado para USD/EUR, USD/JPY, USD/COP y EUR/COP usando algoritmo de BCVAR con $h=12$ (días a futuro).

TABLA COMPARATIVA BCVAR: $h=12$				
	USD/EUR	USD/JPY	USD/COP	EUR/COP
MAE	0.0186	3.9948	130.8464	181.9048
RMSE	0.0199	4.0724	147.3817	188.9187
MAPE	1.89%	2.92%	3.00%	4.12%

Como se puede observar, con este algoritmo a medida que aumentan la cantidad de días a estimar, desciende la precisión del mismo.

4.2.3 RENDIMIENTO: CNN-LSTM MODEL

Este modelo, como se mostró anteriormente, pretende combinar redes neuronales convolucionales con redes neuronales de memoria a corto-largo plazo.

Una vez se replicó la arquitectura de la figura 7, se realizó el mismo ejercicio del punto 4.2.1, con el mismo conjunto de divisas. Los resultados se muestran a continuación.

USD/EUR

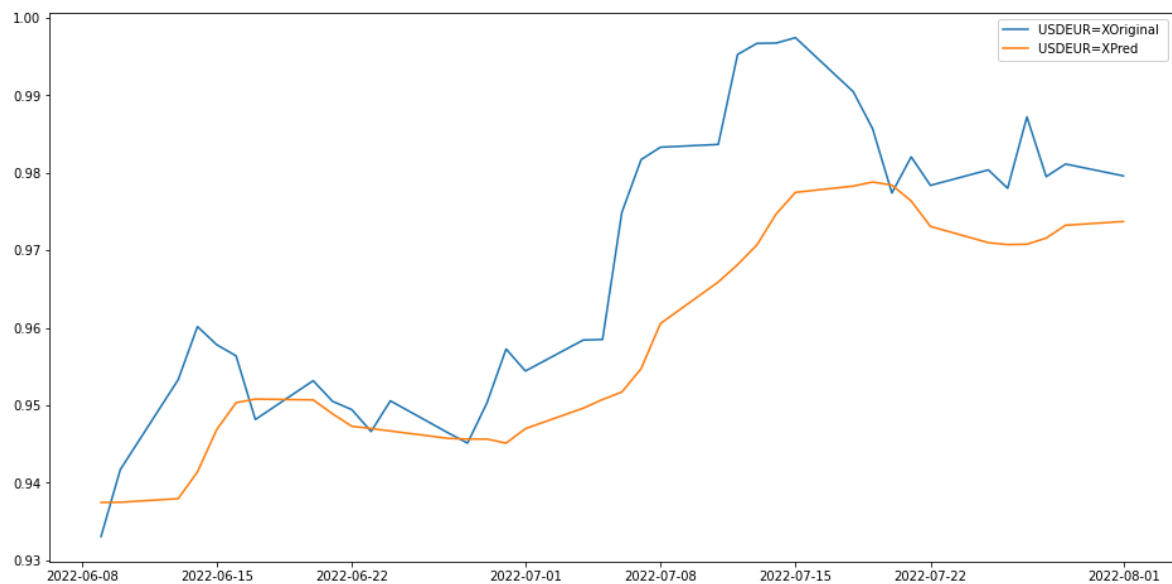


Figura 16. Gráfico de valor real vs valor estimado para USD/EUR usando algoritmo de CNN+LSTM.

MAE: 0.0101

RMSE:0.0130

MAPE: 1.04%

USD/JPY

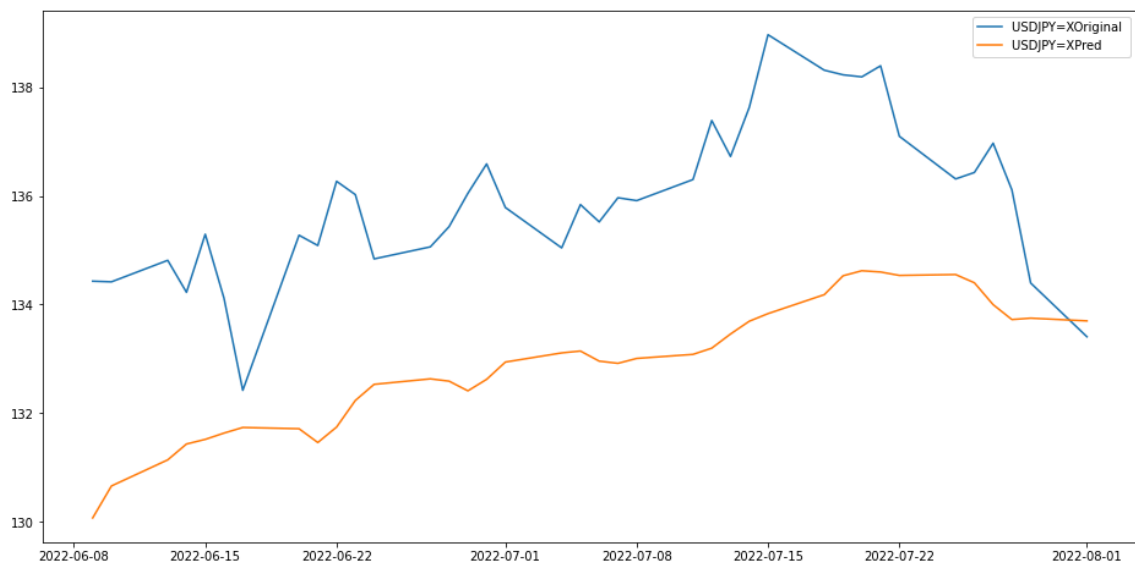


Figura 17. Gráfico de valor real vs valor estimado para USD/JPY usando algoritmo de CNN+LSTM.

MAE: 3.0473

RMSE: 3.2234

MAPE: 2.24%

USD/COP

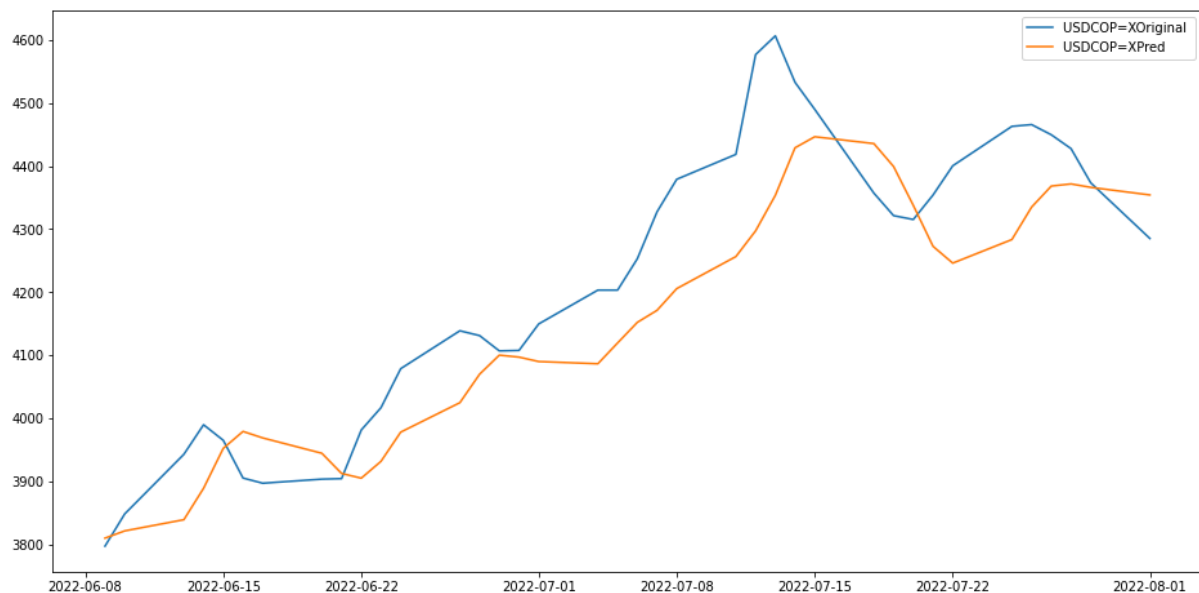


Figura 18. Gráfico de valor real vs valor estimado para USD/COP usando algoritmo de CNN+LSTM.

MAE: 88.9647

RMSE: 109.1022

MAPE: 2.07%

EUR/COP

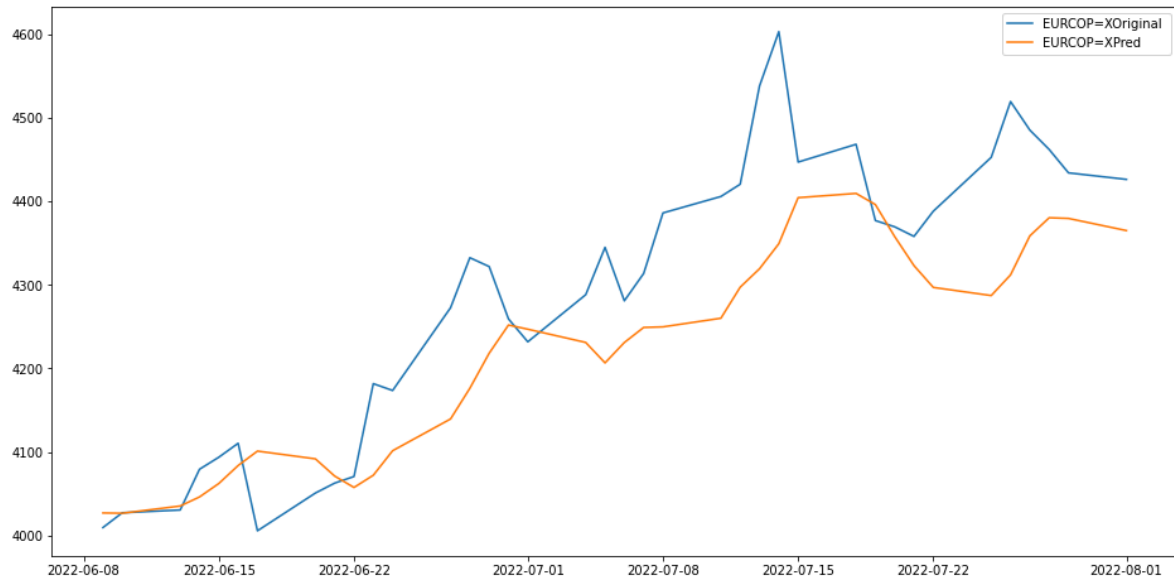


Figura 19. Gráfico de valor real vs valor estimado para EUR/COP usando algoritmo de CNN+LSTM.

MAE: 79.3030

RMSE: 102.1296

MAPE: 1.81%

4.2.4 RENDIMIENTO: ANÁLISIS DE SENTIMIENTOS - TWITTER

Este algoritmo tiene como objetivo construir una matriz de entrada utilizando los precios de cierre de las divisas y, además, emplear análisis de sentimientos para obtener el puntaje de sentimientos positivos, negativos y neutros de las interacciones en Twitter relacionadas con una palabra clave específica. Una vez que se replicó el algoritmo, se procedió a utilizarlo con las diferentes monedas y obtener su rendimiento en términos de precisión, al igual que para los algoritmos anteriores.

USD/EUR

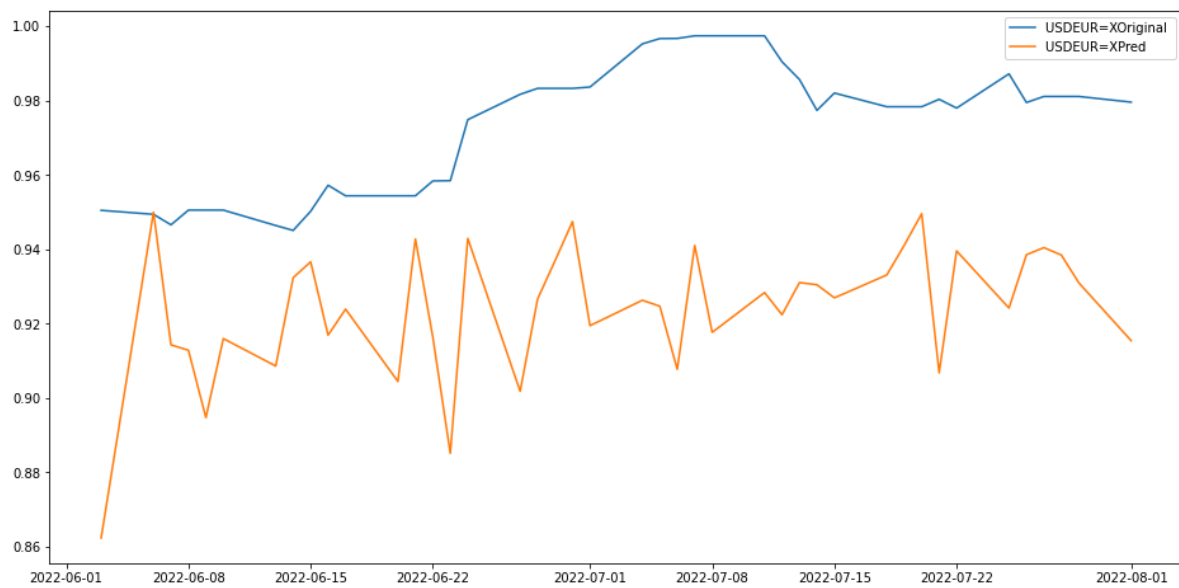


Figura 20. Gráfico de valor real vs valor estimado para USD/EUR usando algoritmo de análisis de sentimientos con información de Twitter.

MAE: 0.04905

RMSE: 0.0532

MAPE: 5.02%

USD/JPY

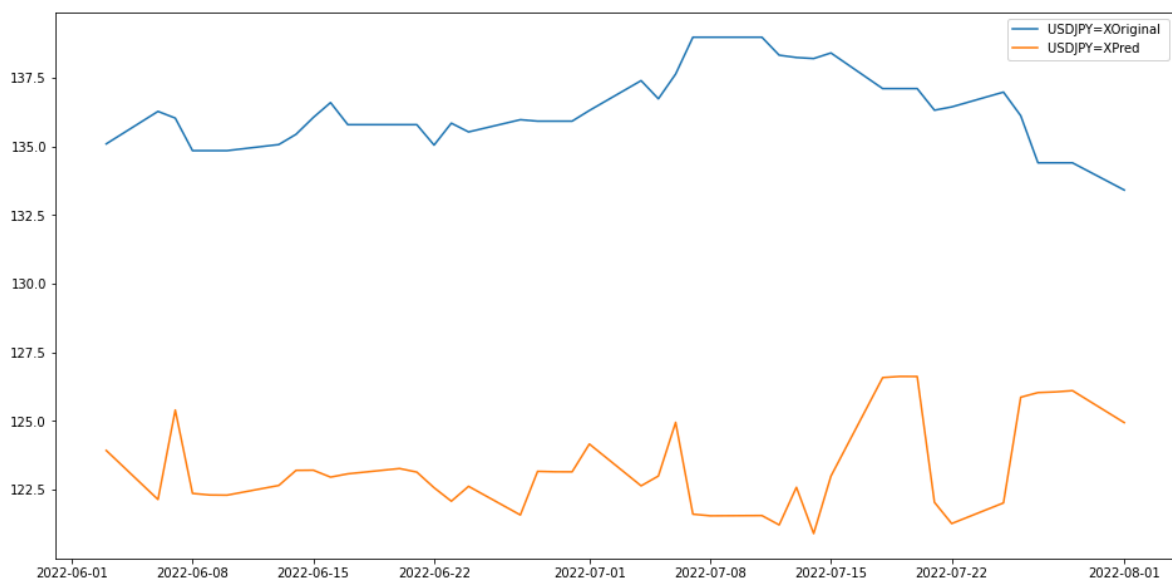


Figura 21. Gráfico de valor real vs valor estimado para USD/JPY usando algoritmo de análisis de sentimientos con información de Twitter.

MAE: 12.9648

RMSE: 13.1910

MAPE: 9.50%

USD/COP

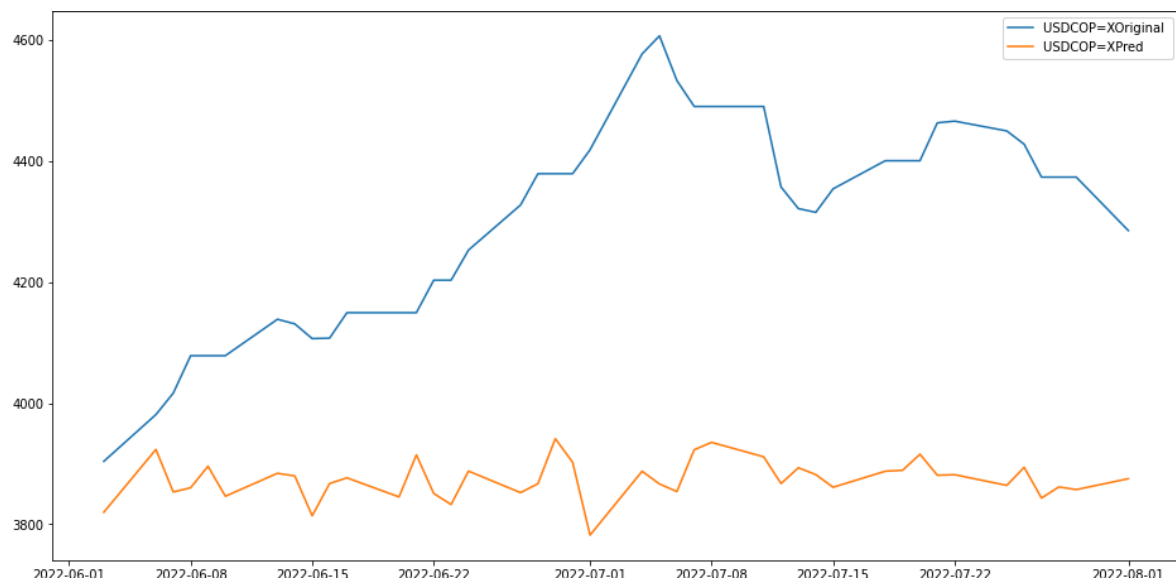


Figura 22. Gráfico de valor real vs valor estimado para USD/COP usando algoritmo de análisis de sentimientos con información de Twitter.

MAE: 424.5500

RMSE: 455.6473

MAPE: 9.73%

EUR/COP

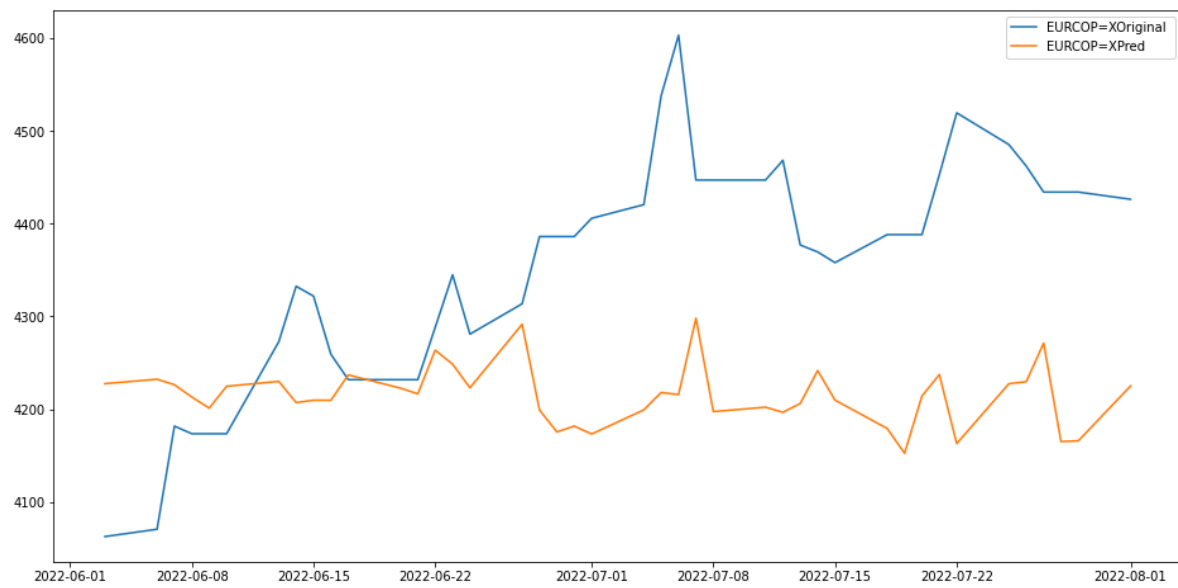


Figura 23. Gráfico de valor real vs valor estimado para EUR/COP usando algoritmo de análisis de sentimientos con información de Twitter.

MAE: 160.9246

RMSE: 189.2871

MAPE: 3.65%

4.2.5 RENDIMIENTO: RED NEURONAL - RBF/BP

La lógica detrás de utilizar una red neuronal con RBF (Radial Basis Function) y BP (Back Propagation) es encontrar los conjuntos de datos que mejor se ajustan al conjunto de características clasificadas en la fase de entrenamiento.

Al igual que los otros algoritmos, se repite el proceso para los cuatro pares de monedas mencionados anteriormente. A continuación se muestran los resultados.

USD/EUR

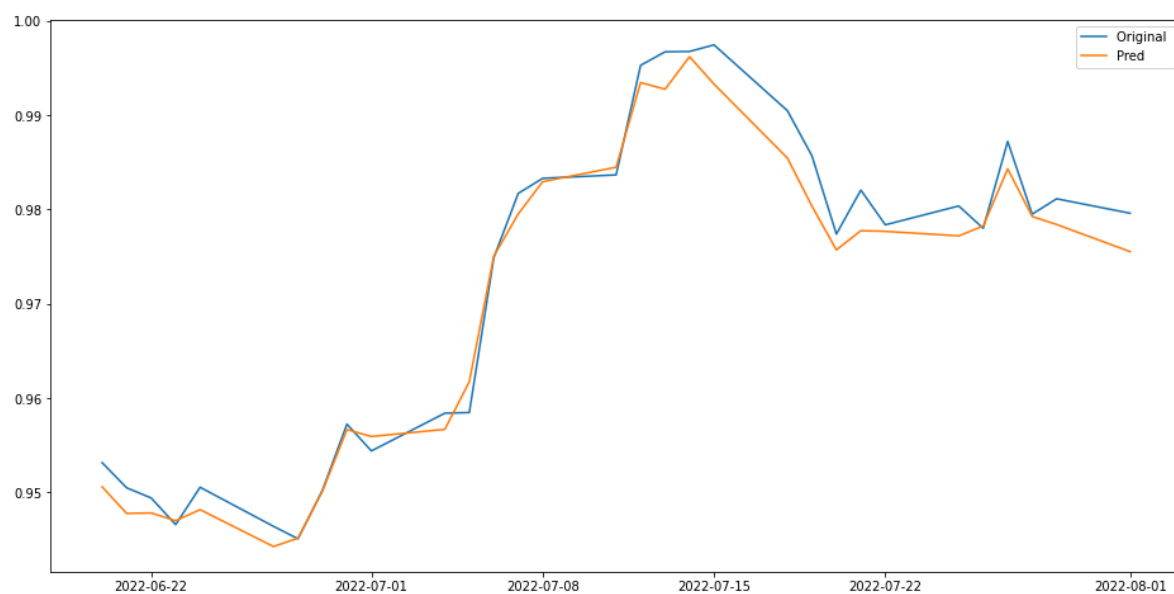


Figura 24. Gráfico de valor real vs valor estimado para USD/EUR usando algoritmo de red neuronal con RBF + BP.

MAE: 0.0020

RMSE: 0.0026

MAPE: 0.21%

USD/JPY

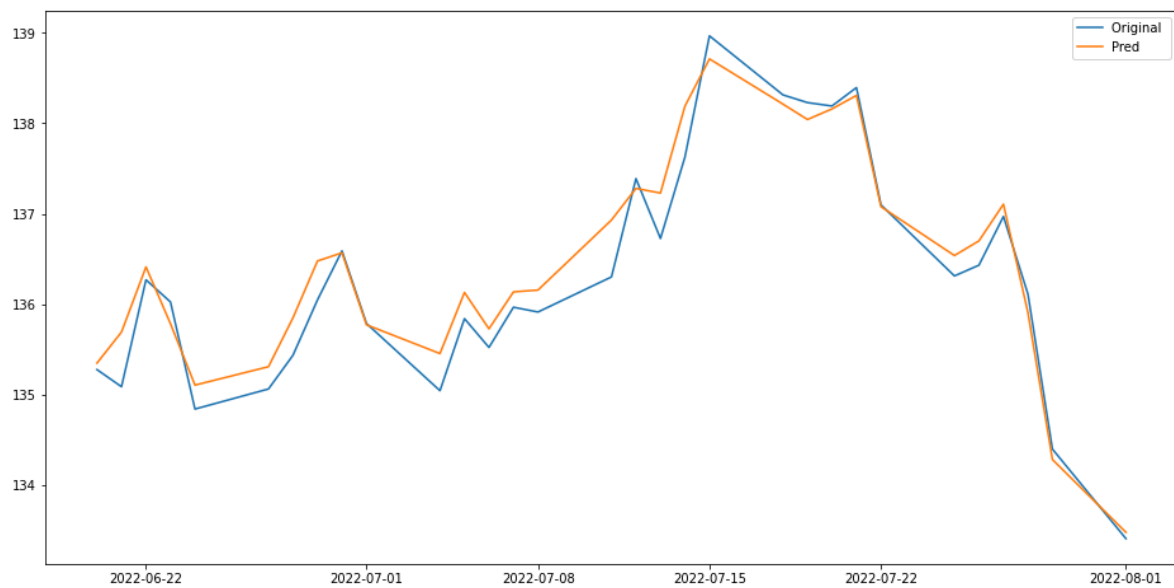


Figura 25. Gráfico de valor real vs valor estimado para USD/JPY usando algoritmo de red neuronal con RBF + BP.

MAE: 0.2347

RMSE: 0.2905

MAPE: 0.17%

USD/COP

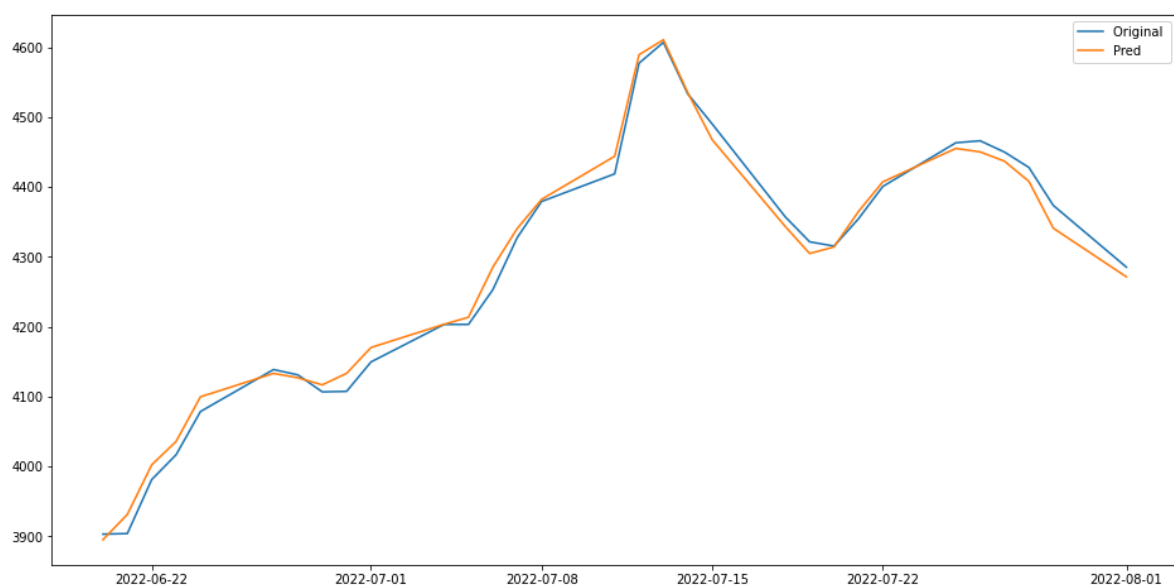


Figura 26. Gráfico de valor real vs valor estimado para USD/COP usando algoritmo de red neuronal con RBF + BP.

MAE: 14.1582

RMSE: 16.7518

MAPE: 0.33%

EUR/COP

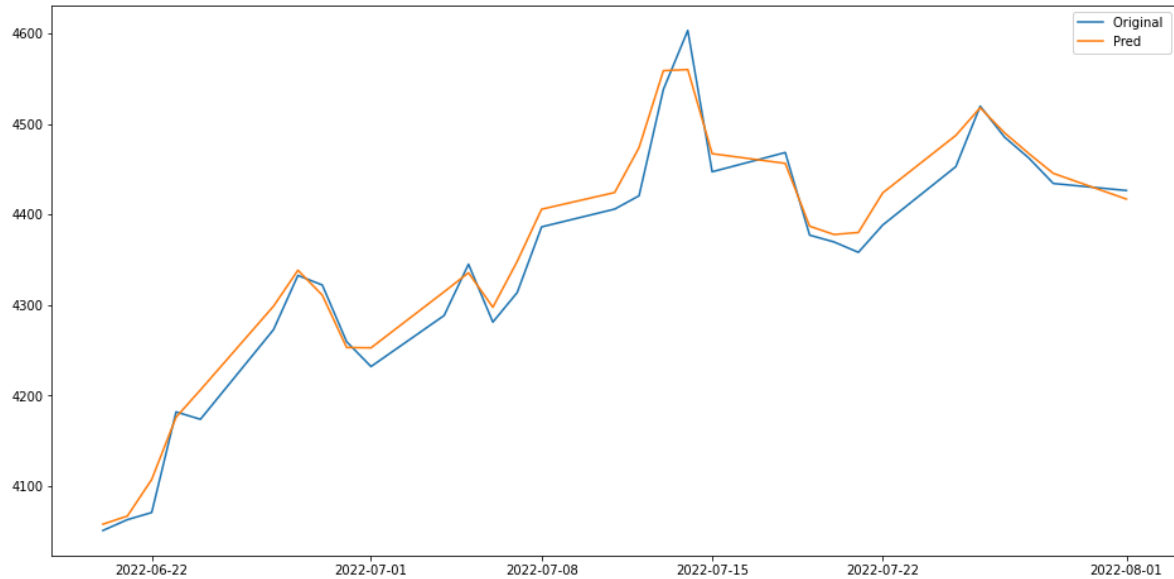


Figura 27. Gráfico de valor real vs valor estimado para EUR/COP usando algoritmo de red neuronal con RBF + BP.

MAE: 18.4165

RMSE: 22.5500

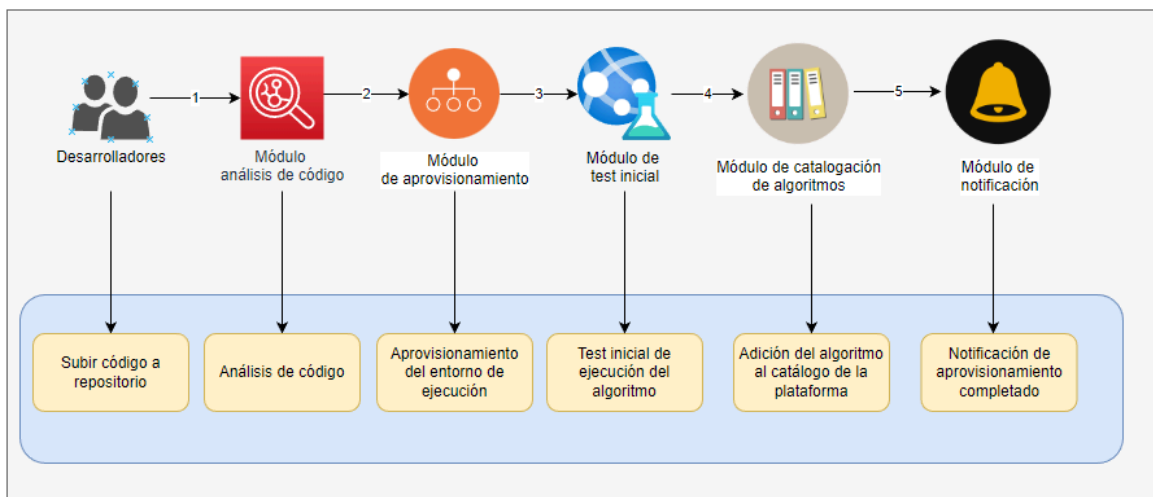
MAPE: 0.42%

5. ESPECIFICACIÓN A ALTO NIVEL DEL SISTEMA

Siguiendo las fases dos y tres de la metodología planteada, en la siguiente sección se describe la funcionalidad de la plataforma a alto nivel y las historias de usuario del proyecto, también llamadas en algunas metodologías requisitos conductores. Dichas historias de usuario se elaboran basándose en los objetivos del proyecto y trabajos previos realizados en el tema, como la plataforma Naveer presentada por Sung N. et al. (2017) o Acumos presentada por Shuai et al. (2018). Ambos trabajos se describen en el marco teórico del proyecto.

5.1 MODELO CONCEPTUAL DEL SISTEMA

Desde la perspectiva de los desarrolladores, que son los encargados de agregar nuevos algoritmos a la plataforma, conceptualmente se sigue el proceso descrito en la Figura 28, en la cual se plantea la posibilidad de que la plataforma tome el código de algún repositorio y realice el aprovisionamiento y catalogación de dicho módulo dentro del sistema.



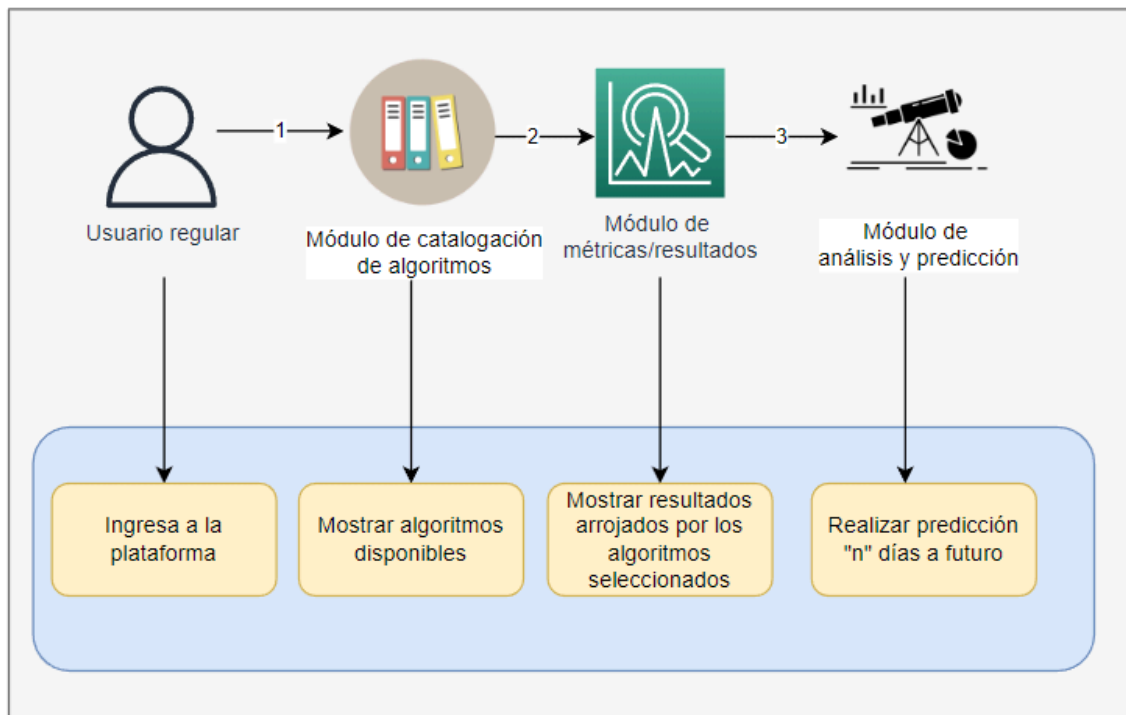
1. El desarrollador sube su código al repositorio y arranca el proceso de vinculación a la plataforma. Ésta ejecuta el módulo de análisis.
2. Después de haber analizado el código en busca de vulnerabilidades, se lanza el módulo de aprovisionamiento de entorno de ejecución.
3. Una vez se ha aprovisionado el entorno de ejecución, se realiza una prueba inicial al algoritmo.
4. Si la prueba fue exitosa, se agrega el algoritmo al catálogo.
5. Una vez se ha agregado el algoritmo al catálogo, se realiza una notificación a los desarrolladores.

Figura 28. Modelo conceptual del proceso que sigue un desarrollador para vincular un nuevo algoritmo, de predicción con ML, a la plataforma.

Ahora bien, desde la perspectiva de un usuario que requiere hacer análisis sobre el rendimiento de los diferentes algoritmos en el catálogo, la plataforma debe permitir

seleccionar algoritmos del catálogo a los que tenga acceso el usuario y ejecutarlos con diferentes conjuntos de datos para mostrar los resultados en relación a tres métricas: MAE, RMSE y MAPE.

Luego, si el usuario lo desea, se podría realizar una predicción de "n" días a futuro para la serie de tiempo que se ingrese como parámetro. Este comportamiento se describe en la figura 29.



1. El usuario regular busca y agrega uno o varios algoritmos para comparar del catálogo.
2. El módulo de métricas muestra los resultados de cada algoritmo para su análisis.
3. Una vez se muestran los resultados, está la posibilidad de correr cualquiera de los algoritmos para hacer una predicción a futuro de la serie de tiempo.

Figura 29. Modelo conceptual del proceso que sigue un usuario regular para usar y analizar los modelos de predicción con ML dentro la plataforma

5.2 MODELO DE PROCESOS

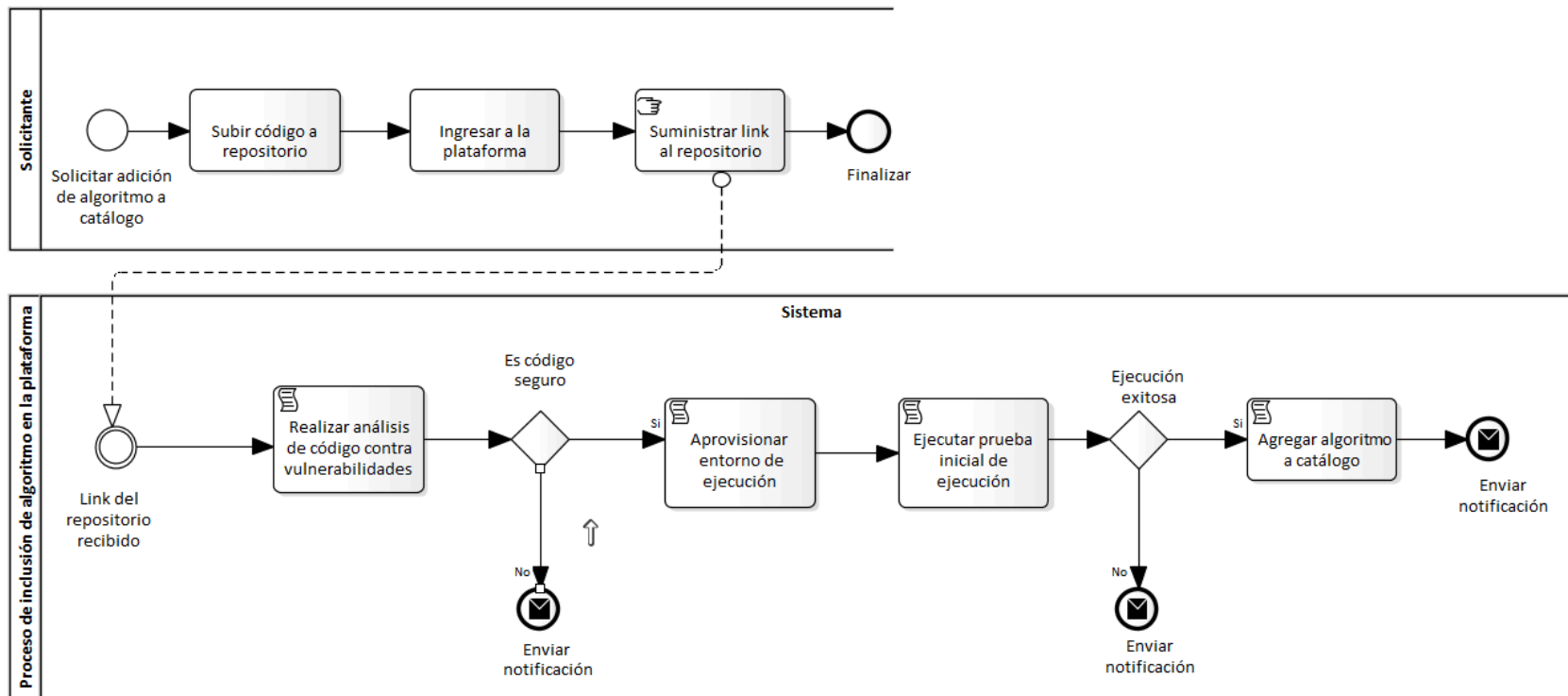


Figura 30. Modelo de proceso de negocio para la vinculación de un nuevo algoritmo de ML en la plataforma.

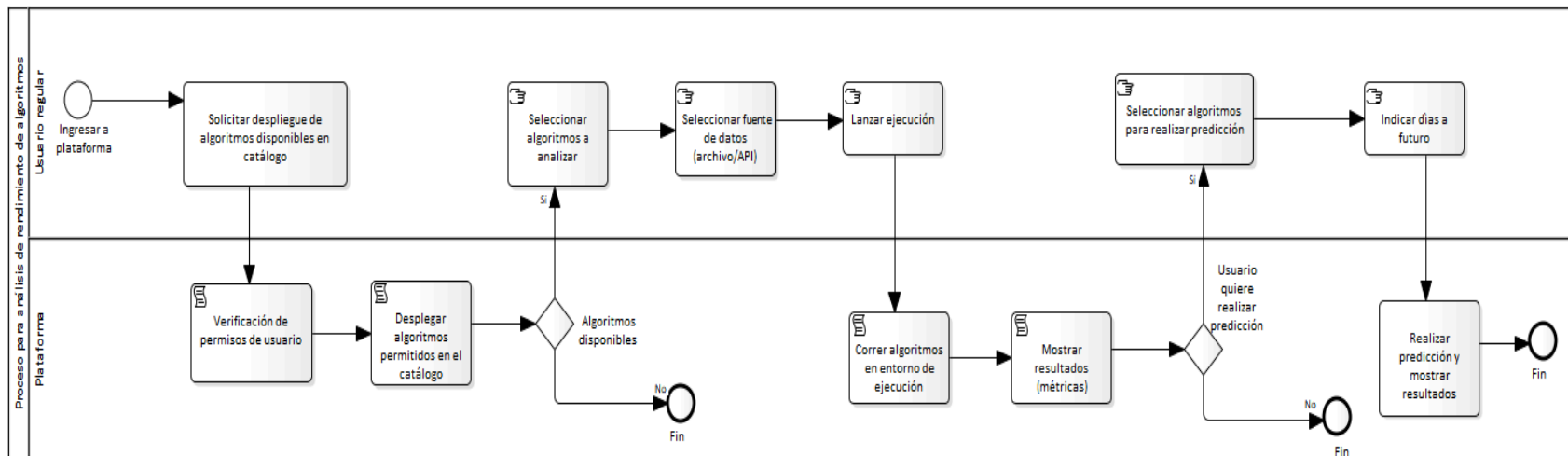


Figura 31. Modelo de proceso de negocio para uso de los algoritmos de ML dentro de la plataforma.

Para la incorporación de un nuevo algoritmo al catálogo de la plataforma, se sigue el proceso de negocio mostrado en la figura 30. Un usuario solicita la adición de un nuevo algoritmo a la plataforma a través de un enlace al repositorio del código. A continuación, el sistema realiza un análisis en busca de posibles vulnerabilidades en el código. Si se detecta alguna vulnerabilidad, se notifica al usuario. De lo contrario, se aprovisiona el entorno de ejecución y se realiza una prueba inicial de ejecución. Finalmente, si la ejecución fue exitosa, se añade el algoritmo al catálogo y se notifica al solicitante.

Por otro lado, para el uso regular de la plataforma, se sigue el proceso descrito en la figura 31. Primero, el usuario solicita que se muestre la lista de todos los algoritmos a los que tiene acceso. Segundo, el sistema realiza la validación de roles y permisos y despliega la lista; si no existen algoritmos para mostrar, el proceso finaliza. Tercero, una vez desplegada la lista de algoritmos, el usuario tiene la opción de seleccionar cuál o cuáles desea usar. Cuarto, el usuario selecciona cuál va a ser la fuente de datos a utilizar, que puede ser un archivo externo o la fuente de datos del sistema. Quinto, el usuario solicita la ejecución de los algoritmos seleccionados y espera los resultados. Por último, el sistema muestra los resultados de precisión para cada algoritmo y da la opción al usuario de realizar una predicción utilizando los algoritmos entrenados, si así lo desea.

5.3 CARACTERIZACIÓN DE USUARIOS DEL SISTEMA

Basados en el modelo de negocio, se pudieron identificar tres grandes actores en el sistema: desarrolladores, usuarios regulares y administradores. A continuación, se caracterizan y describen brevemente cada uno de estos.

Usuario	Descripción
Desarrollador	Este rol corresponde al desarrollador del algoritmo de aprendizaje automático que lo integra a la plataforma para su ejecución posterior por los usuarios. Se pueden identificar tres funciones principales: 1. Vinculación del algoritmo desarrollado a la plataforma mediante el repositorio del código. 2. Solicitud de análisis de vulnerabilidades del código a la plataforma.

	3. Ejecución del algoritmo en la plataforma.
Usuario regular	Este usuario es aquel que puede ingresar a la plataforma y hacer uso de los algoritmos existentes asignados en el catálogo a dicho usuario. Se identifican dos funciones principales: 1. Visualización y uso del catálogo de algoritmos de ML. 2. Ejecución y visualización de los resultados del algoritmo en la plataforma.
Usuario administrador	Este usuario tiene como función principal la administración de la plataforma. Es el encargado de la creación de roles, usuarios y permisos. Se identifican siete funciones principales: 1. Gestión de usuarios y roles. 2. Administración del catálogo y permisos. 3. Administración de los entornos y parámetros de ejecución. 4. Gestión de fuentes de datos de entrada y tipos de datos. 5. Gestión de eventos de notificación y listas de envío. 6. Gestión de proveedores de repositorios a usar. 7. Gestión de reglas de análisis de vulnerabilidades de código en la plataforma.

5.4 HISTORIAS DE USUARIO

Basados en los actores del sistema encontrados, se procede a realizar las historias de usuario para cada uno de estos.

Para la descripción de las historias de usuario se hace uso de la notación BABOK-UML (Business Analysis Body of Knowledge - Unified Modeling Language), que nos permite realizar un mejor análisis de negocio y describir de una manera más clara los requerimientos conductores.

5.4.1 Épica principal

Épica	Estado	Descripción
Comparativa de algoritmos	Propuesto	Algunas personas que trabajan con modelos de predicción mediante Machine Learning sobre series de tiempo, como en el caso de la predicción del mercado de divisas, necesitan realizar comparativas de la eficacia entre algoritmos. Estos algoritmos pueden ser implementaciones de trabajos realizados por varios autores en artículos científicos, algoritmos existentes o algoritmos implementados por otros desarrolladores. Aunque los algoritmos pueden ser implementados en varios lenguajes de programación, es difícil adecuar el entorno de ejecución para cada uno de ellos y generar métricas que permitan la comparación.

5.4.2 Historias de usuario - Desarrollador

Como se puede observar en la figura 32, a partir de la épica: “Comparativa de algoritmos” se desprenden tres historias de usuario principales para el rol “Desarrollador”: Hist01, Hist02 e Hist03. Además, se encuentran siete criterios de aceptación que se pueden observar en la figura 33.

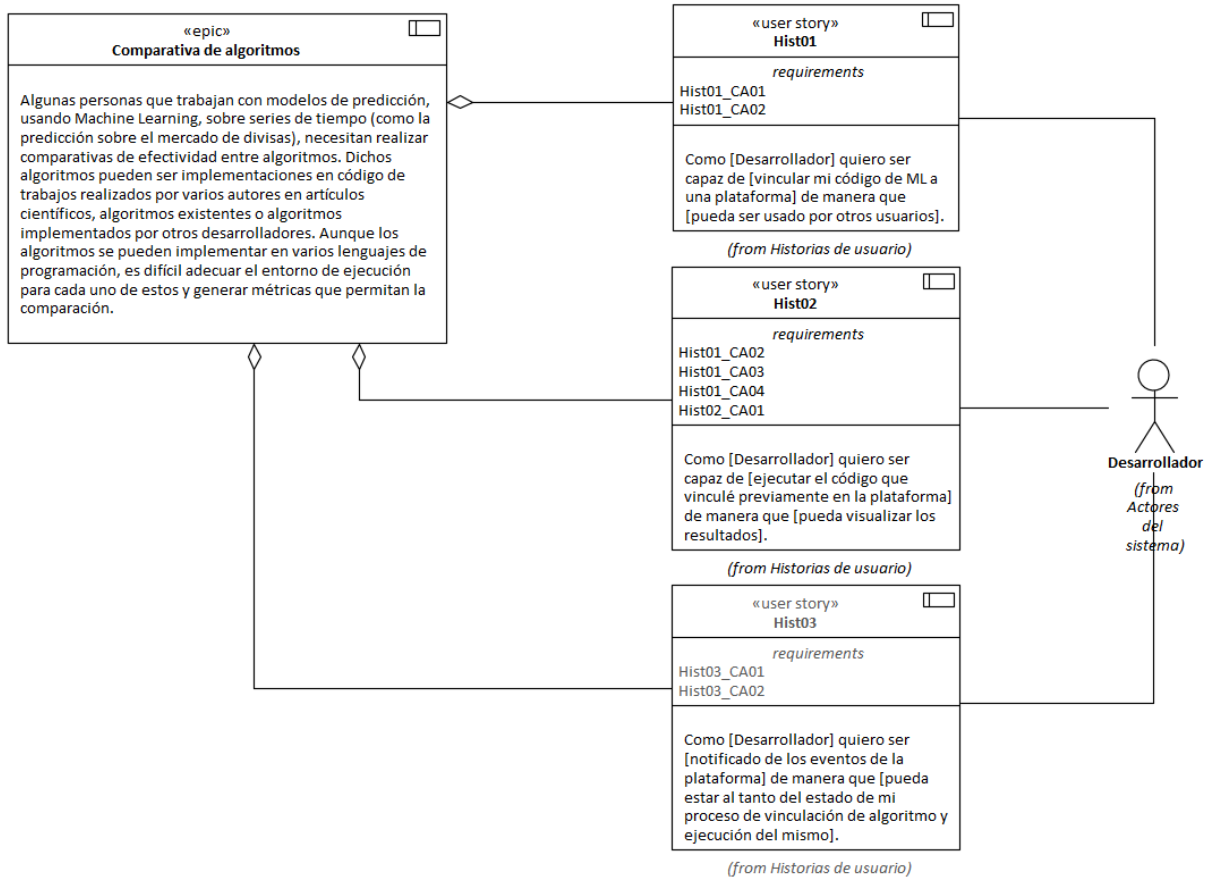


Figura 32. Modelo de épicas e historias de usuario usando notación BABOK para el rol de desarrollador.

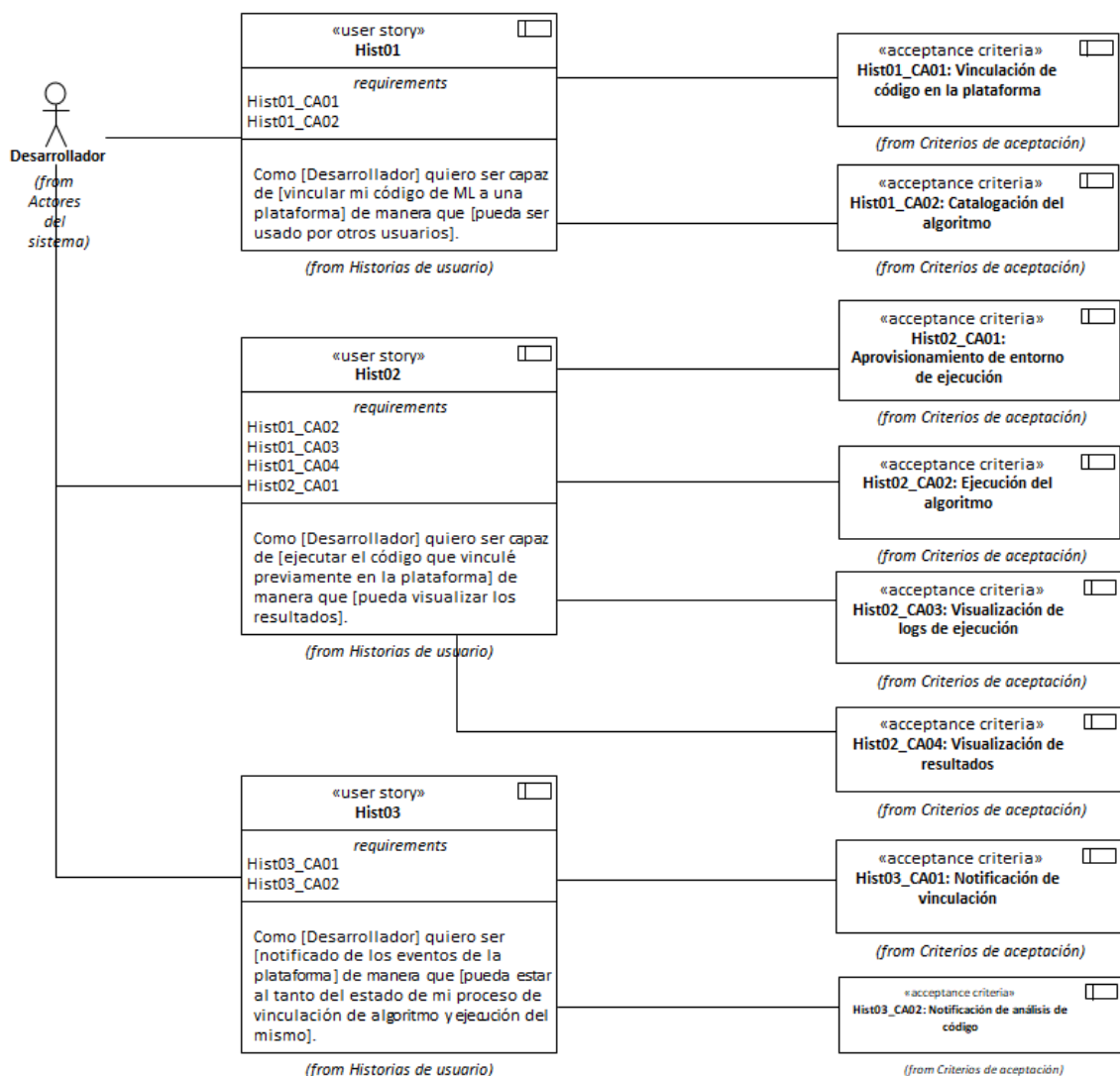


Figura 33. Modelo de historias de usuario y criterios de aceptación usando notación BABOK para el rol de desarrollador.

La información de cada historia de usuario y criterios de aceptación se muestran a continuación.

Tabla de historias - Desarrollador

Historia	Estado	Descripción
Hist01	Propuesto	Como [desarrollador], quiero ser capaz de [vincular mi código de Machine Learning a una plataforma] para que [otros usuarios puedan hacer uso del mismo].
Hist02	Propuesto	Como [Desarrollador] quiero ser capaz de [ejecutar el código de ML que vinculé previamente en la plataforma] de manera que [pueda obtener resultados precisos y útiles para los usuarios].
Hist03	Propuesto	Como [Desarrollador], quiero ser [notificado por la plataforma] de manera que [pueda estar al tanto del estado de mi proceso de vinculación de algoritmo y ejecución del mismo].

Tabla de criterios de aceptación - Desarrollador

Criterio de aceptación	Estado	Descripción
Hist01_CA01: Vinculación de código en la plataforma	Propuesto	El usuario puede tomar su código de un repositorio público, solicitar su vinculación a la plataforma y el sistema realiza un análisis en busca de posibles vulnerabilidades antes de aprovisionar el entorno de ejecución para su uso en la plataforma.
Hist01_CA02: Catalogación del algoritmo	Propuesto	El algoritmo vinculado debe aparecer en el catálogo de algoritmos para que otros usuarios puedan usarlo.
Hist02_CA01: Aprovisionamiento de entorno de ejecución	Propuesto	Una vez que un algoritmo está vinculado, se debe aprovisionar un entorno de ejecución.
Hist02_CA02: Ejecución del algoritmo	Propuesto	El algoritmo se puede ejecutar en la plataforma.
Hist02_CA03: Visualización de logs de ejecución	Propuesto	Después de la ejecución, se deberían poder visualizar los registros de ejecución de la aplicación (incluyendo posibles errores).

Hist02_CA04: Visualización de resultados	Propuesto	Se deben poder mostrar los resultados de la ejecución del algoritmo.
Hist03_CA01: Notificación de vinculación	Propuesto	El usuario recibe una notificación indicando si la vinculación del algoritmo fue exitosa o no, en caso de fallo, se especifica el motivo del mismo.
Hist03_CA02: Notificación de análisis de código	Propuesto	Cuando se realice el análisis del código en busca de vulnerabilidades y se encuentre alguna inconsistencia, se espera que se notifique al usuario correspondiente.

5.4.3 Historias de usuario - Usuario regular

Como se puede observar en la figura 34, a partir de la épica "Comparativa de algoritmos", se derivan dos historias de usuario principales para el rol "Usuario regular": Hist04 y Hist05. Además, se presentan tres criterios de aceptación que se pueden ver en la figura 35.

El usuario regular, como se describió anteriormente, es aquel que hace uso de los algoritmos que están en el catálogo de la plataforma y lleva a cabo los análisis pertinentes para realizar predicciones a futuro.

A continuación, se presentan las tablas de historias de usuario y criterios de aceptación.

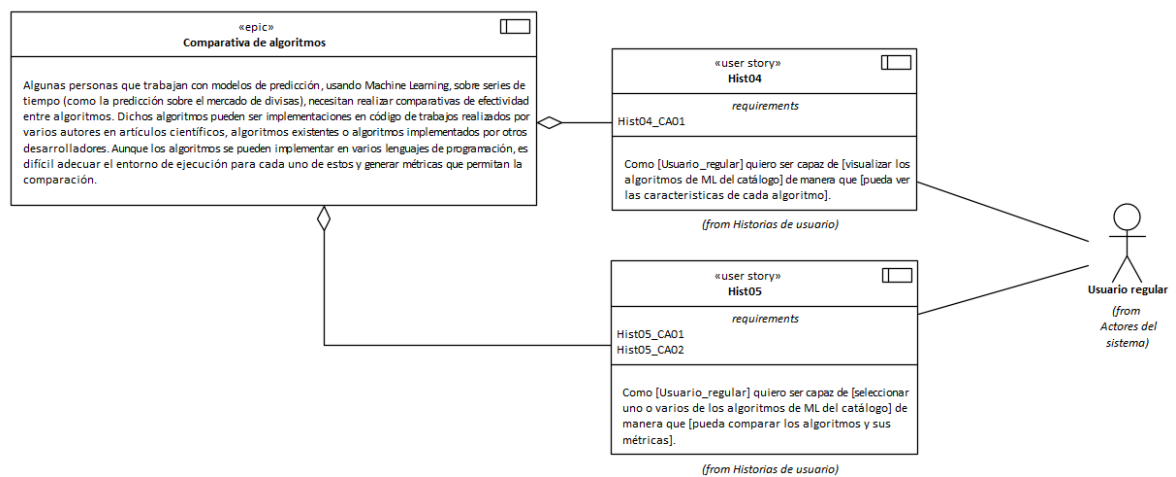


Figura 34. Modelo de épicas e historias de usuario usando notación BABOK para el rol de usuario regular.

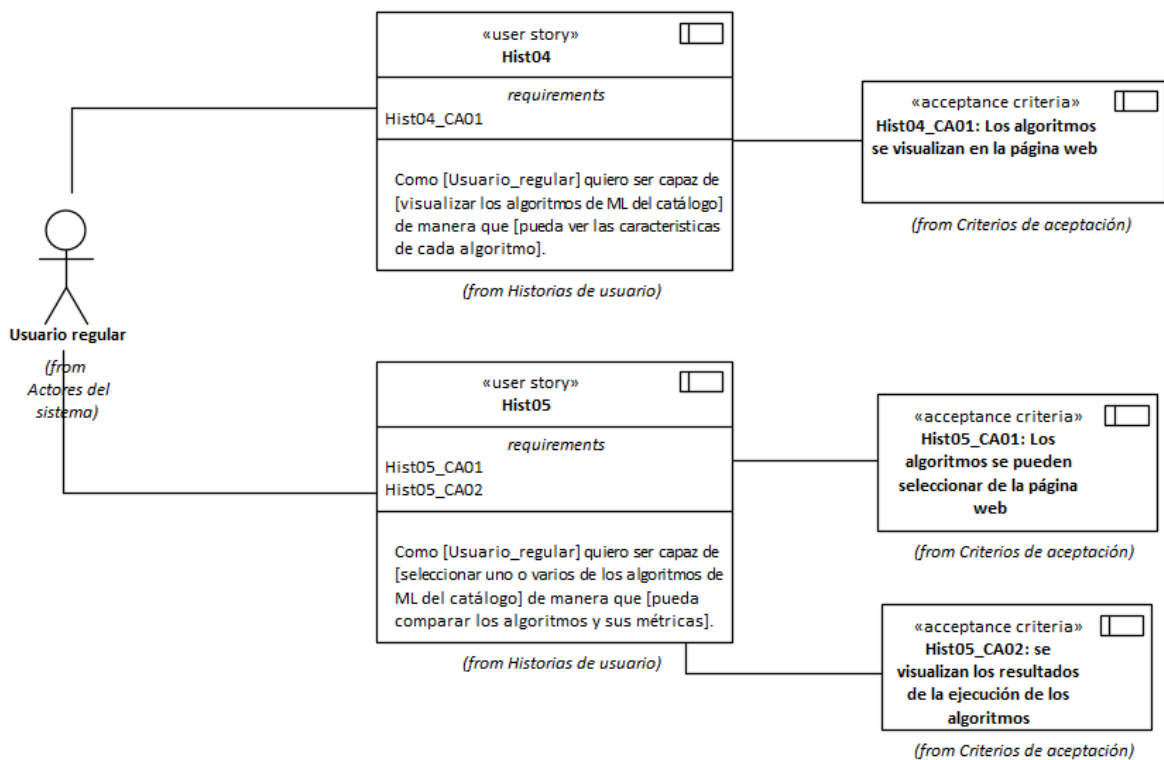


Figura 35. Modelo de historias de usuario y criterios de aceptación usando notación BABOK para el rol de usuario regular.

Tabla de historias - Usuario regular

Historia	Estado	Descripción
Hist04	Propuesto	Como [Usuario_regular] quiero ser capaz de visualizar los algoritmos de ML del catálogo [de manera que pueda ver las características de cada algoritmo].
Hist05	Propuesto	Como [Usuario_regular] quiero ser capaz de seleccionar uno o varios algoritmos de ML del catálogo, [de manera que pueda compararlos y evaluar sus métricas].

Tabla de criterios de aceptación - Usuario regular

Criterio de aceptación	Estado	Descripción
Hist04_CA01: Los algoritmos se visualizan en la página web	Propuesto	El catálogo de algoritmos muestra los algoritmos asociados al usuario que ingresa a la plataforma con las características de cada algoritmo. Incluyendo elementos de entrada y salida.
Hist05_CA01: Los algoritmos se pueden seleccionar de la página web	Propuesto	La plataforma permite seleccionar los algoritmos de la página web y correrlos con los datos de entrada.

Hist05_CA02: se visualizan los resultados de la ejecución de los algoritmos	Propuesto	Después de la ejecución de los algoritmos, se deben mostrar las métricas de ejecución.
---	-----------	--

5.4.4 Historias de usuario - Administrador

Como se puede observar en la figura 36, a partir de la épica "Comparativa de algoritmos", se derivan seis historias de usuario principales para el rol "Administrador": Hist06, Hist07, Hist08, Hist09, Hist10 y Hist11. Además, se encuentran nueve criterios de aceptación que se pueden observar en la figura 37.

Como se describió anteriormente, el usuario administrador tiene la responsabilidad de gestionar la mayoría de los parámetros de la plataforma, desde los parámetros del entorno de ejecución hasta los usuarios y roles del sistema.

A continuación, se presentan las tablas de historias de usuario y criterios de aceptación.

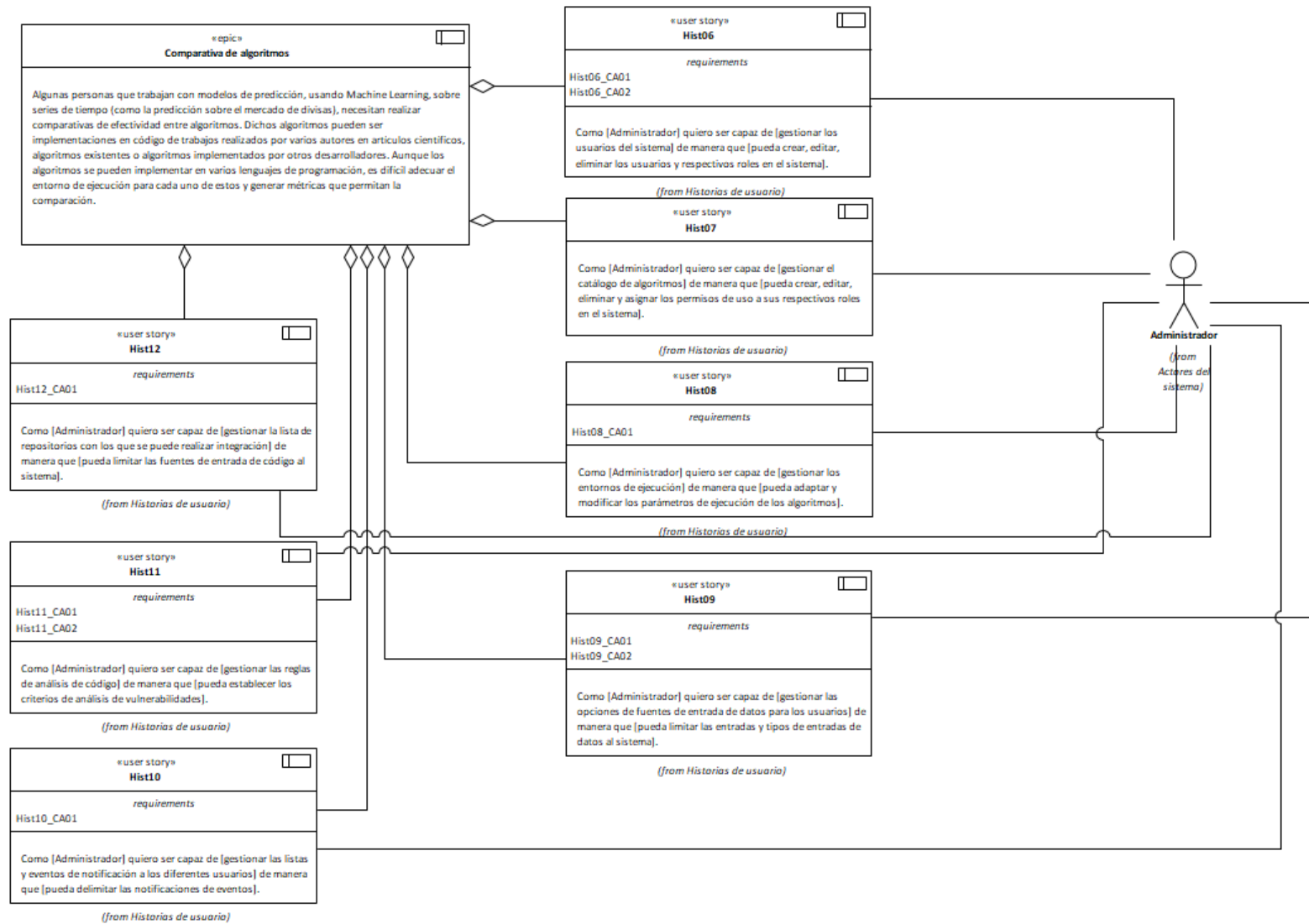


Figura 36.

Modelo de historias de usuario y criterios de aceptación usando notación BABOK para el rol de administrador.

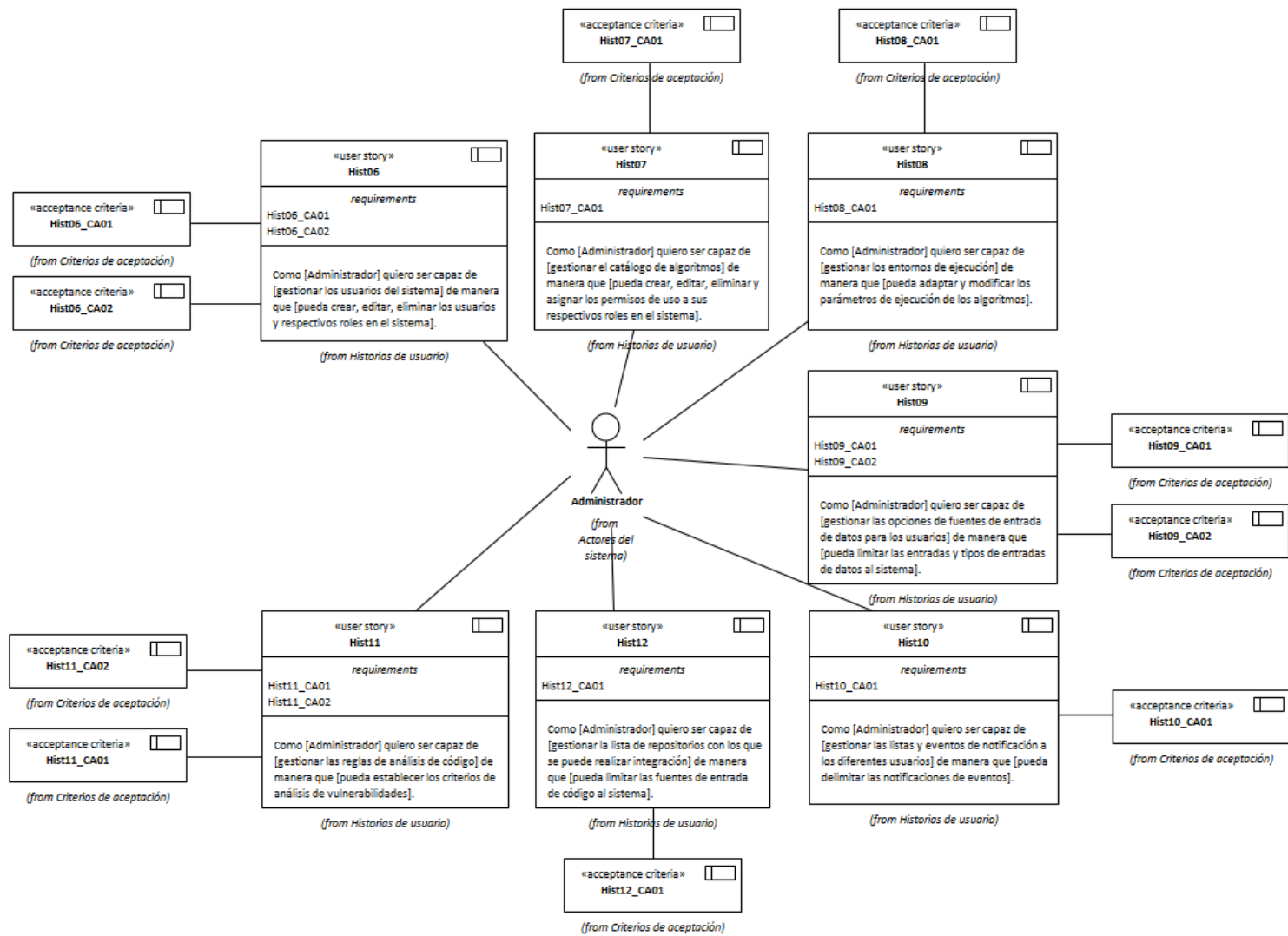


Figura 37. Modelo de historias de usuario y criterios de aceptación usando notación BABOK para el rol de usuario regular.

Tabla de historias - Administrador

Historia	Estado	Descripción
Hist06	Propuesto	Como [Administrador] quiero ser capaz de [gestionar los usuarios del sistema] de manera que [pueda crear, editar, eliminar los usuarios y respectivos roles en el sistema].
Hist07	Propuesto	Como [Administrador] quiero ser capaz de [gestionar el catálogo de algoritmos] de manera que [pueda crear, editar, eliminar y asignar los permisos de uso a los roles del sistema correspondientes].
Hist08	Propuesto	Como [Administrador] quiero ser capaz de [gestionar los entornos de ejecución] de manera que [pueda adaptar y modificar los parámetros de ejecución de los algoritmos].
Hist09	Propuesto	Como [Administrador] quiero ser capaz de [gestionar las opciones de fuentes de entrada de datos para los usuarios] de manera que [pueda configurar y restringir las fuentes de entrada de datos al sistema].
Hist10	Propuesto	Como [Administrador] quiero ser capaz de [gestionar las listas y eventos de notificación para los diferentes usuarios] de manera que [pueda

		configurar y personalizar las notificaciones según el evento y el usuario correspondiente].
Hist11	Propuesto	Como [Administrador] quiero ser capaz de [gestionar las reglas de análisis de código] de manera que [pueda establecer los criterios de análisis de vulnerabilidades].
Hist12	Propuesto	Como [Administrador] quiero ser capaz de [gestionar la lista de repositorios con los que se puede realizar integración] de manera que [pueda controlar y limitar las fuentes de entrada de código al sistema].

Tabla de criterios de aceptación - Administrador

Criterio de aceptación	Estado	Descripción
Hist06_CA01	Propuesto	El usuario puede crear/editar y eliminar usuarios.
Hist06_CA02	Propuesto	El usuario, puede administrar y asignar roles a los usuarios.
Hist07_CA01	Propuesto	El administrador puede crear, editar y eliminar algoritmos del catálogo. Además, puede

		gestionar los diferentes roles para asignar los algoritmos a los que tiene acceso cada rol..
Hist08_CA01	Propuesto	Los parámetros de memoria, unidades de procesamiento y demás parámetros del entorno de ejecución, son modificables por el usuario.
Hist09_CA01	Propuesto	El usuario puede agregar y quitar fuentes de datos.
Hist09_CA02	Propuesto	El usuario puede agregar y quitar tipos de datos de entrada del sistema en general.
Hist10_CA01	Propuesto	El usuario puede agregar/editar y eliminar eventos de notificación y listas de envío.
Hist11_CA01	Propuesto	El usuario puede crear, modificar y eliminar reglas de análisis de código.
Hist11_CA02	Propuesto	El usuario puede elegir la herramienta de análisis de código que va a usar el sistema a nivel general.
Hist12_CA01	Propuesto	El usuario puede elegir los proveedores de repositorios de código con los que se puede integrar el sistema.

5.5 CASOS DE USO

Basado en las historias de usuario descritas anteriormente, se procede a realizar la descripción de los casos de uso para los diferentes actores del sistema. Además, se utilizará el Lenguaje Unificado de Modelado (UML) para representar, a través de diagramas de casos de uso, las diferentes interacciones de los usuarios con el sistema.

Estos casos de uso permitirán, más adelante, proponer una arquitectura para el sistema y diseñar los módulos del mismo.

5.5.1 CU01 - Vinculación de código del algoritmos a la plataforma

Con base en las historias de usuario, desde la perspectiva del desarrollador de los algoritmos de ML, su función se limita a adicionar el programa al catálogo de algoritmos para que pueda ser usado por los otros usuarios de la plataforma, siempre y cuando tenga los permisos necesarios para hacerlo.

Para vincular un algoritmo a la plataforma, el desarrollador debe asegurarse de que el código esté alojado en un repositorio al que la plataforma tenga acceso. Una vez que tenga el enlace del repositorio, deberá autenticarse en la aplicación y agregar el enlace del repositorio en la plataforma. Posteriormente, el sistema analizará el código en busca de vulnerabilidades y aprovisionará el entorno de ejecución si el análisis es exitoso. Este proceso se muestra en la figura 38.

Algunos de estos casos de uso incluyen otros casos de uso, como cargar las reglas de análisis de memoria o cargar y compilar el código. Además, existen extensiones de los mismos, como la opción de ejecutar la prueba de ejecución del algoritmo, que está disponible para el desarrollador antes de finalizar la vinculación.

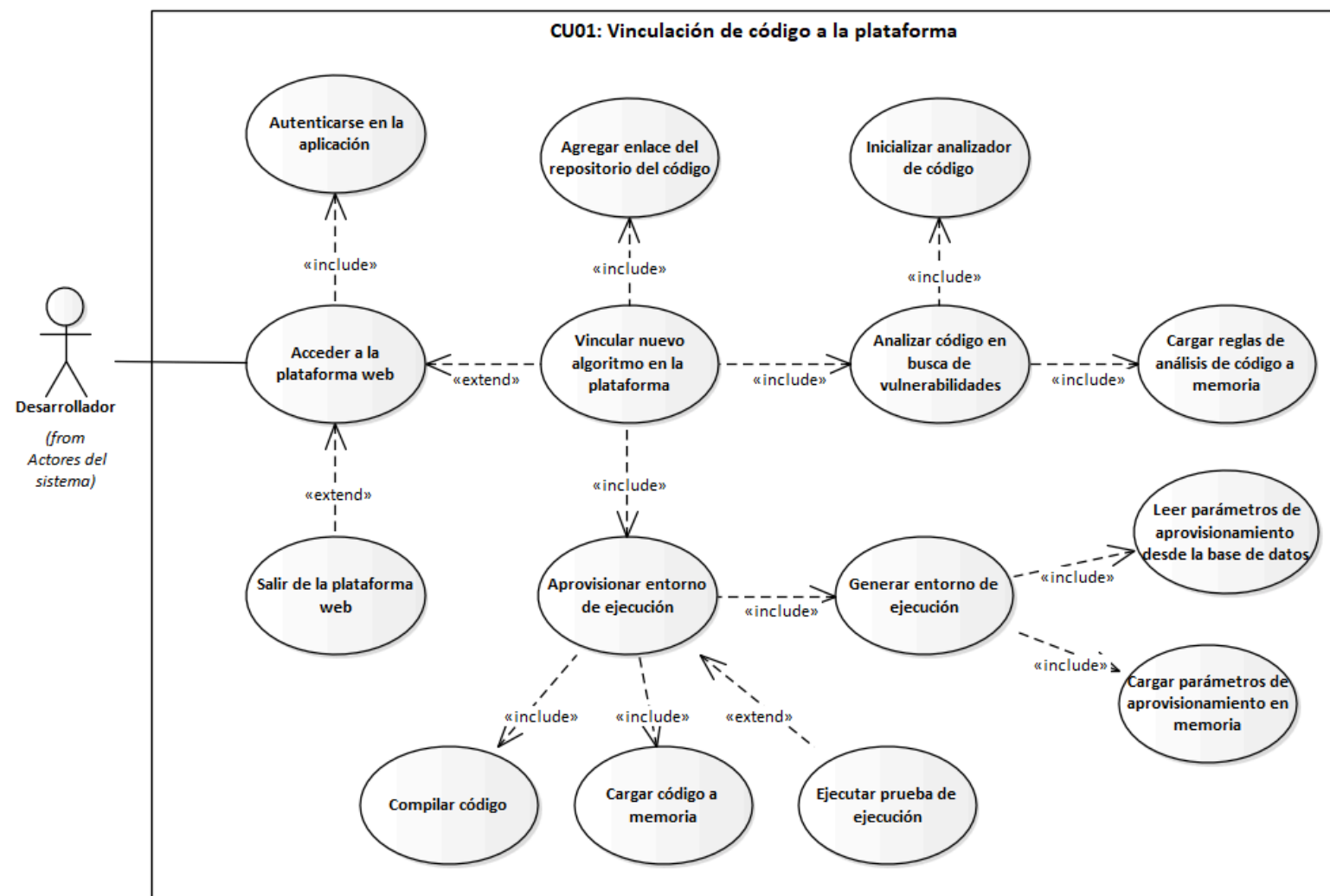


Figura 38. Diagrama de casos de uso CU01 - Vinculación de código del algoritmos de ML a la plataforma, desde la perspectiva del Desarrollador del algoritmo.

5.5.2 CU02 - Visualización de catálogo de algoritmos en la plataforma

Desde la perspectiva del analista o usuario regular, se busca acceder al catálogo de algoritmos disponibles, seleccionar los que desea utilizar, ejecutarlos con un conjunto de datos, y finalmente, visualizar los resultados de rendimiento a través de métricas de comparación.

Como se muestra en la figura 39, el usuario accede a la plataforma mediante autenticación y puede abrir la opción de visualización del catálogo. Automáticamente el sistema deberá obtener la lista de algoritmos disponibles para ese usuario, cargarlos en memoria y desplegarlos en la interfaz gráfica. El usuario tendrá la opción de ordenarlos alfabéticamente y seleccionar el algoritmo que desea probar.

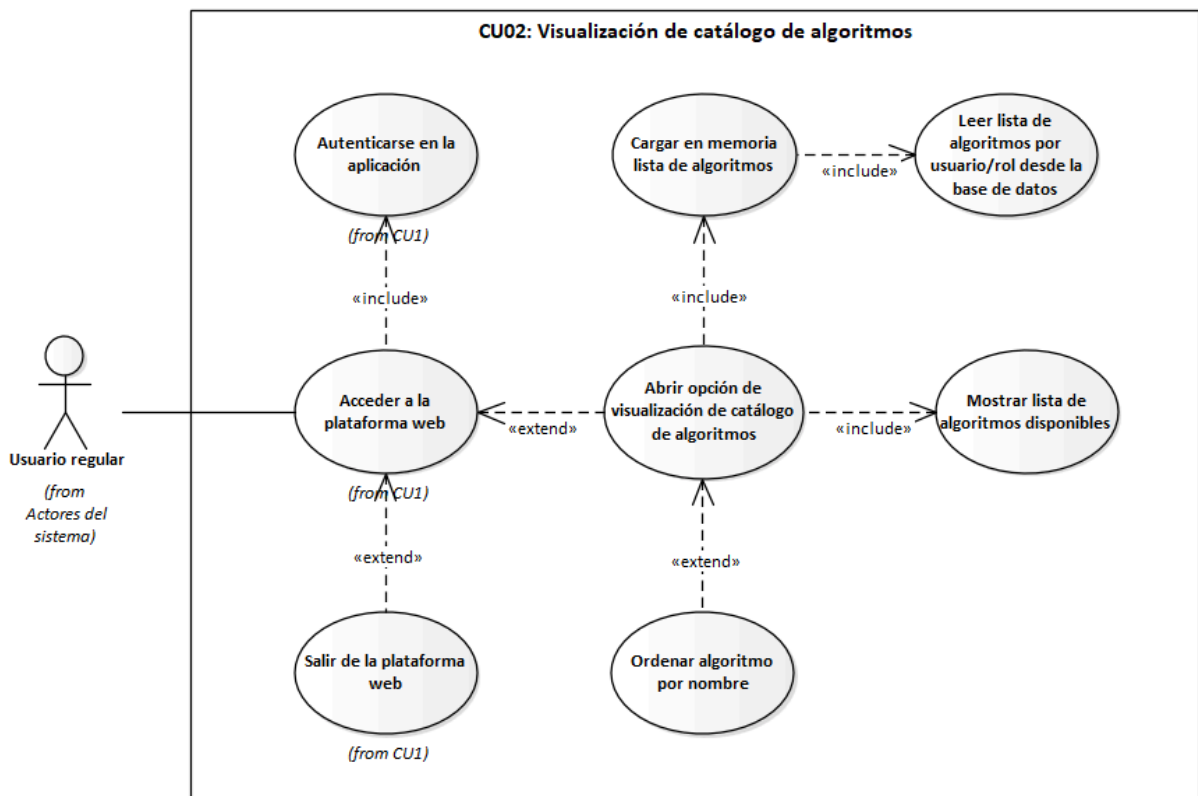


Figura 39. Diagrama de casos de uso CU02 - Visualización de catálogo de algoritmos de ML en la plataforma, desde la perspectiva del usuario regular.

5.5.3 CU03 - Ejecución de algoritmos

Como se describe en la figura 40, el usuario regular puede hacer uso de los algoritmos que se visualizan en el catálogo. Primero, el usuario debe seleccionar los algoritmos a ejecutar. Luego, debe elegir la fuente de entrada de datos para entrenar los modelos seleccionados. Las fuentes de datos disponibles son habilitadas por el administrador del sistema. Principalmente existen dos opciones: tomar los datos de una API proveedora o de un archivo de texto.

Una vez que el usuario ha seleccionado las opciones para ejecutar los algoritmos, se activa el entorno de ejecución, se carga el código en memoria y se prepara todo para su ejecución. Los parámetros de ejecución, tales como la cantidad de memoria, unidades de procesamiento, almacenamiento y otras variables, son almacenados en una base de datos.

5.5.4 CU04 - Visualización de resultados

Después de que la ejecución de los algoritmos seleccionados haya terminado, se podrán visualizar los resultados de rendimiento de cada algoritmo para su respectivo análisis. Las tres métricas principales para calcular el rendimiento de cada algoritmo son: MAE, RMSE y MAPE. Cada una de estas fue descrita con anterioridad en la sección 4.1.

Como se puede observar en la figura 41, una vez autenticado en la plataforma, el usuario tiene la opción de acceder a la visualización de resultados. Esta opción incluye el caso de uso de cargar los resultados a memoria y desplegar los elementos gráficos correspondientes. Para mostrar las métricas de rendimiento, se deben calcular el MAE, RMSE y MAPE con los resultados obtenidos de cada algoritmo.

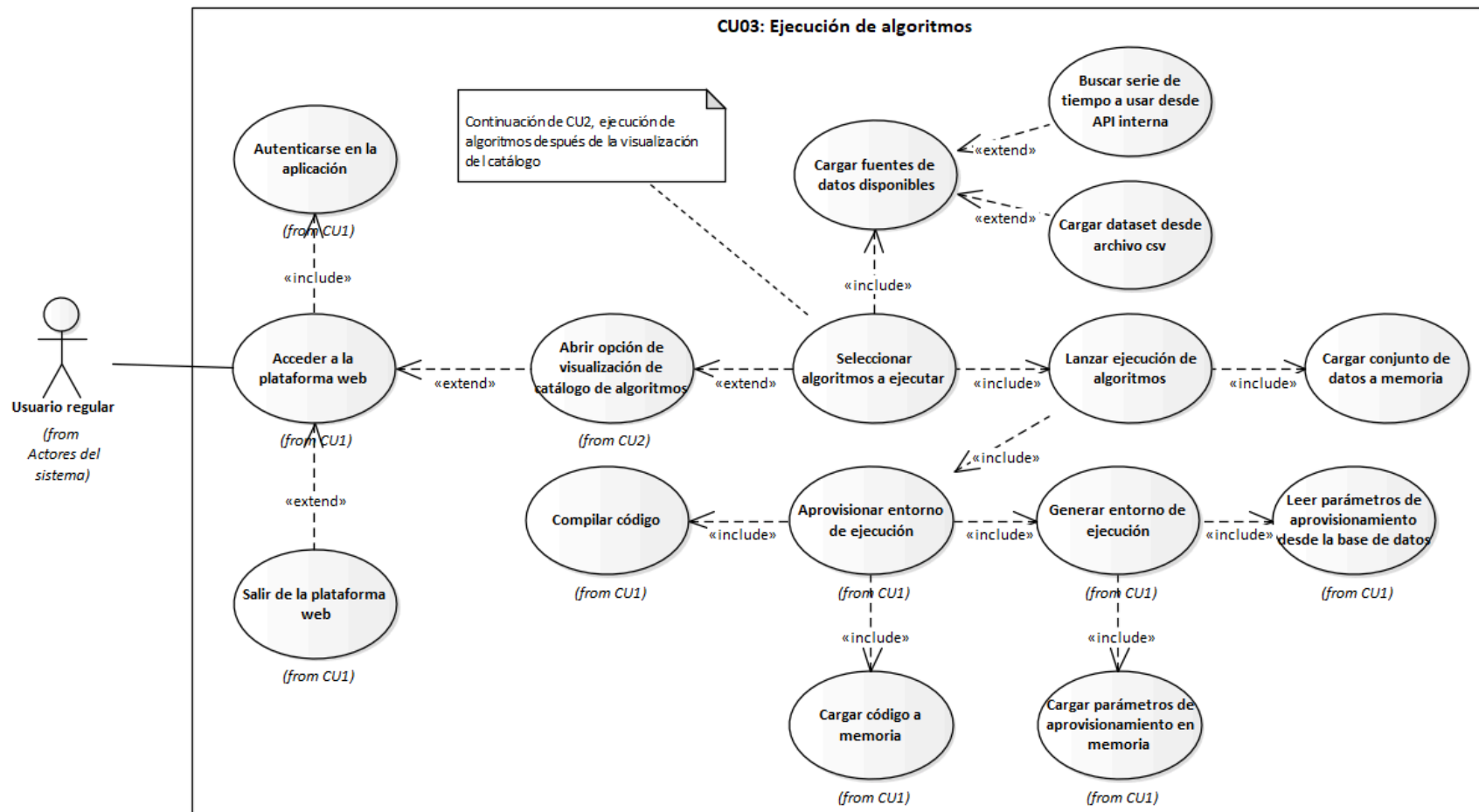


Figura 40. Diagrama de casos de uso CU03 - Ejecución de algoritmos de ML seleccionados en el catálogo, desde la perspectiva del usuario regular.

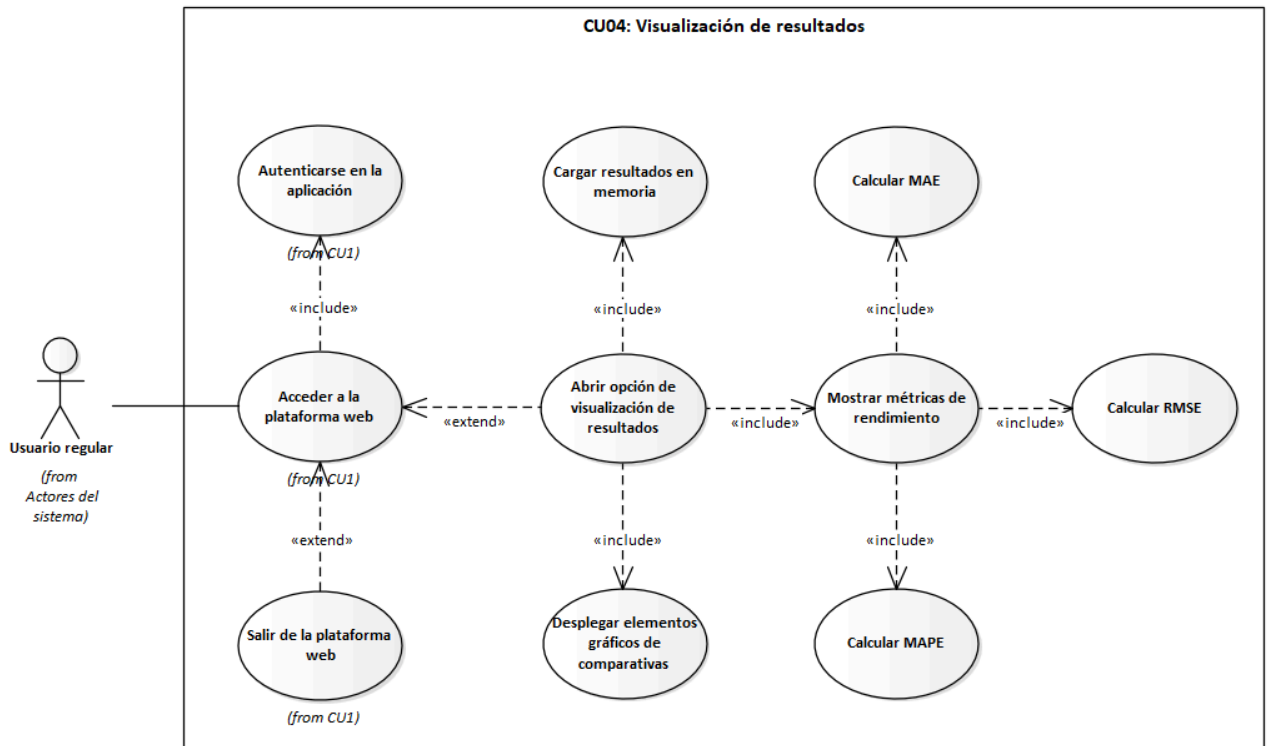


Figura 41. Diagrama de casos de uso CU04 - Visualización de resultados, desde la perspectiva del usuario regular.

5.5.5 CU05 - Administración de usuarios y roles

Desde la perspectiva del rol de administrador de la plataforma, se deben poder gestionar diferentes configuraciones de la plataforma, incluyendo el manejo de usuarios y roles.

El rol de cada usuario es una característica que permite agregar una capa de autorización a la plataforma. En pocas palabras, el rol limita las acciones que un usuario puede o no hacer una vez se ha autenticado.

Por otro lado, para la autenticación de los usuarios se debe utilizar algún mecanismo seguro para almacenar las credenciales de acceso. Normalmente, estos sistemas se denominan "directorios activos". Como se ilustra en la figura 42, el usuario con el rol de administrador podrá crear, editar y eliminar tanto roles como usuarios. Algunos de estos casos de uso incluyen la actualización del directorio activo y de la base de datos.

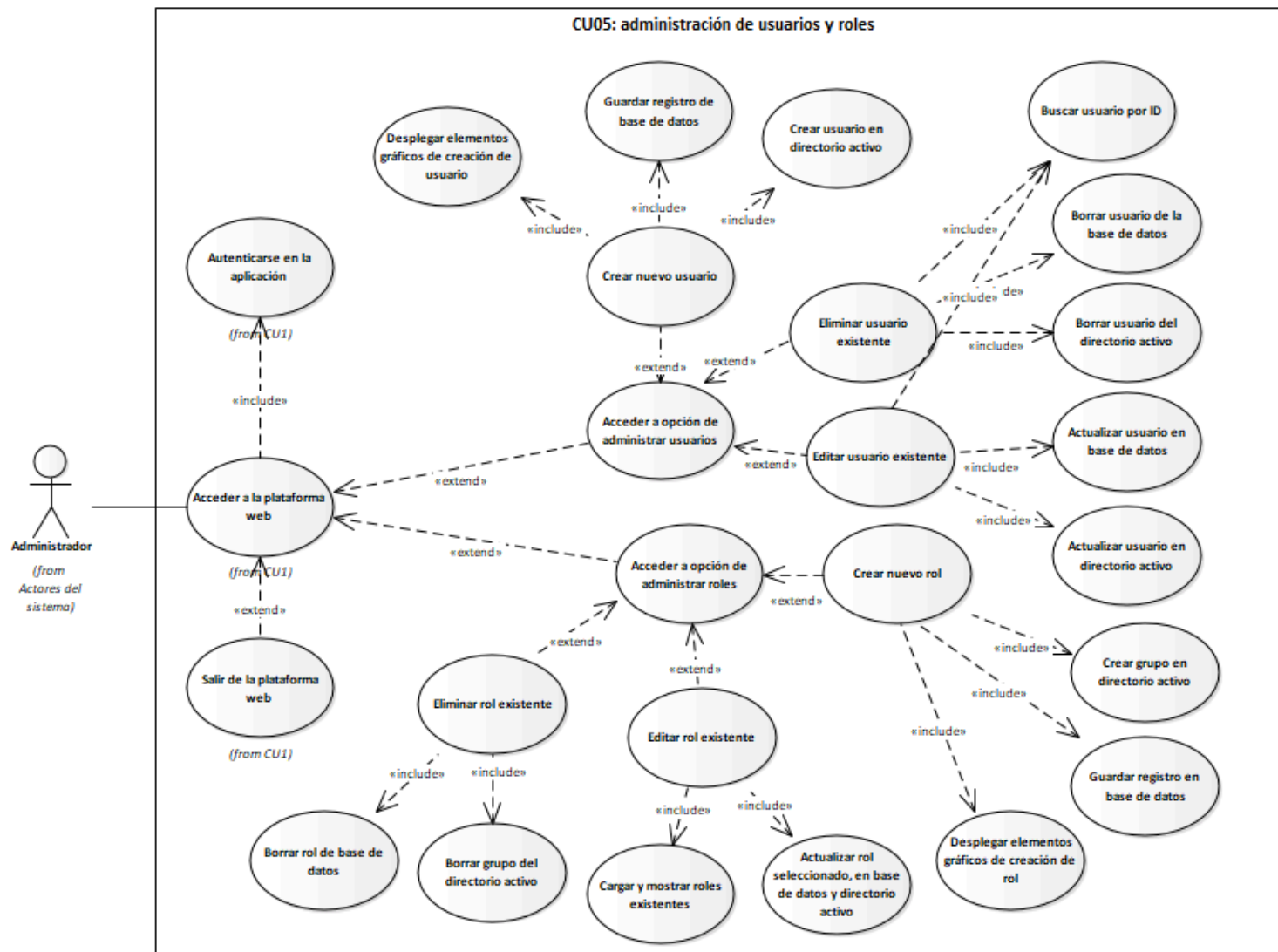


Figura 42. Diagrama de casos de uso CU05 - Gestión de usuarios y roles, desde la perspectiva del usuario administrador.

5.5.6 CU06 - Administración de catálogos

Otra de las funciones que tiene el administrador del sistema es la gestión de los catálogos de algoritmos. Dichos catálogos estarán asignados a grupos específicos de usuarios. La función del administrador es, como se muestra en la figura 43, crear, editar y eliminar catálogos y asignar los grupos que tendrán acceso a estos.

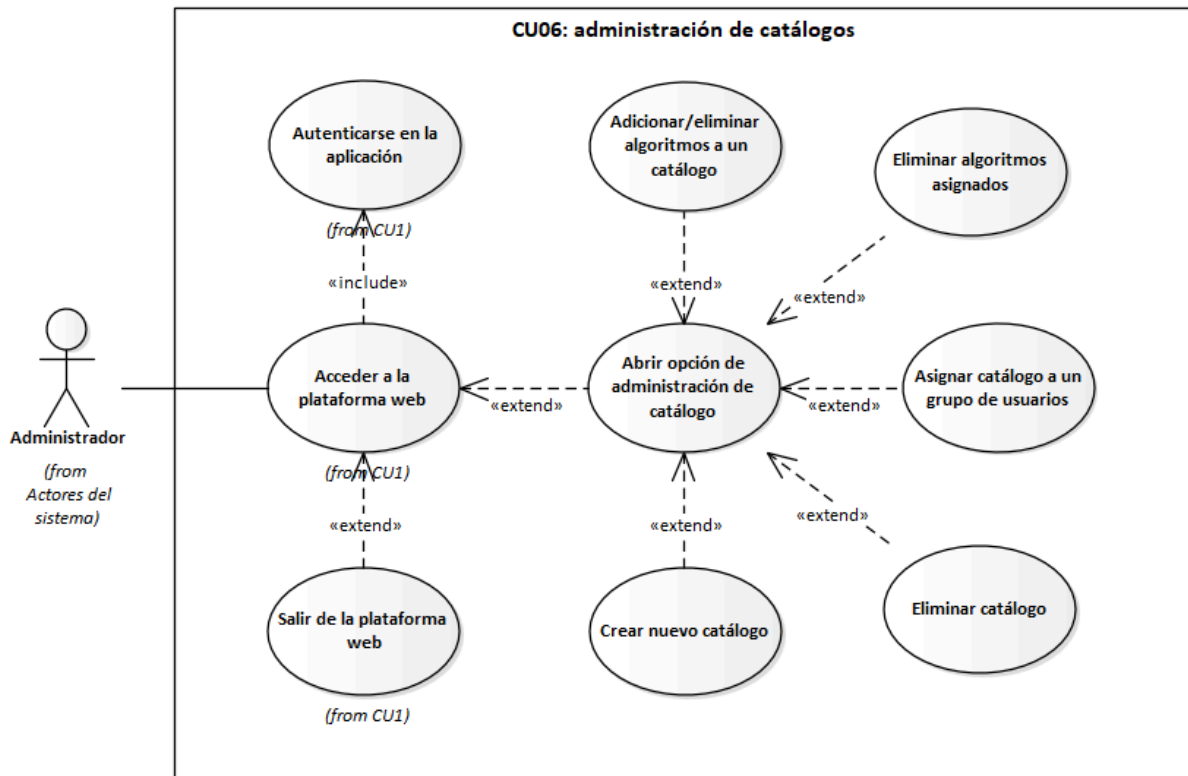


Figura 43. Diagrama de casos de uso CU06 - administración de catálogos de algoritmos, desde la perspectiva del usuario administrador.

5.5.7 CU07 - Administración de fuentes de entrada de datos

Dado que los algoritmos de aprendizaje automático (Machine Learning) requieren de entradas de datos para realizar el entrenamiento y las predicciones, la plataforma debe ofrecer la opción de administrar las fuentes de entrada de datos. Para esto, como se describe en el diagrama de casos de uso de la figura 44, el usuario administrador puede agregar, editar y eliminar los tipos de fuentes de datos que serán soportados por el sistema. Las fuentes de datos pueden ser obtenidas mediante una API que proporcione las series de tiempo de entrada o a través del uso de archivos de texto con una estructura definida, como es el caso de los archivos en formato .csv.

Una de las APIs más utilizadas para obtener información de series de tiempo financieras, como el precio de divisas y acciones del mercado, es Yahoo Finance. La idea es que la plataforma pueda integrarse con esta API y obtener información de allí, así como también conectarse con otros proveedores, como Google Finance.

En cuanto a la integración con archivos de texto, se deberá proveer la estructura del archivo, especificando por ejemplo si es un archivo por columnas o por filas, qué tipo de delimitador se utiliza, la extensión del archivo o si es un archivo JSON o XML, se debe proveer la meta-estructura del mismo.

5.5.8 CU08 - Gestión de entornos de ejecución

Cada algoritmo requiere un entorno de ejecución donde correr. Es tarea del administrador del sistema gestionar los parámetros de aprovisionamiento de dichos entornos, parámetros como: memoria ram, cantidad de unidades de procesamiento (cpu), unidades de almacenamiento, entre otros.

Como se muestra en la figura 45, el administrador, una vez autenticado en la plataforma, puede crear, modificar y eliminar configuraciones de dichos entornos. Dichos casos de uso incluyen, por ejemplo, consultar y traer a memoria la información desde la base de datos, al igual que actualizar y eliminar registros de la misma.

Estos parámetros son luego utilizados en otras funciones del sistema, por ejemplo, en el caso de uso número 3, se puede observar que los parámetros son cargados y con base en esto se genera el entorno donde se ejecuta el algoritmo.

Figura 44. Diagrama de casos de uso CU07 - administración de fuentes de entrada de datos, desde la perspectiva del usuario administrador.

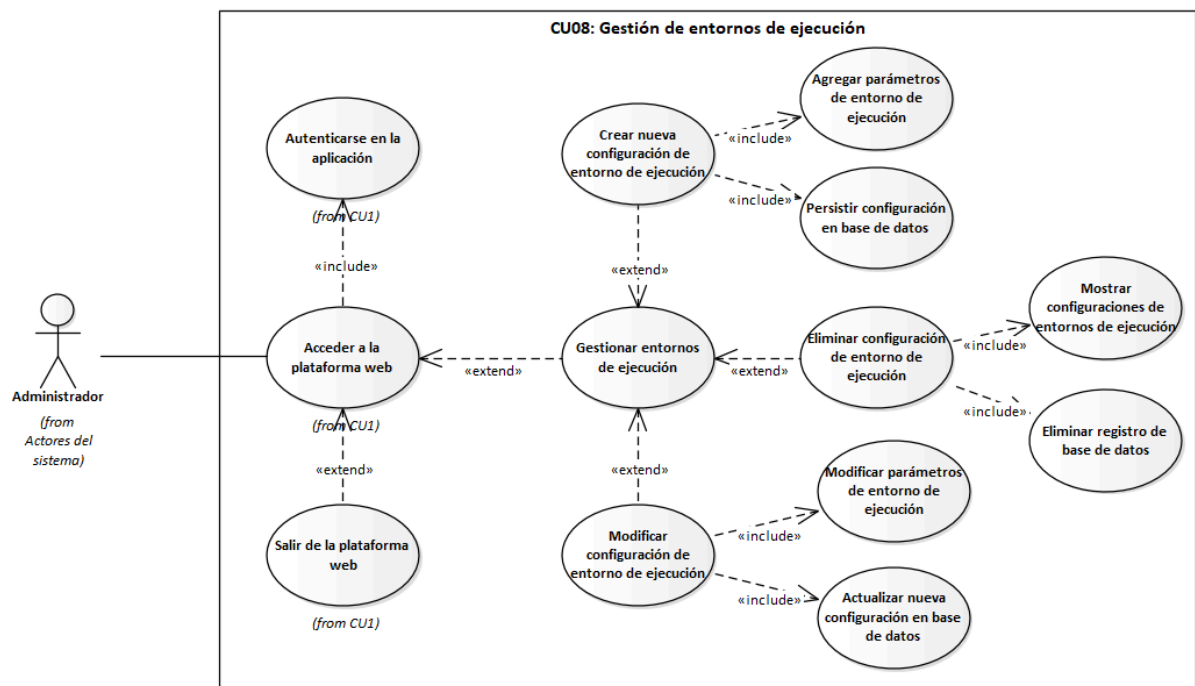


Figura 45. Diagrama de casos de uso CU08 - Gestión de entornos de ejecución, desde la perspectiva del usuario administrador.

5.5.9 CU09 - Gestión de notificaciones

Dentro de los requisitos principales, hay algunas historias de usuario que establecen la necesidad de notificar a los usuarios sobre los diferentes eventos del sistema. Por ejemplo, cuando un entorno de ejecución esté listo o cuando la ejecución de un algoritmo haya finalizado, se espera que se lance un evento de alerta que será recibido por los suscriptores o interesados en dicho evento.

El usuario administrador tiene entre sus funciones la gestión de la lista de envío y los eventos que deben ser notificados. En la figura 46 se muestra la interacción de este usuario con la plataforma. Una vez autenticado, el administrador puede agregar, modificar o eliminar eventos, así como modificar los suscriptores a dichos eventos.

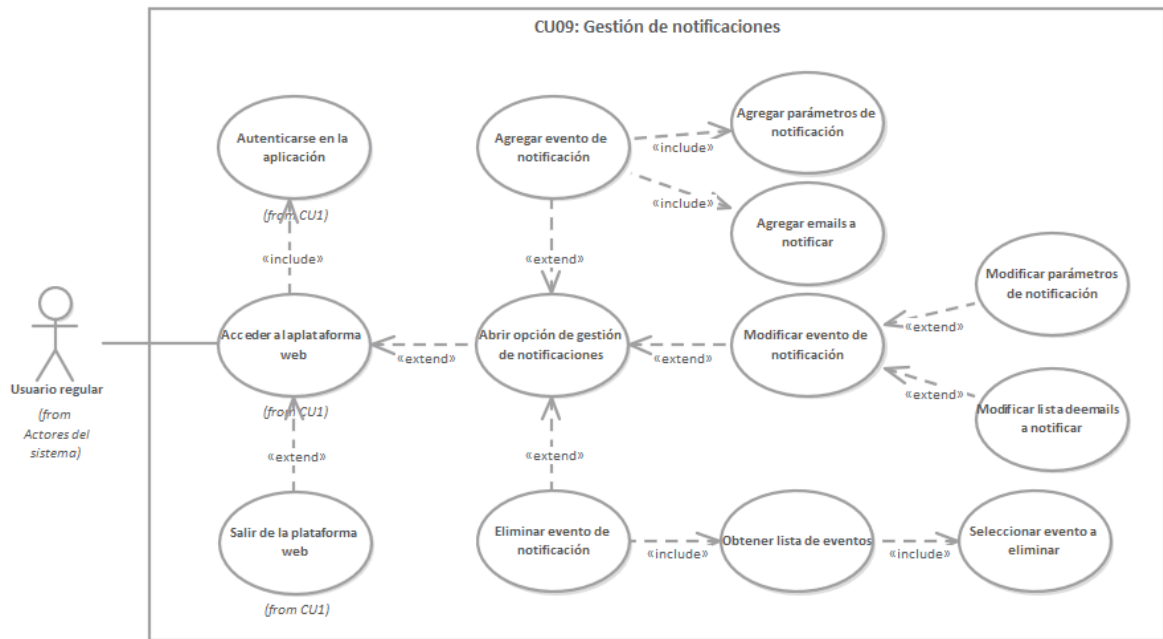


Figura 46. Diagrama de casos de uso CU09 - Gestión de notificaciones, desde la perspectiva del usuario regular.

5.5.10 CU10 - Gestión de proveedores de repositorios

Dado que el código de cada algoritmo, como se mostró en el caso de uso número uno (CU01), debe ser integrado mediante un repositorio de código, el administrador del sistema deberá tener la capacidad de administrar los proveedores con los que la plataforma se puede integrar.

Como se ilustra en la figura 47, una vez que el administrador se ha autenticado en la plataforma, tiene la opción de gestionar los proveedores de repositorios. Cada proveedor tiene ciertos parámetros de integración que pueden ser agregados, modificados o eliminados. Esta información se guarda en la base de datos o en algún medio de persistencia de la configuración.

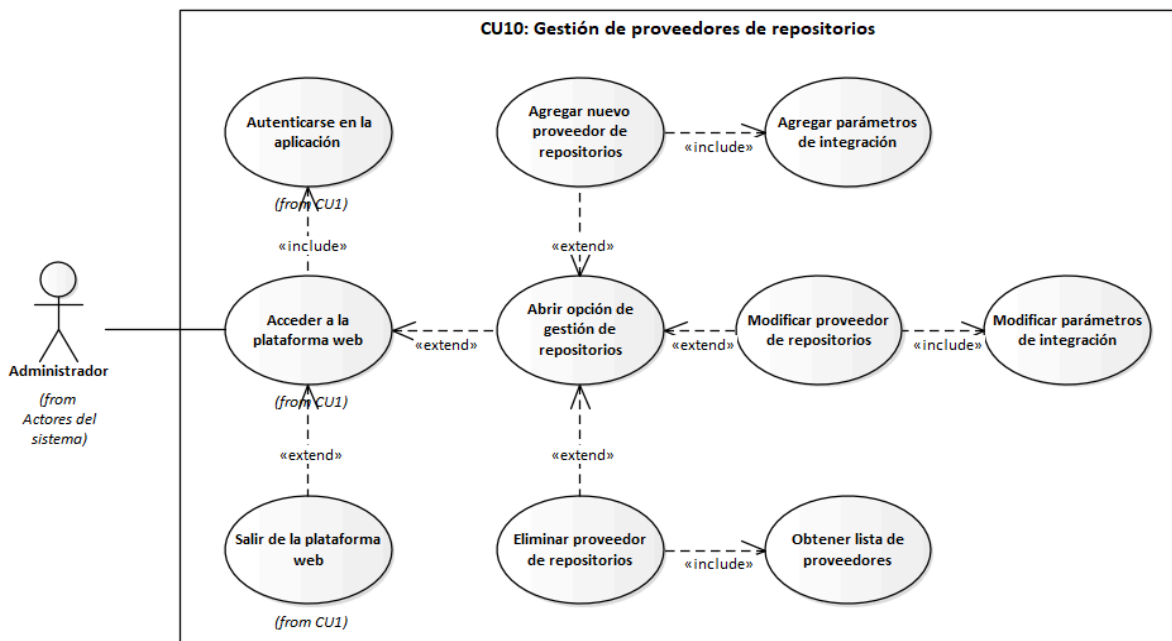


Figura 47. Diagrama de casos de uso CU10- Gestión de proveedores de repositorios, desde la perspectiva del usuario administrador.

5.5.11 CU11 - Gestión de proveedores de análisis de código

Dado que la plataforma vinculará y ejecutará código externo, se deberá realizar un análisis en busca de posibles vulnerabilidades del código a vincular. Para esto, existen múltiples proveedores que ofrecen herramientas de análisis de seguridad, cada uno con sus propios parámetros de integración y formato de las reglas que se cargan.

Algunas de las funcionalidades a las que tiene acceso el administrador son, como se muestra en la figura 48, agregar, editar y eliminar proveedores de análisis de código. Adicionalmente, puede administrar las reglas de análisis de código que se cargan para cada proveedor.

Pueden existir casos en el que el análisis de código no es necesario o no es posible, para dichos casos existe la posibilidad de deshabilitar/habilitar el escaneo obligatorio de código por grupos de usuarios.

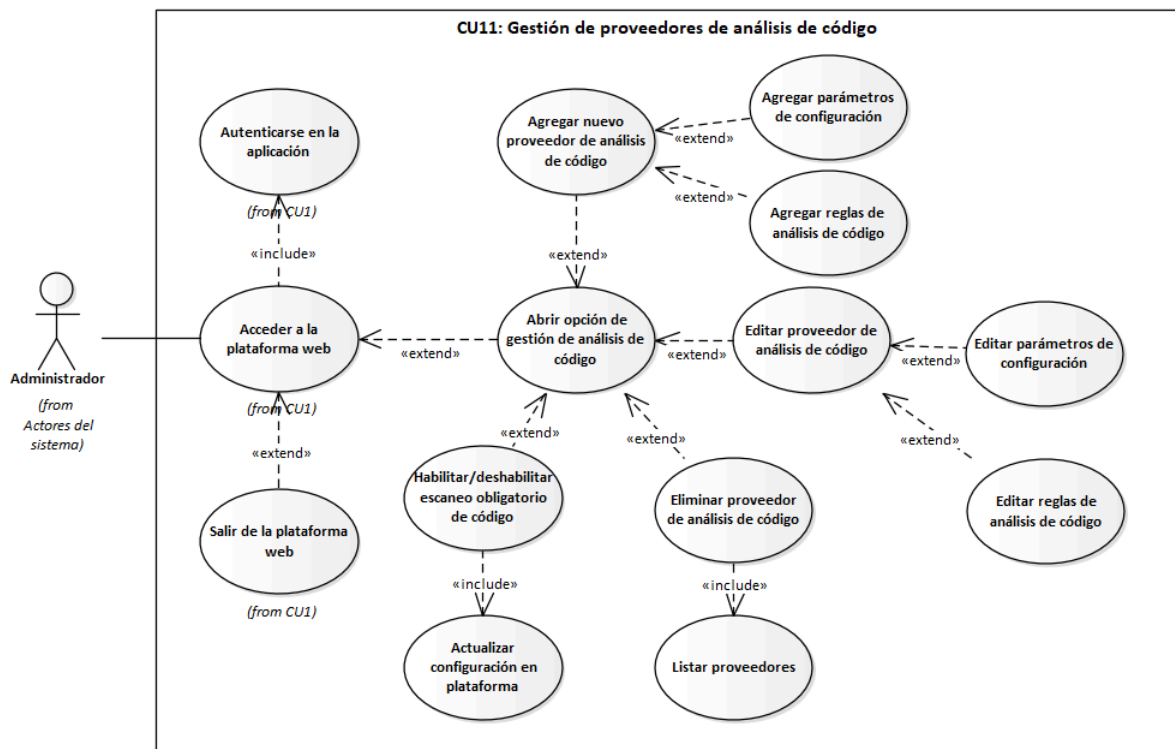


Figura 48. Diagrama de casos de uso CU011- Gestión de proveedores de análisis de código, desde la perspectiva del usuario administrador.

5.6 ARQUITECTURA DE LA APLICACIÓN

La siguiente sección se centra en la definición de la arquitectura de la aplicación y su funcionamiento a alto nivel.

Como se puede observar en la figura 49, empezando de izquierda a derecha, el usuario accede a la plataforma desde una terminal y se comunica con el proveedor de identidad para obtener un token JWT, ingresando usuario y contraseña, siguiendo el protocolo OpenID Connect (OIDC) y OAuth2.0. Una vez que el cliente obtiene el token, lo incluye en la cabecera de la petición HTTPS para acceder a la aplicación. Todas las peticiones entrantes, tanto a la capa web como a la capa de servicios de composición, se realizarán a través de un "gateway" o puerta de enlace.

La arquitectura sigue el patrón arquitectónico de publicador/suscriptor. Gracias al uso del "bus" de eventos, cada servicio se suscribe y recibe mensajes según sea necesario. De esta manera, se garantiza el funcionamiento asíncrono de los servicios y se desacoplan los mismos.

5.6.1 Autenticación

Dentro de la capa de autenticación se tienen principalmente dos componentes. El primer componente es el proveedor de identidad, que será el encargado de validar la identidad del usuario y retornar un token JWT utilizando el protocolo de autenticación OAuth2.0/OIDC. Este token tendrá los permisos de los servicios a los que el usuario puede acceder.

El segundo componente es el repositorio de usuarios. En este repositorio se guardarán los datos de los usuarios, la información de los grupos a los que pertenecen y los permisos que tienen para consumir los diferentes servicios del sistema.

5.6.2 Servicio de composición de IGU

La parte web o "frontend" tendrá su propia base de datos en caché para mantener los estados que se mostrarán al usuario. Por otro lado, en la capa de "servicios de composición de GUI (Interfaz Gráfica de Usuario)", habrá un único componente que

permitirá la composición de la parte visual. Dichos componentes se describen a continuación.

- **Componente IGU:** está a cargo de la generación de las diferentes interfaces o pantallas que se muestran al usuario final. Por otro lado, se comunica con algunos servicios internos para la vinculación y gestión del catálogo de algoritmos.

5.6.3 *Microservicios Internos*

La parte que soporta la lógica de la aplicación o "backend" estará compuesta por microservicios que se comunican mediante un bus de eventos. Cada microservicio tendrá su propia base de datos para mantener el estado y la configuración del servicio que proporciona.

A continuación, se describe cada una de las funcionalidades de los servicios a desarrollar.

5.6.3.1 *Microservicios de vinculación y ejecución de código:*

Los servicios que se presentan en color naranja, en la figura 49, son servicios que se encargarán de la gestión de la configuración y ejecución de los algoritmos a vincular en la plataforma.

- **Microservicio de vinculación de código:** Este servicio es el encargado de tomar el código alojado en el repositorio del cliente y agregarlo a la plataforma de manera que pueda ser integrado y ejecutado posteriormente.
- **Microservicio de gestión de entornos de ejecución:** En el caso de uso CU03, se establece la necesidad de ejecutar el código que se vincula a la plataforma. Dicha ejecución requiere de un ambiente con los requerimientos de hardware y software necesarios para poder correr. Es por esto que el servicio correspondiente se encargará de mantener la configuración de dichos ambientes y de lanzar la ejecución de los algoritmos almacenados en el catálogo. Además, el repositorio del código deberá incluir un archivo de Docker Compose para poder generar el entorno de ejecución.

- **Microservicio de detección de vulnerabilidades en el código:** Dado que el código se va a incluir en la plataforma, se debe establecer un mecanismo para el escaneo de posibles vulnerabilidades. Para ello, es necesario contar con un servicio encargado de realizar un análisis estático del código en busca de posibles brechas de seguridad y almacenar únicamente el código que no presente vulnerabilidades.
- **Servicio de gestión de catálogo:** Este servicio es responsable de mantener la lista de algoritmos, tanto públicos como privados, y de generar el mensaje necesario para su ejecución. Además, tiene un estado interno que permite determinar si un algoritmo está en fase de entrenamiento o no.

5.6.3.2 Microservicios de gestión de la plataforma:

Los servicios que se presentan en color morado en la figura 49, son servicios que se encargarán de la gestión de la configuración de la plataforma, por ejemplo, la configuración de notificaciones o del catálogo de algoritmos.

- **Microservicio de gestión de parámetros de las API (Application Programming Interface) a generar:** Dado que cada algoritmo tiene ciertos parámetros de entrada y salida, el usuario que desarrolla el algoritmo debe especificar estas características al vincular el código. El trabajo de este servicio es gestionar estos parámetros de manera que, una vez generado el servicio del algoritmo de ML a consumir, el usuario consumidor pueda conocer los parámetros que debe proporcionar y el tipo de respuesta que debe esperar.
- **Microservicio de gestión de notificaciones:** En el caso de uso CU09, se establece la necesidad de administrar los eventos de notificación para cada parte del proceso, por ejemplo, cuando una vinculación es exitosa o cuando una ejecución ha finalizado. Este servicio se encarga de dicha administración.
- **Servicio de gestión de usuarios:** Siguiendo lo descrito en el CU05, este servicio se encargará de la gestión de los usuarios en la plataforma, así como de asignar y gestionar los roles que cada uno de ellos tendrá dentro de la misma.
- **Microservicio de gestión de proveedores de repositorios:** El caso de uso CU10 plantea la necesidad de administrar los diferentes proveedores de repositorios con los que la plataforma se puede comunicar. Por tal motivo, este servicio se encarga de mantener y modificar dicha configuración.

5.6.3.3 Microservicios de resultados:

Los servicios que se presentan en color azul claro en la figura 49, son servicios que se encargarán de las métricas y gestión de resultados arrojados por los diferentes algoritmos de ML vinculados a la plataforma.

- **Microservicio de métricas:** Este servicio es el encargado de mantener las métricas de rendimiento que se pueden aplicar sobre los algoritmos en el catálogo. Inicialmente, deberá tener solo las métricas presentadas en la sección 4.1 y almacenar sus entradas y salidas en la base de datos correspondiente.
- **Microservicio de gestión de resultados:** Una vez que el usuario ha utilizado alguno de los algoritmos del catálogo, es posible que desee conocer los resultados. Debido a que cada algoritmo produce diferentes salidas, este servicio se encargará de presentar los resultados de acuerdo con la configuración establecida en el momento de la vinculación. Además, enviará eventos para que el servicio de composición de pantallas de visualización de resultados pueda desplegar los elementos gráficos correspondientes.

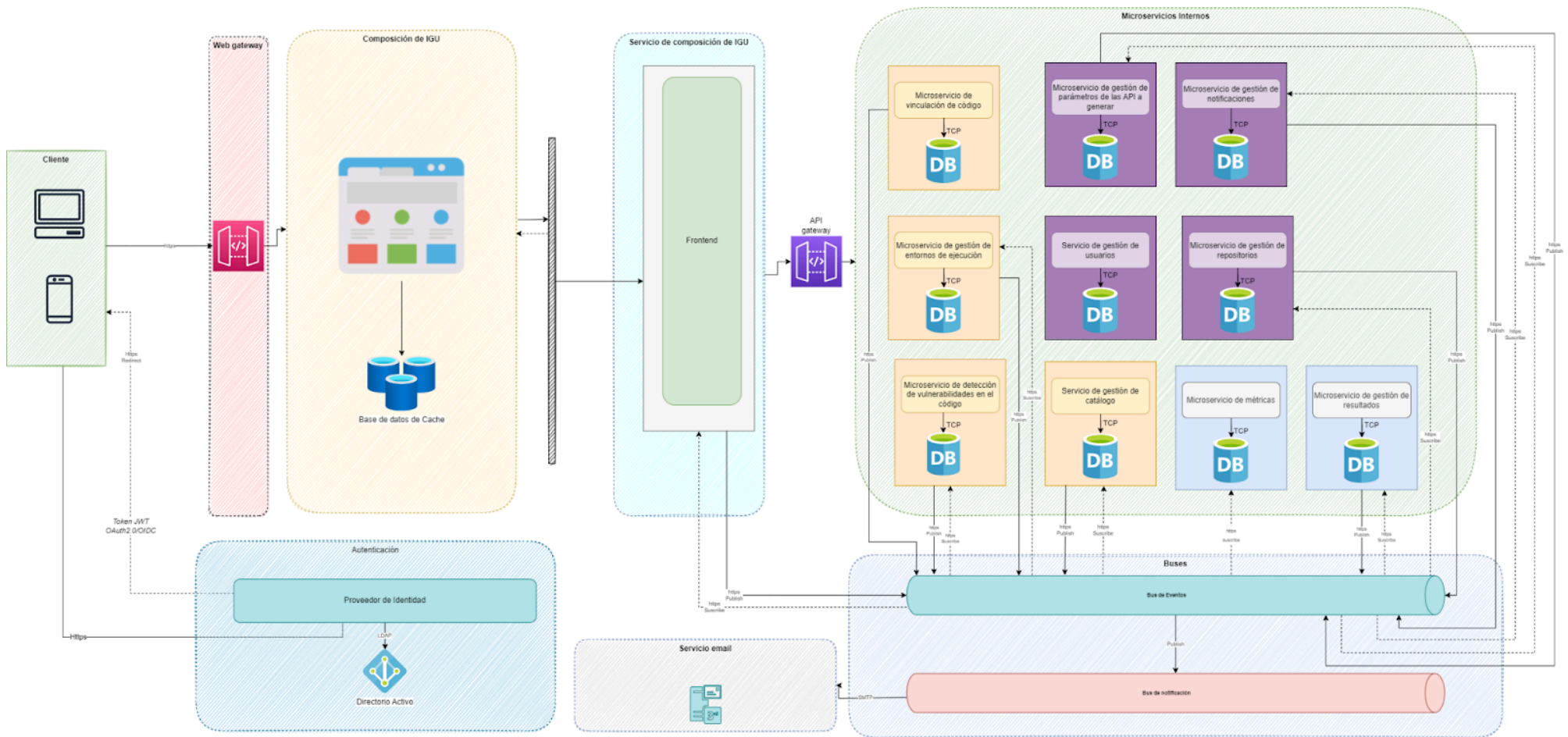


Figura 49. Arquitectura de la aplicación utilizando Bus de eventos y microservicios.

5.6.4 Tópicos y suscriptores

Ya que la arquitectura de la aplicación ha sido diseñada de manera que los microservicios se comuniquen a través de un bus de eventos que tiene determinados tópicos a los que pueden suscribirse los diferentes componentes del sistema y que serán notificados cuando llegue un nuevo mensaje, se describen a continuación los tópicos, los suscriptores y los generadores de los mensajes. Además, se incluye la ruta del servicio que el tópico deberá notificar.

Tópicos		
Nombre	Suscriptores (componentes)	Endpoint
Analisis_codigo_input	Analizador_codigo	/analizar
Analisis_codigo_output	VinculadorCodigo	/actualizarVinculacion
	Generador_entornos_ejecucion	/generarImagen
	GestorParametrosAPI	/saveAPIConfig
	GestorNotificaciones	/notificarEvento
Generacion_Imagen_output	VinculadorCodigo	/actualizarVinculacion
	GestorNotificaciones	/notificarEvento
Gestor_param_API_output	VinculadorCodigo	/actualizarVinculacion
Vinculador_codigo_output	GestorCatalogo	/agregarCatalogoGral
MiCatalogo_input	GestorCatalogo	/agregarMiCatalogo
Generacion_entorno_input	Generador_entornos_ejecucion	/generarEntorno
Generacion_entorno_output	GestorCatalogo	/actualizarMiCatalogo
	GestorNotificaciones	/notificarEvento
MiCatalogo_modelo_input	GestorCatalogo	/generarLlamadoModeloMI
Llamar_modelo_input	GestorResultados	/entrenarModelo
Resultado_modelo_output	GestorCatalogo	/actualizarMiCatalogo
	Metricas	/calcularMetricas
	GestorNotificaciones	/notificarEvento

En cuanto a los generadores de los mensajes, se presenta la siguiente con la información de los componentes notificadores, los tópicos a donde notifican y el ejemplo de mensaje que se envía.

Notificación origen/destino		
Origen (componente)	Destino (Tópico)	Mensaje notificación
VinculadorCodigo	Analisis_codigo_input	<pre>{ "nombreApp": "<App name>", "version": "<App version>", "Mensaje": { "esPublico": true/false, "linkRepo": "<LinkRepo>", "branchRepoName": "<Branch_name>", "idUserario": "<Usuario ID>", "analizadorUsar": "Sonar"/"PMD", } }</pre>
Analizador_codigo	Analisis_codigo_output	<pre>{ "nombreApp": "<App name>", "version": "<App_version>", "idTransaccion": "<ID_generado>", "Mensaje": { "linkRepo": "<LinkRepo>", "branchRepoName": "<Branch_name>", "lastCommithash": "<lastCommithash>", "idUserario": "<Usuario ID>", "analizadorUsar": "Sonar"/"PMD", "codigoVulnerable": false/true, "listaVulnerabilidades": [{ "tipoVulnerabilidad": "<tipo>", "descripcion": "<descripcion>", "severidad": "<severidad>" }, { "tipoVulnerabilidad": "<tipo>", "descripcion": "<descripcion>", "severidad": "<severidad>" }] } }</pre>

Generador_entornos_ejecucion	Generacion_Imagen_output	{ "nombreApp": "<App name>", "version": "<App_version>", "idTransaccion": "<ID_generado>", "Mensaje": { "linkRepo": "<LinkRepo>", "branchRepoName": "<Branch_name>", "lastCommitHash": "<lastCommitHash>", "idUsuario": "<Usuario ID>", "imageTag": "<tag de imagen>", "imagenGenerada": true/false } }
GestorParametrosAPI	Gestor_param_API_output	{ "nombreApp": "<App name>", "version": "<App_version>", "idTransaccion": "<ID_generado>", "Mensaje": { "linkRepo": "<LinkRepo>", "branchRepoName": "<Branch_name>", "lastCommitHash": "<lastCommitHash>", "idUsuario": "<Usuario ID>", "openAPIFileLink": "<link_archivo_guardado>", "openAPIFileCorrect": true/false } }
VinculadorCodigo	Vinculador_codigo_output	{ "nombreApp": "<App name>", "version": "<App version>", "Mensaje": { "esPublico": true/false, "idUsuario": "<Usuario ID>" } }
Frontend	MiCatalogo_input	{ "nombreApp": "<App name>", "version": "<App version>", "idUsuario": "<Usuario ID>" "idTransaccion": "<Generado_por_originador>", "openAPIFileContent": "<Contenido_Open_API>" }
GestorCatalogo	Generacion_entorno_input	{ "nombreApp": "<App name>", "version": "<App version>", "idUsuario": "<Usuario ID>" }

Frontend	MiCatalogo_modelo_input	<pre>{ "nombreApp": "<App name>", "version": "<App version>", "idUser": "<Usuario ID>" "idTransaccion": "<Generado_por_originador>", "metricasCalcular": ["RMSE",/ "MAE",/ "MAPE"/], "IP_API": "<IP_API>", "numeroPuerto": <Puerto>, "openAPIFileContent": "<Contenido_B64>", "datasetLink": "Link_archivo" }</pre>
GestorCatalogo	Llamar_modelo_input	<pre>{ "nombreApp": "<App name>", "version": "<App version>", "idUser": "<Usuario ID>" "idTransaccion": "<Generado_por_originador>", "consecutivo": <consecutivo>, "metricasCalcular": ["RMSE",/ "MAE",/ "MAPE"], "IP_API": "<IP>", "numeroPuerto": <puerto>, "openAPIFileContent": "<JSON B64>", "datasetLink": "<Link_archivo>", "endPoint": "<endpoint_llamar>" }</pre>
GestorResultados	Resultado_modelo_output	<pre>{ "nombreApp": "<App name>", "version": "<App version>", "idUser": "<Usuario ID>" "idTransaccion": "<Generado_por_originador>", "consecutivo": <consecutivo>, "valoresReales": [<arreglo_valores_reales>], "valoresCalculados": [<arreglo_valores_calculados>] "metricasCalcular": ["RMSE",/ "MAE",/ "MAPE"] }</pre>

5.7 DISEÑO DE LOS COMPONENTES DE LA APLICACIÓN

A continuación, se describe de manera estructural cada uno de los componentes de la solución. Adicionalmente, se describe el modelo de datos de la base de datos para cada componente.

5.7.1 Microservicio de detección de vulnerabilidades en el código

5.7.1.1 Modelo estructural

Tal como se muestra en la figura 50, se realiza el bosquejo del diagrama de clases para el microservicio encargado de analizar el código a vincular en busca de vulnerabilidades.

Este servicio se implementará utilizando el framework "Spring Boot". La estructura estará conformada por una clase "Controller" encargada de recibir y despachar solicitudes a través del protocolo HTTP, una clase que implemente la lógica del servicio que se ofrece y una capa lógica que contenga la implementación del análisis del código.

Por otro lado, este servicio utiliza el patrón fachada para ocultar el llamado a los diferentes proveedores de análisis de código, como PMD y Sonar. Debido a que cada proveedor tiene una lógica diferente para cada llamado a sus implementaciones, se utiliza el patrón adaptador de objeto para tomar el mensaje de entrada, transformarlo a las necesidades de cada SDK (kit de desarrollo de software) y retornar una respuesta a la capa de servicio.

Por último, se implementa el patrón Método fábrica para obtener los diferentes adaptadores de análisis dependiendo del valor que llega en el objeto Mensaje.

Microservicio de análisis de código

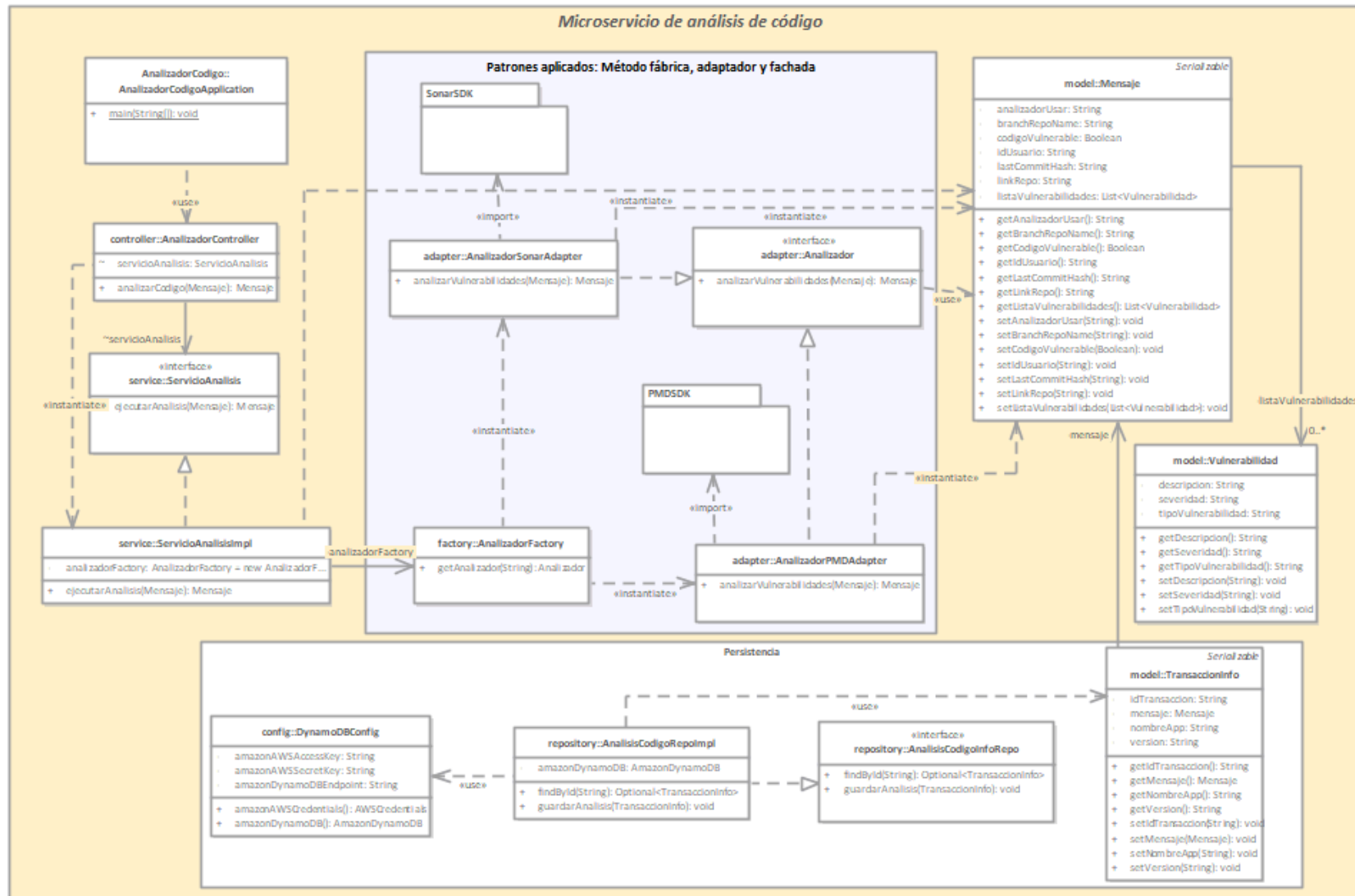


Figura 50. Diagrama de clases para el microservicio de análisis de vulnerabilidades en el código.

5.7.1.2 Modelo de datos

Este componente, como se mostró en el capítulo anterior, utilizará una base de datos interna para almacenar los resultados de sus funciones. Debido a que los datos que se deben almacenar no tienen mayor complejidad, solo consta de una tabla y los mensajes de notificación vienen en formato JSON, se decide utilizar una base de datos NoSQL documental con formato de almacenamiento JSON.

Como se muestra en la figura 51, se tendrá un documento con índice (pk) compuesto por: “nombreApp”, “version”, “idTransaccion”. Además, este documento se compone de un mensaje que a su vez tiene una lista de vulnerabilidades. En la figura mencionada se puede apreciar la estructura y un documento de ejemplo.

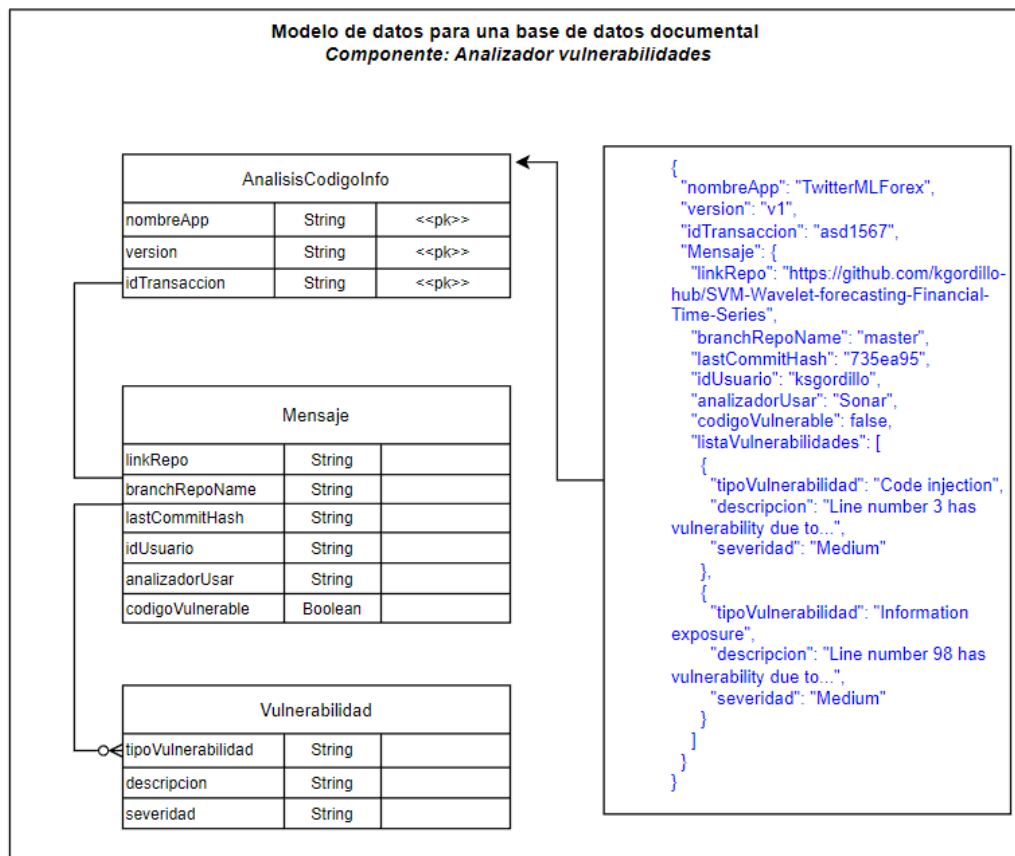


Figura 51. Modelo de datos para base de datos documental del componente de análisis de código.

5.7.2 Microservicio de generación de entornos de ejecución

5.7.2.1 Modelo estructural

Este microservicio tiene dos funcionalidades: generar la imagen Docker del algoritmo y generar el ambiente donde se corre dicha imagen.

La primera funcionalidad tiene la tarea de ejecutar cuatro pasos principales. En primer lugar, se clona en el ambiente local el repositorio que se pasa como mensaje. En segundo lugar, se lanza la ejecución del comando “Docker build” para la generación de la imagen docker localmente. En tercer lugar, se pone una etiqueta a la imagen generada en el paso dos. Por último, se ejecuta el comando de Docker para publicar la imagen en un repositorio externo.

La segunda funcionalidad, por otro lado, se encarga de descargar la imagen del repositorio que se quiere ejecutar, provisionar un entorno de ejecución y actualizar la base de datos con la información de la dirección IP y puerto donde se aloja la aplicación.

Este componente fue diseñado aplicando el patrón Comando. Dicho patrón nos permite encolar y gestionar la ejecución de comandos, lo que facilita la planificación y la restauración de las operaciones.

Se eligió este patrón para solucionar el problema de restauración a un estado anterior cuando alguno de los pasos listados anteriormente no se ejecute correctamente. Por ejemplo, si el paso uno falla, se deberá eliminar la carpeta local (si es que existe) para liberar espacio en disco y evitar problemas futuros.

Como se muestra en la figura 52, se crea una clase abstracta "Command" con las funciones de ejecutar y restaurar ("rollback"). Cada uno de los comandos específicos extiende esta clase e implementa su propia lógica en los métodos abstractos. Además, se tiene una clase que mantiene una lista en memoria de los comandos ejecutados llamada "CommandHistory". Por último, la clase "Invoker" se encarga de realizar el llamado a los diferentes comandos y de ejecutar una restauración total en caso de ser necesario.

Como requisito para vincular un nuevo algoritmo, el repositorio del código deberá tener un archivo Docker que especifica la creación del entorno virtual a crear. El archivo deberá llevar por nombre “Dockerfile” y estar en la raíz del directorio principal.

5.7.2.2 Modelo de datos

La base de datos para este servicio será una base de datos documental que sigue la estructura de la figura 53. Se tiene un documento llamado “ImagenDockerInfo” donde la partición primaria está conformada por: “nombreApp”, “versión” e “idTransacción”. Este último atributo se agrega ya que la generación del entorno puede fallar o ser exitoso en cada llamado, es por esto que se debe asociar a una transacción.

Por otro lado, se tiene el documento “AmbienteDockerInfo”, está diseñado para que contenga la información de dónde queda alojada la aplicación (dirección IP y puerto).

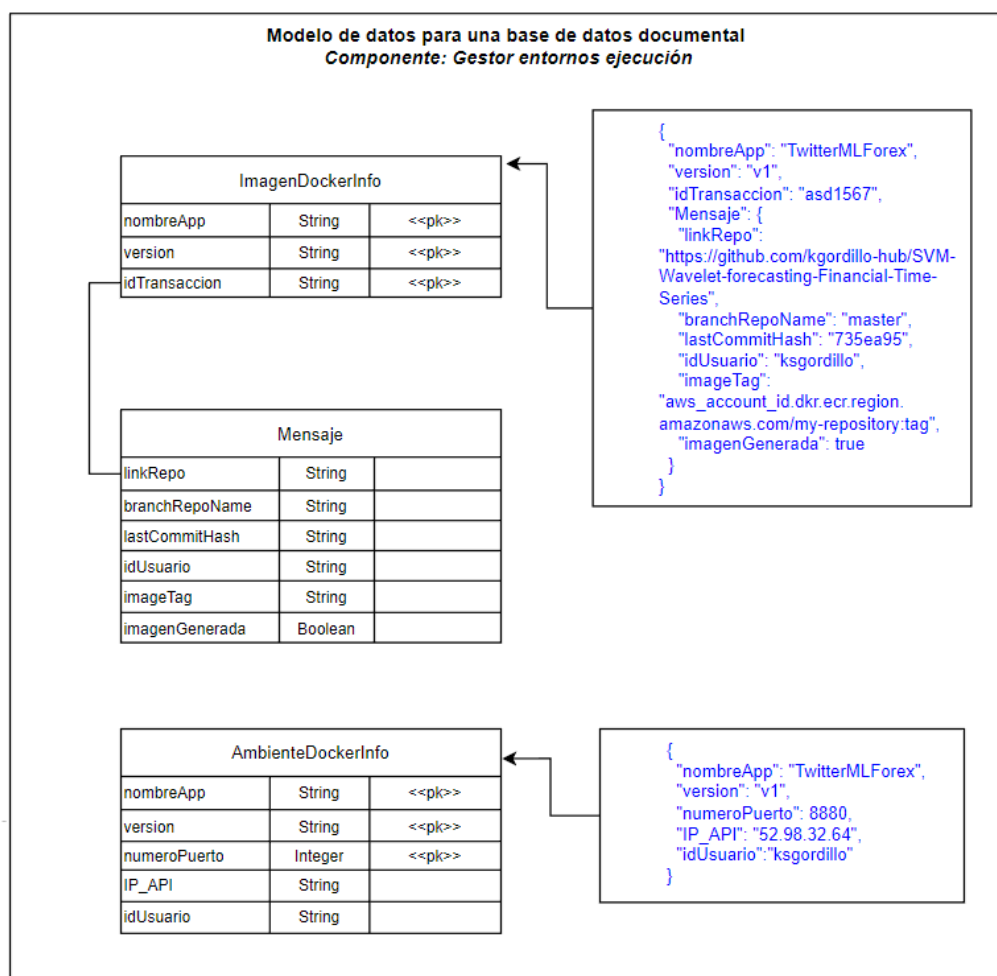


Figura 53. Modelo de datos para base de datos documental del componente de gestor de entornos de ejecución.

5.7.3 Microservicio de gestión de catálogo

5.7.3.1 Modelo estructural

Dentro del sistema, existen dos catálogos: el catálogo general y el catálogo personal. El catálogo general contiene toda la información acerca de los algoritmos que están vinculados a la plataforma. Por otro lado, el catálogo personal contiene únicamente la información de los algoritmos que interesan al usuario.

Una vez que se ha vinculado correctamente un algoritmo, éste se agrega al catálogo general. Luego, cuando el usuario quiera utilizar uno de estos algoritmos, lo agrega al catálogo personal desde donde podrá entrenar el modelo y hacer llamados a sus métodos.

Este servicio, como se muestra en la figura 54, permite agregar, actualizar y consultar la lista de algoritmos generales y los que se han agregado al catálogo personal. Además, integra una funcionalidad para generar el llamado al algoritmo de ML en concreto que quiere utilizar el usuario.

La estructura en general, al igual que los demás servicios, se compone de un controlador, un servicio y una capa de persistencia para guardar los datos en una base de datos documental.

para determinar si solamente es visible para la organización o, si por el contrario, todo usuario de la plataforma lo puede ver.

En cuanto al siguiente documento “MiCatalogoInfo”, únicamente tiene la información de los algoritmos asociados al usuario y el estado en el que se encuentra. Por ejemplo, puede estar en preparación cuando se está aprovisionando el ambiente de ejecución. O puede estar en entrenamiento cuando está haciendo cálculos con los datos de entrenamiento.

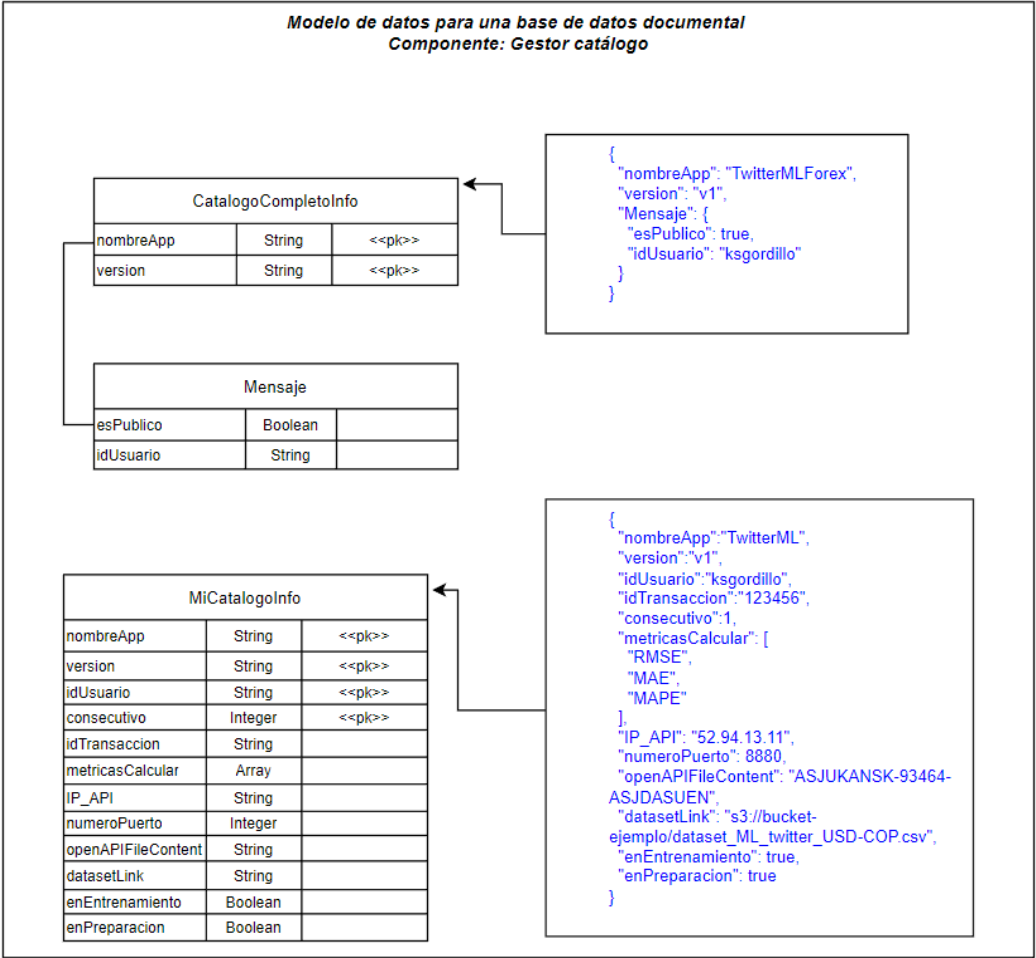


Figura 55. Modelo de datos para base de datos documental del componente de gestión de catálogo.

5.7.4 Microservicio de vinculación de código

5.7.4.1 Modelo estructural

La figura 57 muestra el modelo de datos para una base de datos documental que se diseñó de manera tal que lleve el registro del estado de cada uno de estos filtros utilizando, principalmente, tres atributos: “codigoVulnerable”, “openAPIFileCorrect” y “ImagenGenerada”. La llave principal de este documento está conformada por el nombre de la aplicación y su versión.

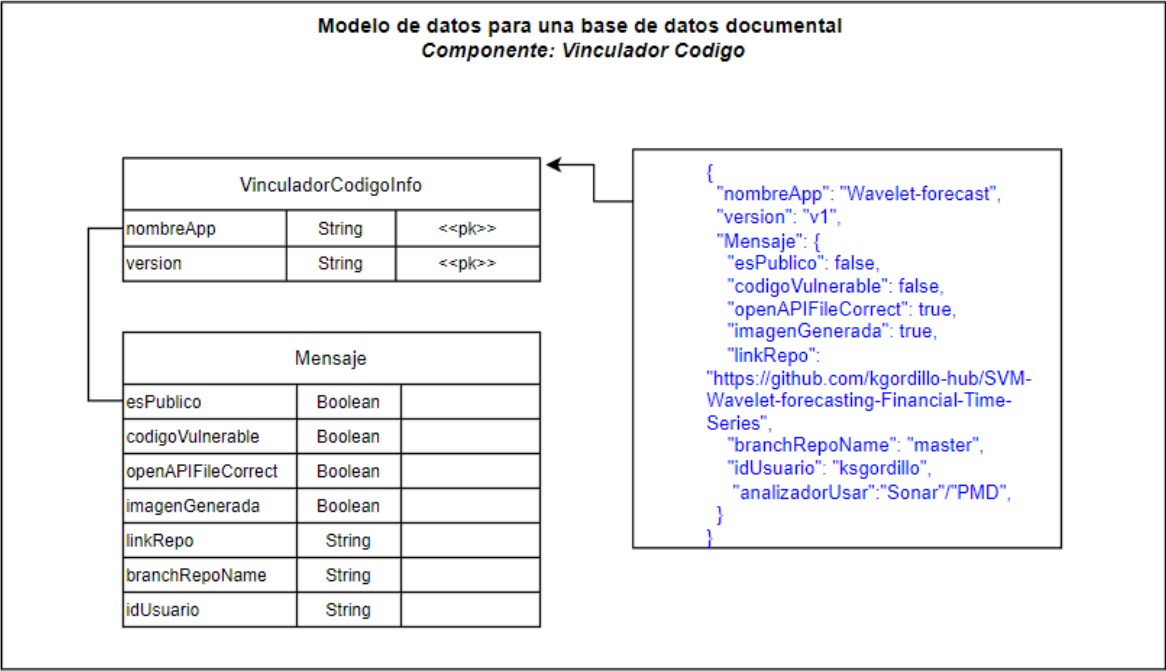


Figura 57. Modelo de datos para base de datos documental del componente de vinculación de código.

5.7.5 Microservicio de gestión de parámetros API

5.7.5.1 Modelo estructural

Este servicio, como se mencionó anteriormente, permite almacenar y gestionar la especificación del API a vincular en la plataforma de manera tal que, los usuarios que quieran hacer uso del algoritmo de ML vinculado puedan saber los métodos, entradas y salidas que pueden usar (OpenAPI Initiative, s.f.).

Se decide utilizar OpenAPI para dicha especificación. OpenAPI, como lo menciona su página web, nos permite realizar la descripción de una API remota que puede ser llamada mediante los protocolos HTTP o basados en HTTP y nos permite dar a conocer al consumidor el cómo utilizar la API.

Dentro de los requisitos para vincular un nuevo algoritmo, la especificación OpenAPI deberá estar contenida en un archivo con extensión ".json". Este archivo deberá estar ubicado en la raíz del repositorio y tendrá el nombre "SpecOpenAPI.json".

Como se muestra en la figura 58, este servicio tiene una función muy sencilla y es persistir el archivo de especificación en una base de datos utilizando los marcos de trabajo Spring Boot y AWS DynamoDB. Una vez se recibe el mensaje, se clona el repositorio en algún directorio local, se extrae el archivo y se sube a algún repositorio de archivos remoto. Una vez se ha subido el archivo, se procede a persistir el enlace y los datos del repositorio en la base de datos.

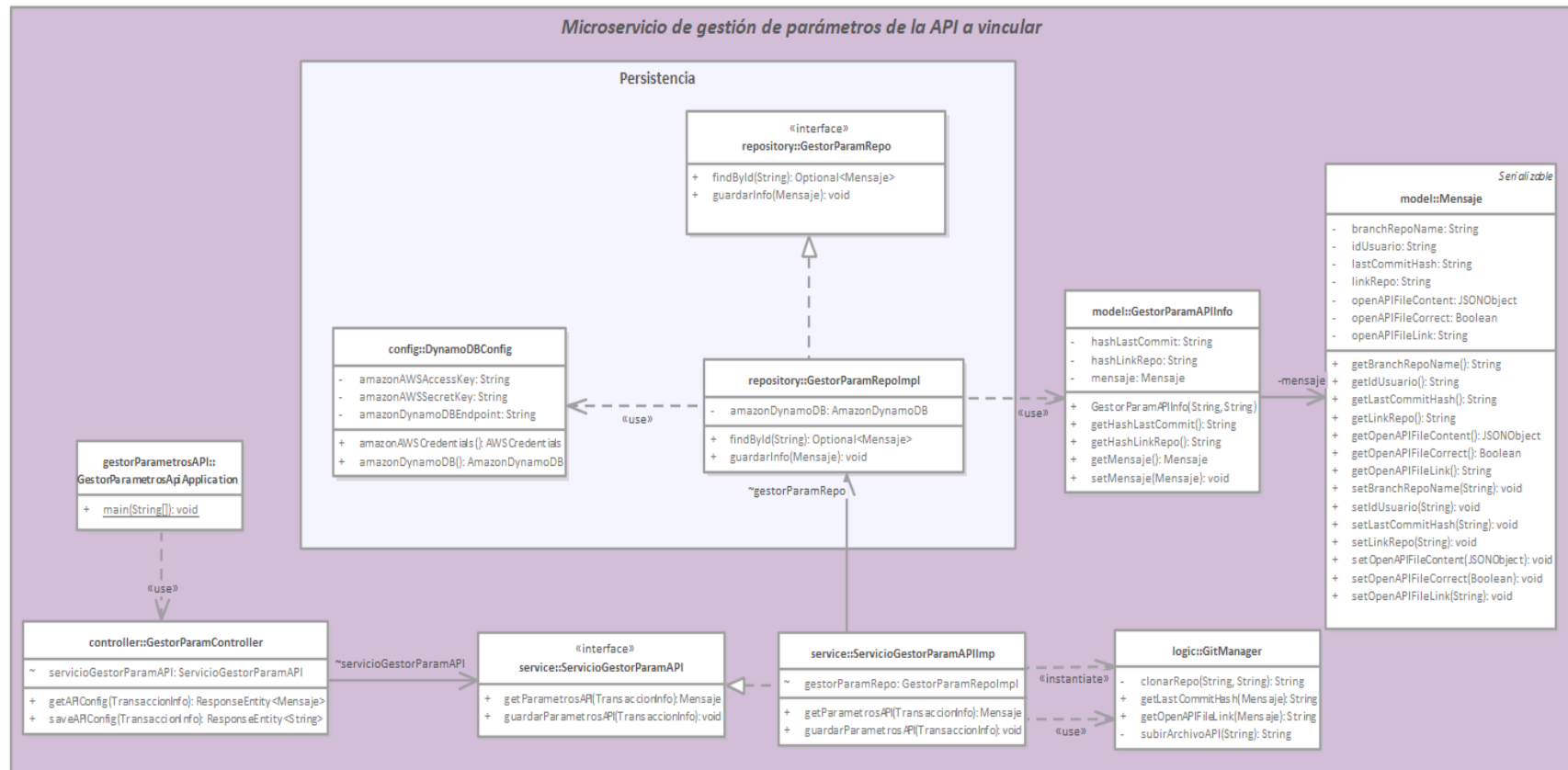


Figura 58. Diagrama de clases para el microservicio de gestión de parámetros de la API a vincular.

5.7.5.2 Modelo de datos

El modelo de datos que se diseñó para este servicio se detalla en la figura 59. La llave principal es el nombre de la aplicación, la versión y el id de la transacción del algoritmos que se está vinculando.

La base de datos documental tiene como función principal, guardar la referencia al archivo de especificación de la API que se está vinculando.

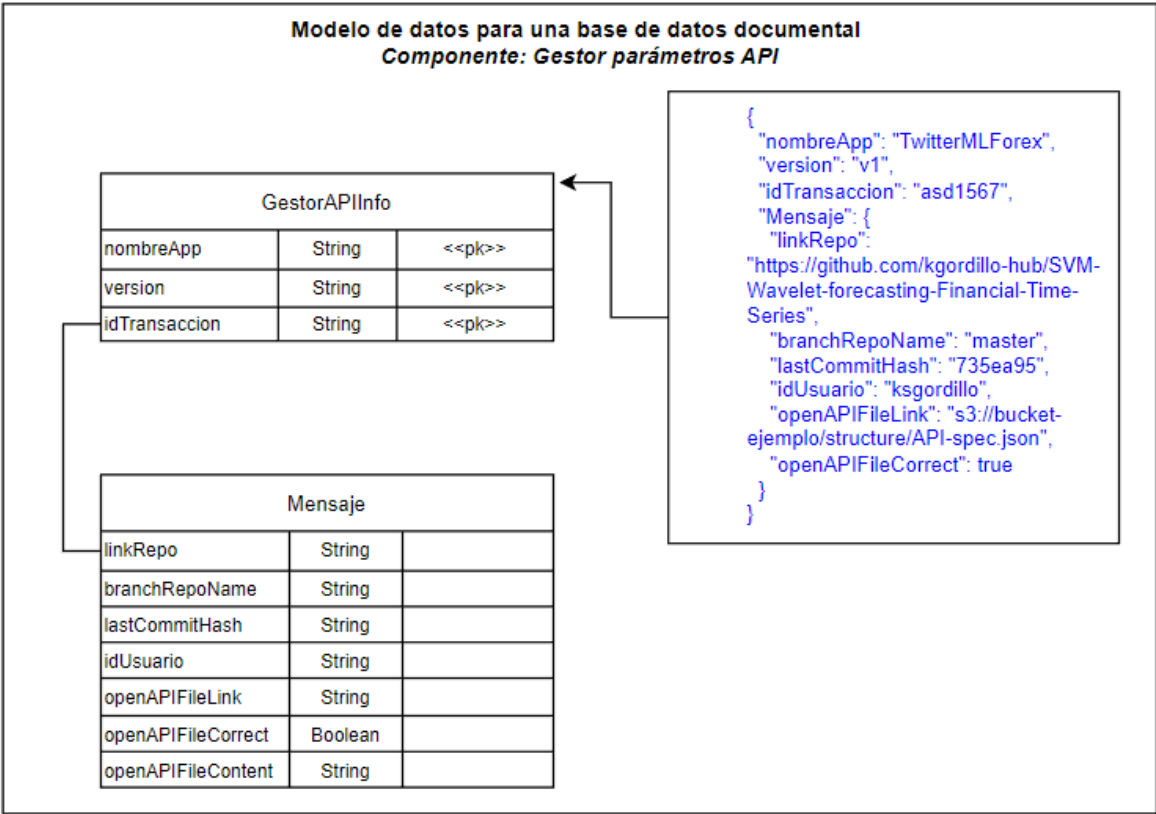


Figura 59. Modelo de datos para base de datos documental del componente de gestión de parámetros de la API de ML.

5.7.6 Microservicio de gestión de repositorios

5.7.6.1 Modelo estructural

Este servicio se encarga de mantener un registro de los proveedores de repositorios que se pueden comunicar con la plataforma, por ejemplo: github, bitbucket, azure repos, etc.

La estructura del servicio es muy sencilla. Consiste en una clase controladora que maneja las peticiones del cliente. Un servicio que permite la conexión e interacción con una base de datos documental y, una capa de persistencia.

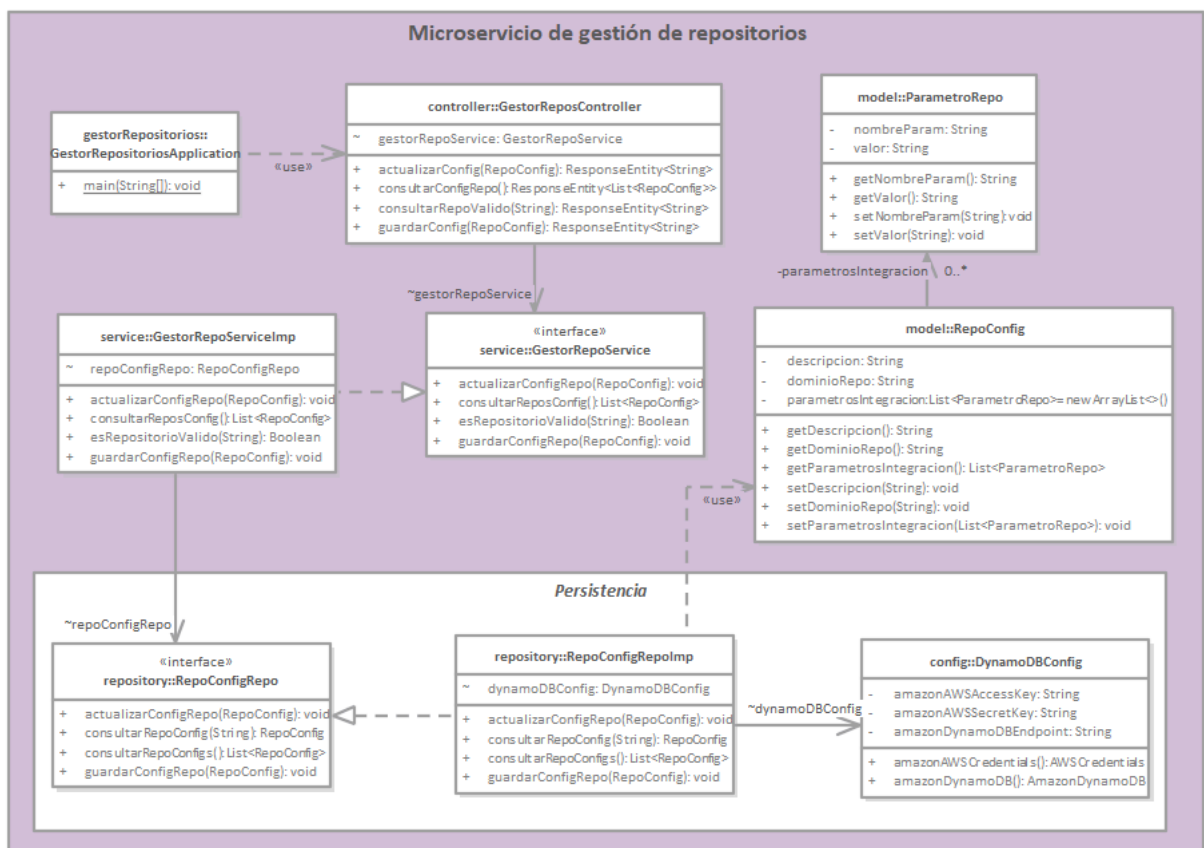


Figura 60. Diagrama de clases para el microservicio de gestión de repositorios.

5.7.6.2 Modelo de datos

El modelo de datos que se describe en la figura 61 está diseñado para una base de datos documental y contiene un documento llamado “RepoConfigInfo” que tiene una colección de elementos que representan los proveedores de repositorios permitidos en el sistema.

La llave principal del documento es el dominio del repositorio. Por ejemplo, para github.com, la llave sería “github”. Por otro lado, se tiene que cada configuración tiene una serie de parámetros de integración representados en la estructura de “parametrosIntegracion”.

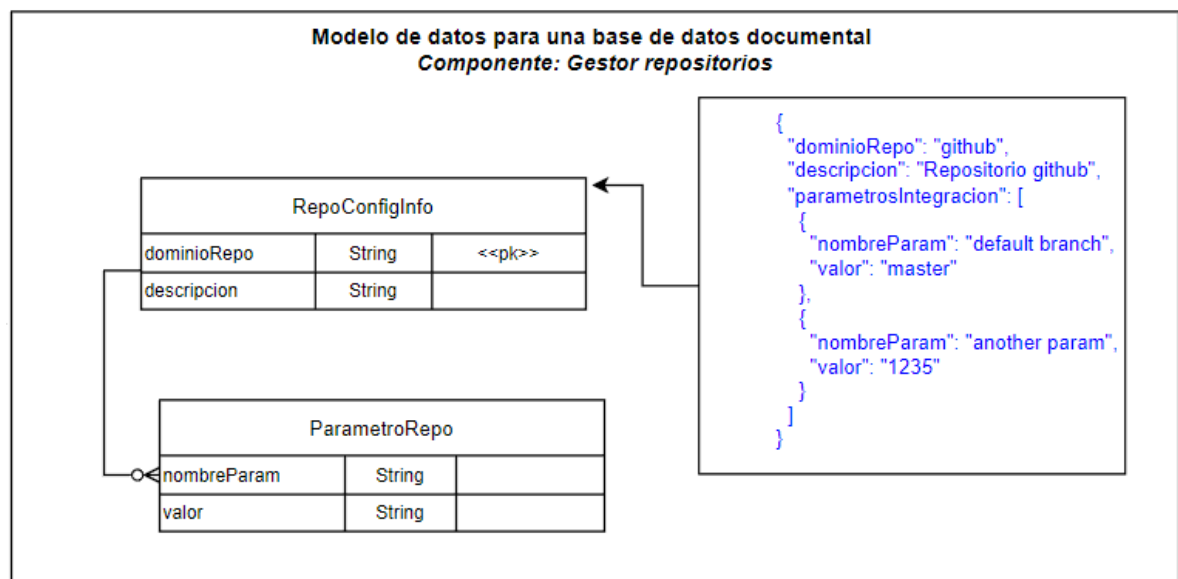


Figura 61. Modelo de datos para base de datos documental del componente de gestión de repositorios.

5.7.7 Microservicio de gestión de notificaciones

5.7.7.1 Modelo estructural

Dentro del sistema existen varios eventos que deben ser notificados según lo requiera el usuario. Por ejemplo, cada vez que el componente de gestor de entornos termine de aprovisionar un ambiente virtual para correr el algoritmo de ML seleccionado, se puede notificar al usuario para que no tenga que esperar en pantalla a que el componente termine.

Los procesos de entrenamiento de los algoritmos de ML pueden demorar varios minutos. En estos casos se puede notificar cuando el algoritmo esté entrenado y listo para recibir información. De esta manera, se tiene un sistema asíncrono y basado en eventos

El microservicio de gestión de notificaciones fue diseñado para que esté al tanto de los eventos dentro del sistema, construya la notificación y la envíe a una cola de correo. Como se puede ver en la figura 62, se hace uso del patrón decorador para la construcción de los mensajes que serán enviados según el evento detectado.

Adicionalmente, se tiene una capa de persistencia que se conecta a una base de datos documental para llevar registro de las notificaciones enviadas.



Figura

62. *Diagrama de clases para el microservicio de gestión de repositorios.*

5.7.7.2 Modelo de datos

El modelo de datos se diseñó para una base de datos documental. La figura 63 muestra la estructura de un documento que tiene como llave principal la identificación del usuario. Además, tiene una lista de eventos a notificar y los correos a los que se debe enviar dicha notificación. Se deja una lista de parámetros adicionales que pueden ser adicionados por el usuario según sea necesario.

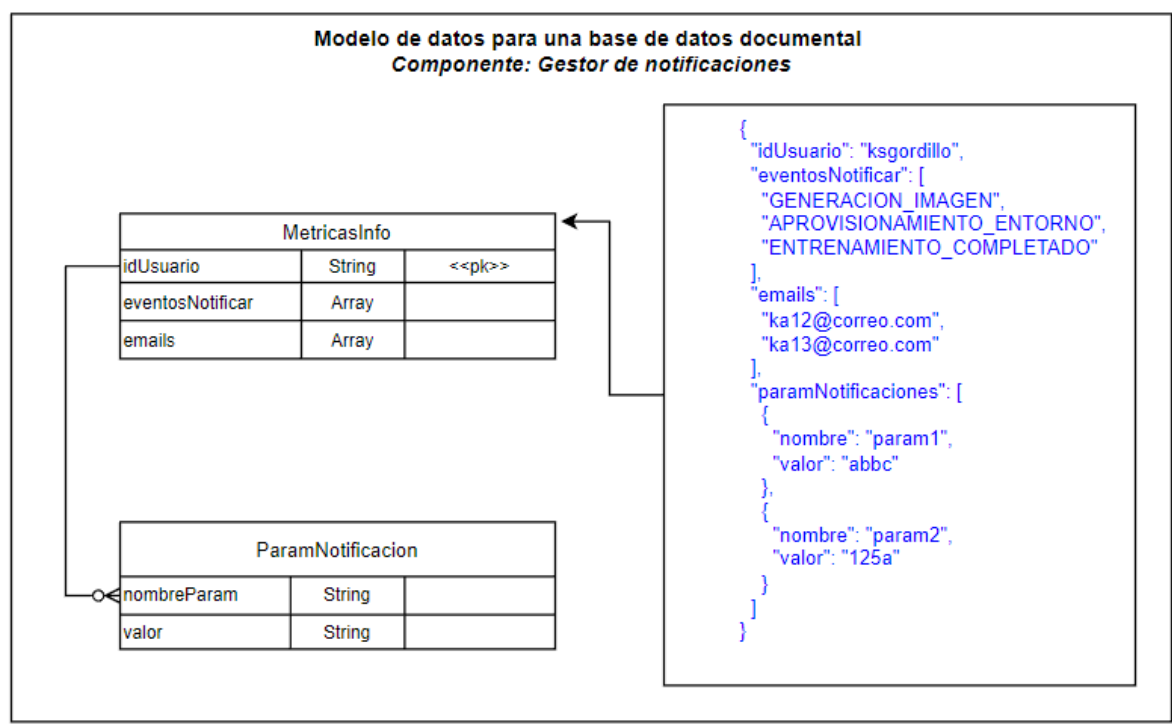


Figura 63. Modelo de datos para base de datos documental del componente de gestión de notificaciones.

5.7.8 Microservicio de gestión de usuarios

5.7.8.1 Modelo estructural

Como se expuso anteriormente, en el capítulo 5.6 acerca de la arquitectura de la aplicación, existe una capa de seguridad que tiene como propósito restringir y permitir a los usuarios el acceso a las funcionalidades del sistema. Es decir, se encarga de la autenticación y autorización de la plataforma.

Esta capa se compone, principalmente, de un directorio activo y un proveedor de identidad. Adicionalmente, se debe llevar un registro de los usuarios registrados en el

sistema, sus roles y la organización a la que pertenecen. Es por esto, que se diseña este servicio de gestión de usuarios.

Como se observa en la figura 64, el módulo está compuesto por tres controladores: controlador de usuarios, controlador de roles y controlador de organizaciones. Cada uno permite realizar operaciones de escritura, actualización, consulta y eliminación de la base de datos relacional donde se encuentran los datos.

Las clases controladoras hacen uso de las interfaces de persistencia llamadas: "OrganizacionRepository", "UsuarioRepository" y "RolRepository"; que a su vez extienden de la librería "ReactiveCrudRepository" para realizar las operaciones sobre la base de datos.

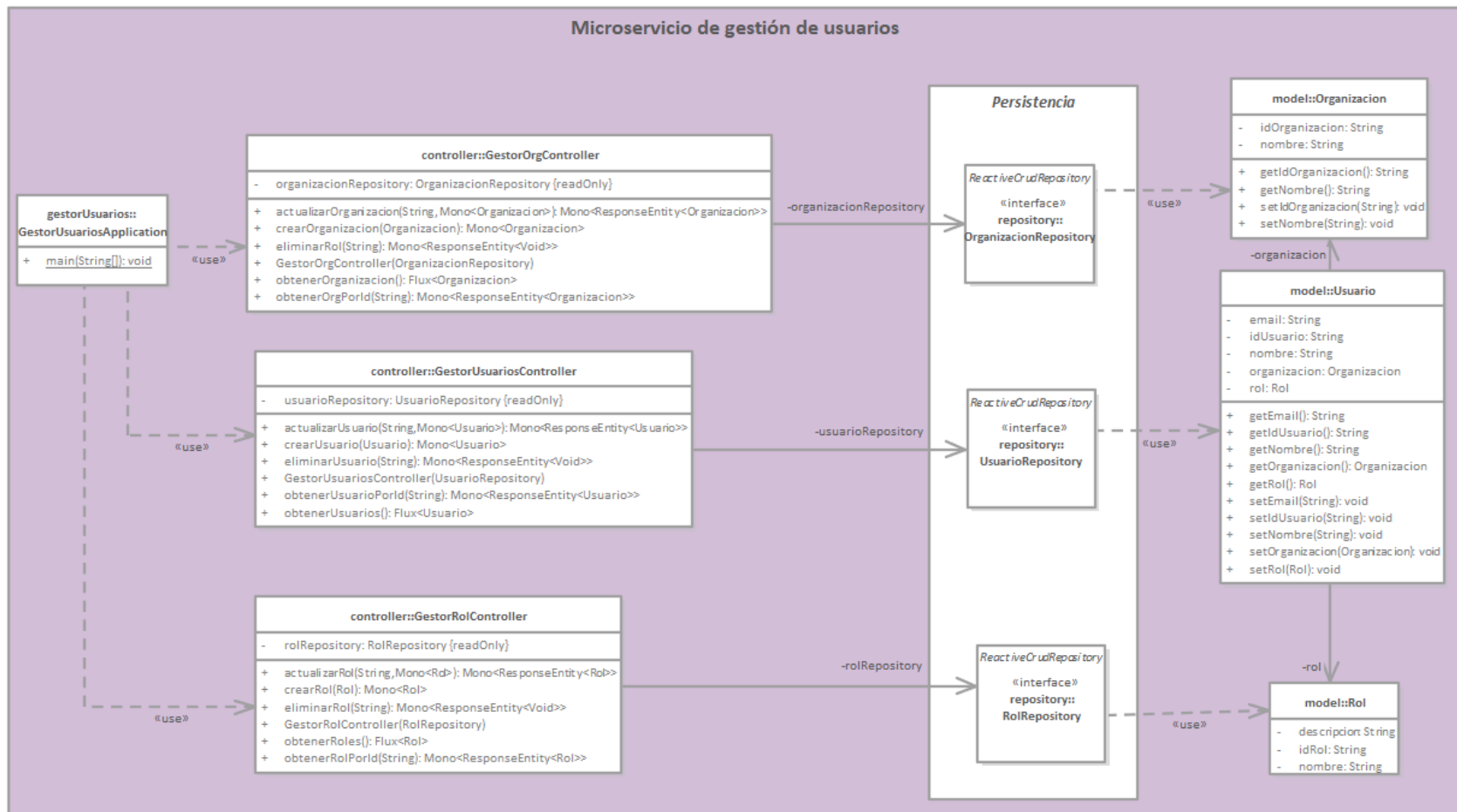


Figura 64. Diagrama de clases para el microservicio de gestión de usuarios.

5.7.8.2 Modelo de datos

El modelo de datos para este componente se diseñó con un modelo de base de datos relacional. Tal como se muestra en la figura 65, se almacenará la información en tres tablas: Usuarios, Rol y Organización.

La tabla Usuarios contendrá el identificador del usuario en el sistema, su correo electrónico, nombre y tendrá dos llaves foráneas hacia las tablas de Rol y Organización.

Respecto a las tablas Rol y Organización. Tienen la información de qué relación de los tipos de relaciones que puede llegar a tener un usuario y la organización a la que representa.

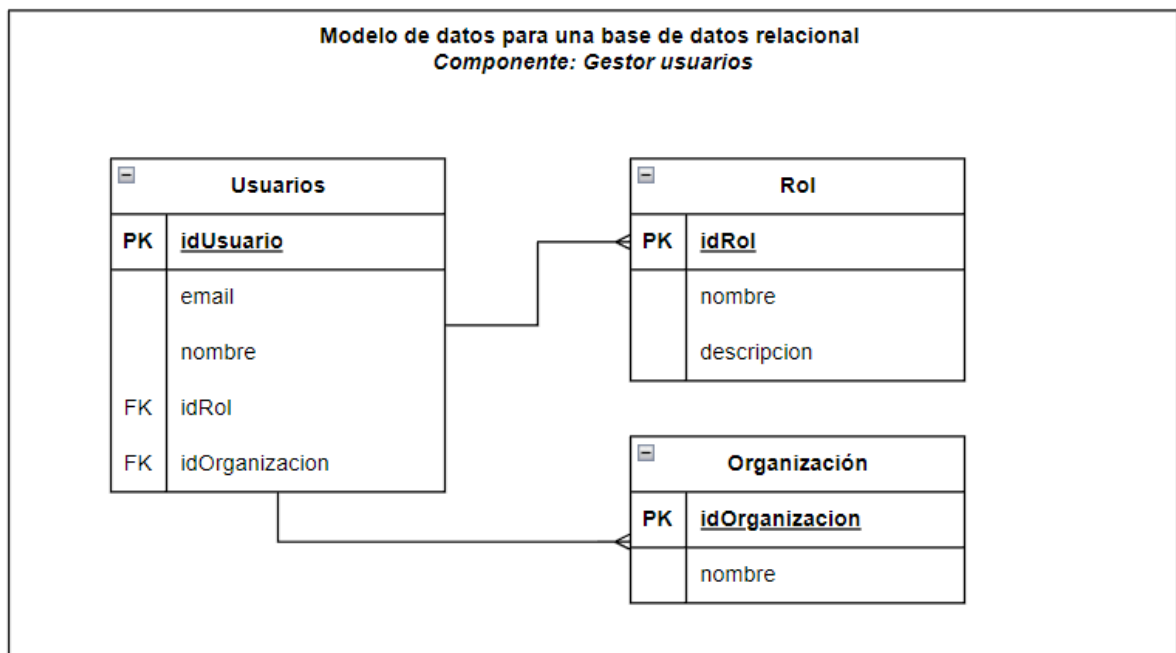


Figura 65. Modelo de datos para base de datos documental del componente de gestión de usuarios.

5.7.9 Microservicio de gestión de resultados

5.7.9.1 Modelo estructural

Cuando se realiza una llamada a alguno de los algoritmos de ML asociados a la plataforma, se pasa como parámetro al servicio una dirección HTTP de notificación. Es decir, una vez que el servicio termine el entrenamiento o procesamiento de los datos, enviará una respuesta a dicha URL informando el resultado en formato JSON

codificado en base 64. Este servicio está diseñado para recibir dicha respuesta y almacenar el resultado en una base de datos documental.

La figura 66 muestra la estructura del servicio. Se compone de un controlador de peticiones HTTP, una capa de servicio y una capa de persistencia. Su finalidad es poder agregar, consultar y actualizar los resultados según lo necesite el cliente.

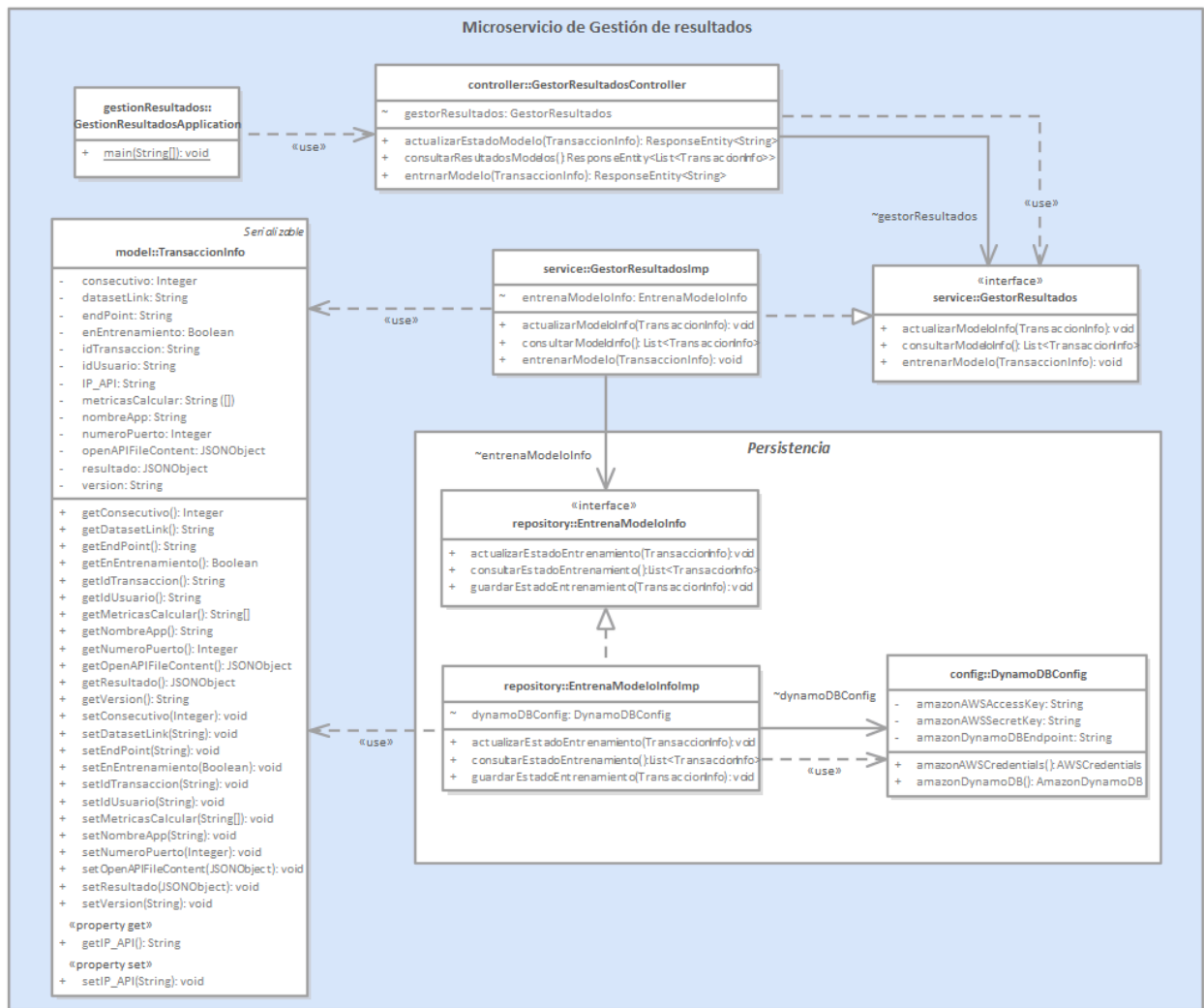


Figura 66. Diagrama de clases para el microservicio de gestión de resultados.

5.7.9.2 Modelo de datos

El modelo de datos para este servicio está diseñado para una base de datos documental. Su estructura se muestra en la figura 67. Como llave principal se tiene una combinación de los parámetros: nombre de la aplicación, versión, id del usuario que hace la petición, id de la transacción y un consecutivo. Este último parámetro se incluye como un identificador de la cantidad de ejemplificaciones que tiene el usuario del mismo algoritmo en su catálogo personal.

El resultado se almacena en base 64 para evitar conflictos con la estructura actual del documento a guardar.

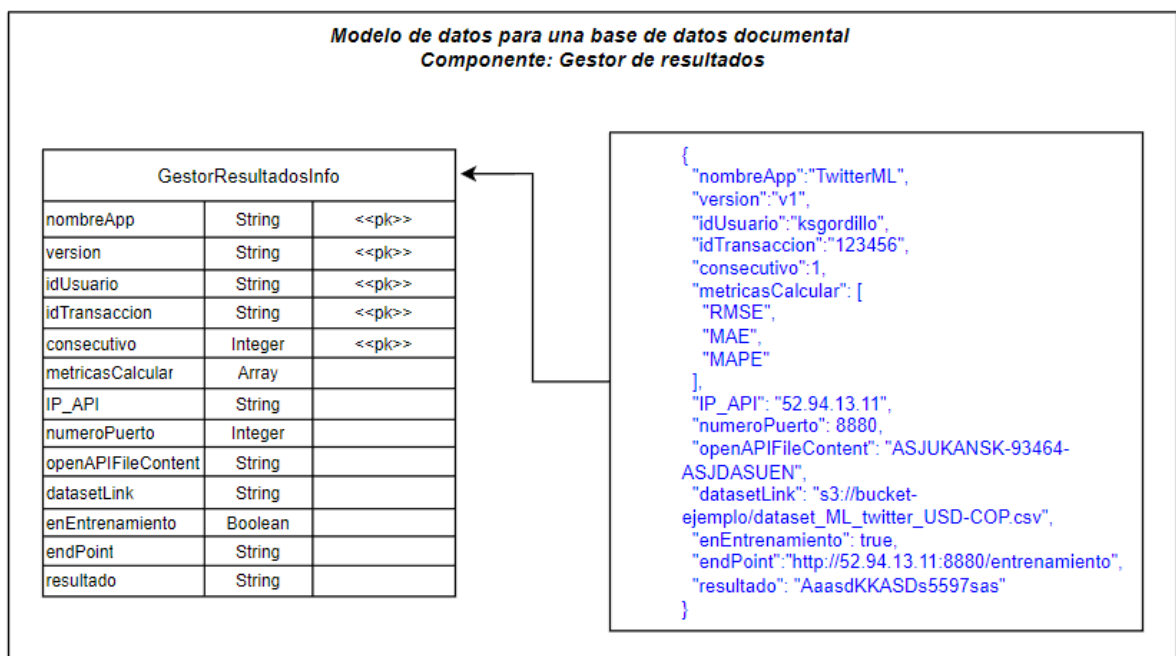


Figura 67. Modelo de datos para base de datos documental del componente de gestión de resultados.

5.7.10 Microservicio de métricas

5.7.10.1 Modelo estructural

Este servicio está encargado de realizar el cálculo de cada una de las métricas que se han incluido con antelación en el sistema. Para este caso de estudio se incluyen tres métricas principales: MAE, RMSE y MAPE, descritas en el capítulo 4.1. La lógica detrás de este servicio consiste en recibir una petición HTTP con los datos reales y los datos calculados por el algoritmo y realizar el cálculo correspondiente de cada métrica.

El diseño estructural, como se muestra en la figura 68, está conformado por una capa de servicio, un controlador, una capa de persistencia y una serie de clases que aplican el patrón estrategia para realizar los cálculos correspondientes. Se decide usar este patrón ya que nos permite separar los algoritmos en clases para poder luego intercambiar sus objetos a través de la clase contexto.

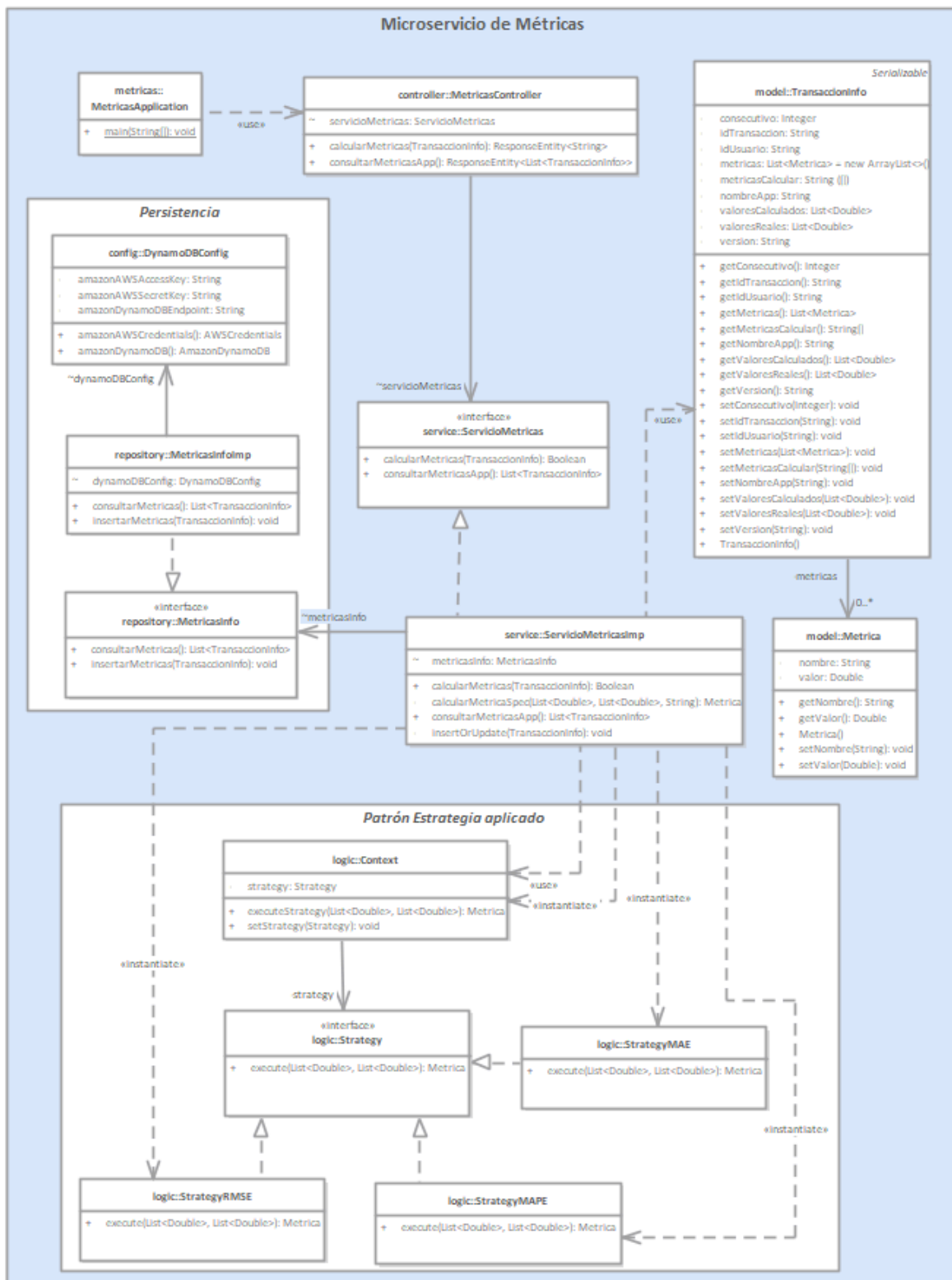


Figura 68. Diagrama de clases para el microservicio de métricas.

5.7.10.2 Modelo de datos

El modelo de datos, como se presenta en la figura 69, se diseñó para una base de datos documental donde se guardará el resultado del cálculo de las métricas y tendrá como índice la llave compuesta por: nombre de la aplicación, versión, id del usuarios, id de la transacción y el consecutivo generado por el componente de gestión del catálogo.

Al igual que los demás esquemas, esta base de datos documental guardará documentos en formato JSON. Por otro lado, el valor calculado de las métricas siempre dará como resultado un valor numérico.

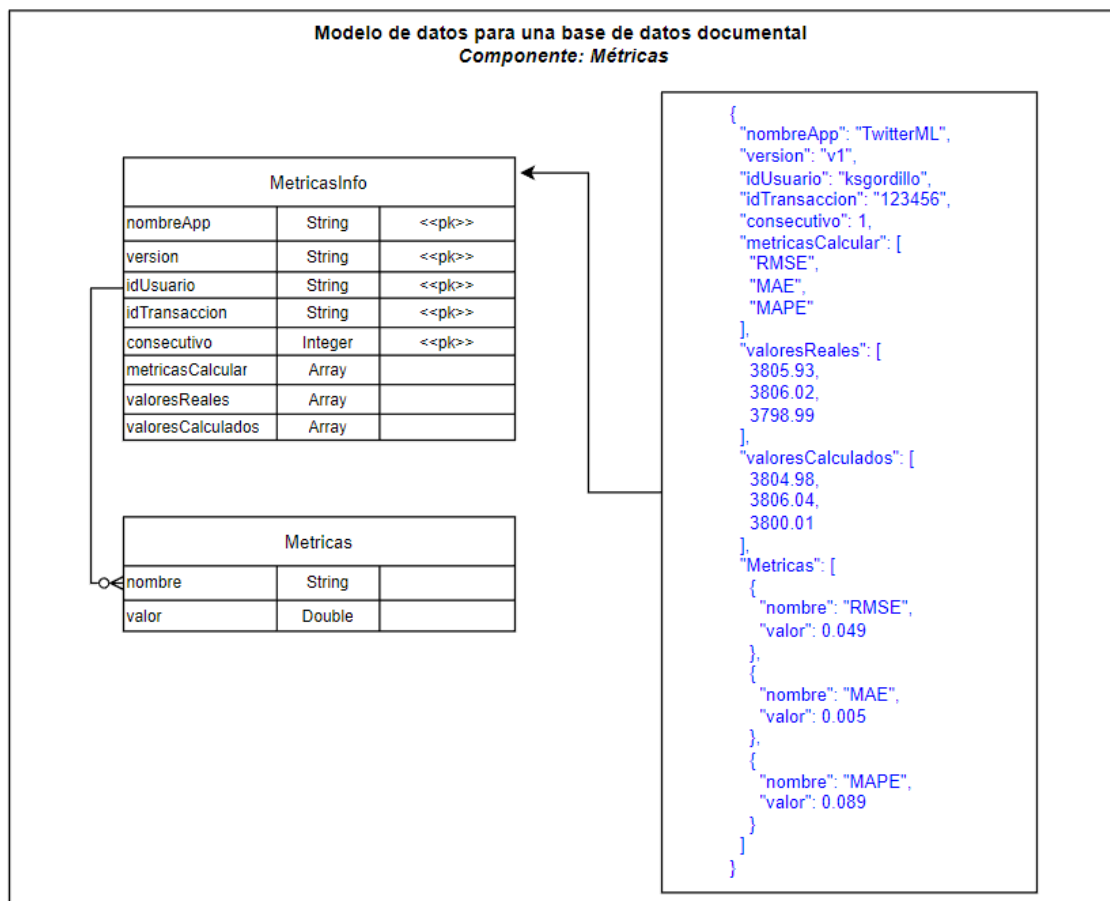


Figura 69. Modelo de datos para base de datos documental del componente de cálculo de métricas.

5.8 ESQUEMAS DE PÁGINAS WEB

En esta sección se presentan los bocetos de esquemas para las páginas web que conforman la parte visual o “frontend” de la plataforma.

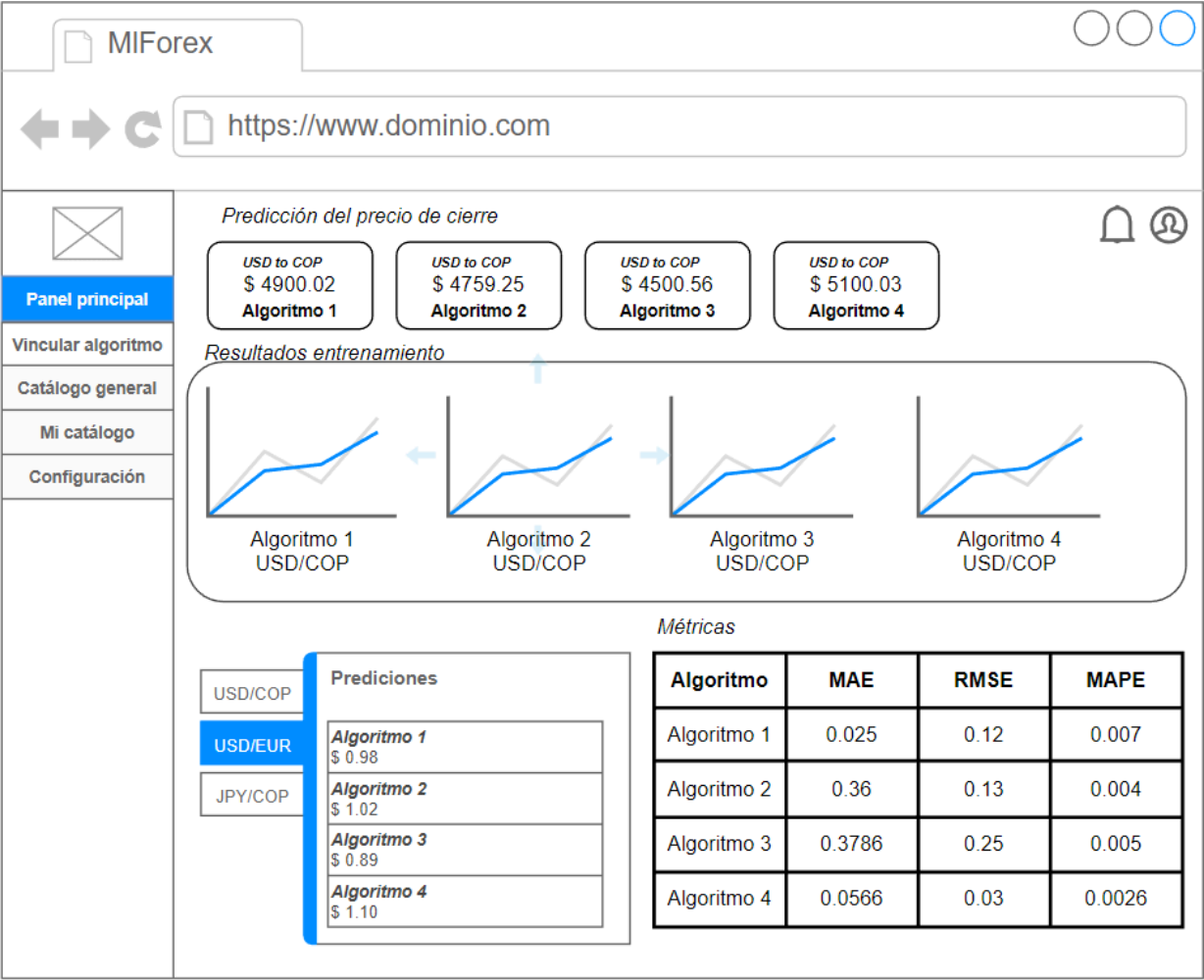


Figura 70. Esquema web de página principal. Se muestran los resultados principales arrojados por los algoritmos.

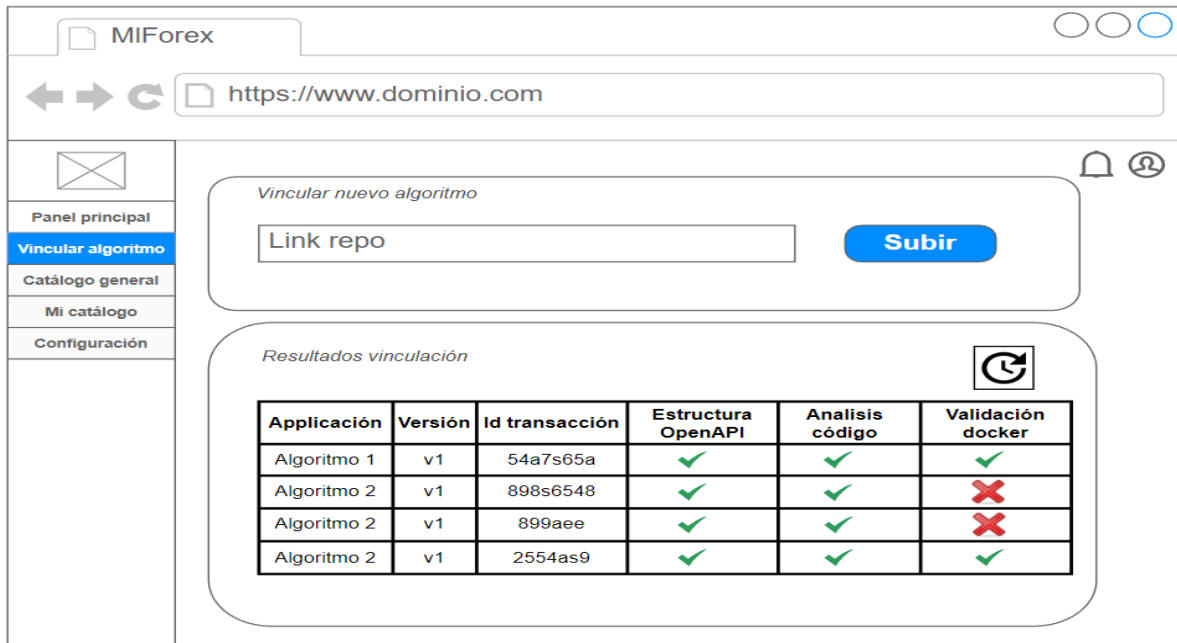


Figura 71. Esquema web de página para la vinculación de algoritmos. El primer panel representa el espacio para incluir el enlace al repositorio. El segundo panel muestra el estado de la vinculación.

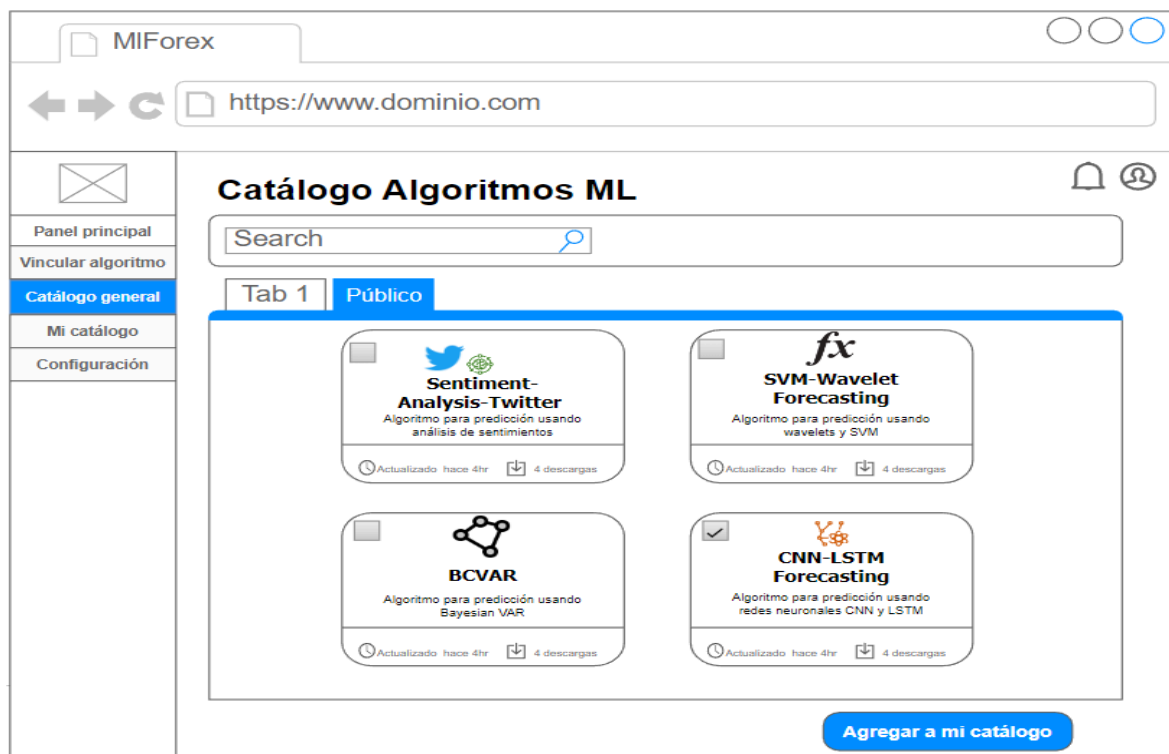


Figura 72. Esquema web de página que muestra el catálogo general del sistema. Permite agregar algoritmos al catálogo personal.

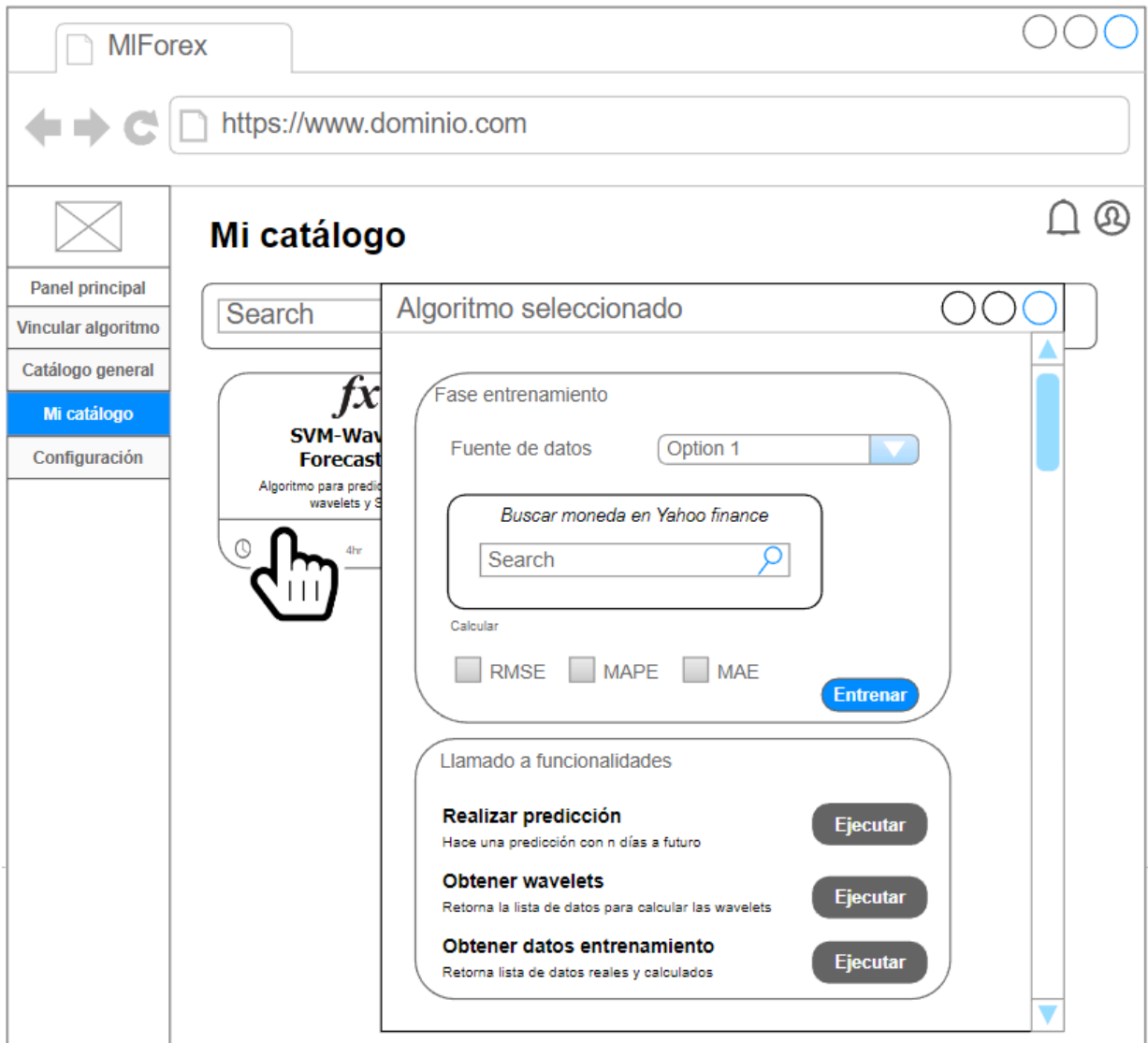


Figura 73. Esquema web de página web que permite interactuar con los algoritmos añadidos al catálogo personal. Permite realizar el llamado al método de entrenamiento y a los demás métodos que ofrece el algoritmo de ML seleccionado.

MIForex

https://www.dominio.com

Configuración

Notificaciones

Emails ☐ Generación ☐ Entrenamiento ☐ Resultados ☐ Aprovisionamiento **Guardar**

Usuarios

Usuario Password Rol Organización **Guardar**

Roles

Nombre Descripción **Guardar**

Organizaciones

Nombre Descripción **Guardar**

Proveedores repositorios

Dominio Parametro Agregar Valor **Guardar**

Figura 74. Esquema web de página web que permite realizar las configuraciones dentro de la plataforma. Los paneles se activan y desactivan dependiendo del rol y permisos del usuario en el sistema.

6. RESULTADOS - PROTOTIPO

Siguiendo el desarrollo de la cuarta y última fase propuesta en la metodología del presente documento, se lleva a cabo la implementación del prototipo utilizando la arquitectura expuesta anteriormente. El objetivo del prototipo es validar la viabilidad del diseño propuesto, incluyendo la transmisión de mensajes entre servicios y la composición de pantallas web.

Teniendo en cuenta que se abordará un prototipo para la validación funcional del diseño propuesto, se realiza, de acuerdo con las especificaciones de la arquitectura, la implementación de los siguientes módulos:

- Interfaz web de usuario (frontend)
- Microservicio de vinculación de código
- Microservicio de gestión de entornos de ejecución
- Servicio de gestión de catálogo
- Microservicio de gestión de parámetros de las API (Application Programming Interface) a generar
- Microservicio de métricas
- Microservicio de gestión de resultados

De acuerdo a los alcances del prototipo no fueron implementados la capa de autenticación, servicios de gestión de usuarios y notificaciones.

Para la implementación de los servicios se eligió Java como lenguaje de programación y el marco de trabajo “Spring framework” para acelerar el proceso de desarrollo. Por otro lado, para la implementación de la interfaz gráfica de usuario, se utiliza el lenguaje de programación JavaScript y el marco de trabajo “ReactJS”. Para la implementación de las bases de datos y bus de datos se utiliza dos tecnologías proporcionadas por Amazon Web Services (AWS): DynamoDB y Simple Notification Service (SNS). Se utiliza AWS ya que esta plataforma ofrece un crédito gratuito de 200 USD para proyectos de investigación, lo que reduce los costos de implementar el prototipo. En la figura 75 se presenta la arquitectura simplificada de los elementos implementados en este prototipo utilizando los elementos anteriormente mencionados.

A continuación, se presentan algunas imágenes con los flujos implementados.

Como se presenta en la figura 76, se implementó el flujo principal de vinculación de algoritmos a la plataforma. Para hacer uso de esta pantalla, el usuario debe primero especificar la URL pública del repositorio donde está alojado el algoritmo a correr. En segundo lugar, debe agregar el nombre del algoritmo, su descripción y la versión a vincular, y hacer clic en el botón "Agregar". Después, debe esperar a que finalicen las validaciones de la estructura de la especificación de la API, la generación de la imagen y el análisis de vulnerabilidades.

Una vez se ha realizado la vinculación, el algoritmo aparecerá en la sección "Catálogo general", como se muestra en la figura 77. En esta sección, el usuario puede hacer clic en "Añadir a mi catálogo" para agregar el algoritmo a su propia lista de algoritmos disponibles. Es importante destacar que esta plataforma está diseñada para permitir que múltiples usuarios vinculen múltiples algoritmos, de modo que puedan ser utilizados de forma colaborativa.

Dentro del catálogo personal, como se ilustra en la figura 78, se puede lanzar el entrenamiento del algoritmo. Para esto, primero se debe seleccionar la moneda de origen y la moneda de destino con la que se quiere realizar el entrenamiento. Luego, se selecciona el periodo de los datos de entrenamiento a utilizar y las métricas a calcular, y se inicia el entrenamiento del modelo. Después de unos minutos, los resultados aparecerán en el panel principal.

Por último, en las figuras 79 y 80 se muestra el panel de resultados. Este panel se compone de dos secciones principales: una sección que muestra el precio estimado por cada uno de los algoritmos para una divisa en específico, y otra sección que muestra el resultado del entrenamiento de cada modelo y sus respectivas métricas de comparación.

Adicionalmente, en la figura 80 se presentan los resultados de desempeño de cada uno de los algoritmos en términos de MAE, MAPE y RMSE. Se observa que el algoritmo que utiliza funciones radiales en una red neuronal (RBF NN) presenta el mejor rendimiento, con un MAPE del 0,22%, un MAE de 10,05 y un RMSE de 11,20 en la fase de entrenamiento, en la predicción del tipo de cambio dólar

estadounidense-peso colombiano (USD/COP).

Los resultados obtenidos en la plataforma siguen la misma tendencia que los resultados obtenidos en la sección 4.2, donde se compararon los distintos modelos. Es decir, el algoritmo de RBF NN muestra el mejor rendimiento en comparación con los otros algoritmos evaluados.

Por otro lado, el algoritmo de análisis de sentimientos utilizando Twitter mostró el peor desempeño con un MAPE del 3.27% para el mismo par de divisas (USD/COP). Esto sugiere que este algoritmo no es la mejor opción para realizar predicciones en este mercado.

Otra característica interesante del prototipo desarrollado es su capacidad de configurarse para utilizar otras monedas del mercado de divisas, así como también del mercado de acciones. Esto se debe a que la entrada definida para cada modelo es una serie de tiempo y la plataforma puede comunicarse con Yahoo Finance. Solo sería necesario agregar una lista de las otras monedas o precios de acciones a consultar y pasarlos como insumos a los modelos. Esto demuestra la versatilidad y escalabilidad del prototipo, lo que lo convierte en una herramienta útil y adaptable para predecir diferentes mercados financieros.

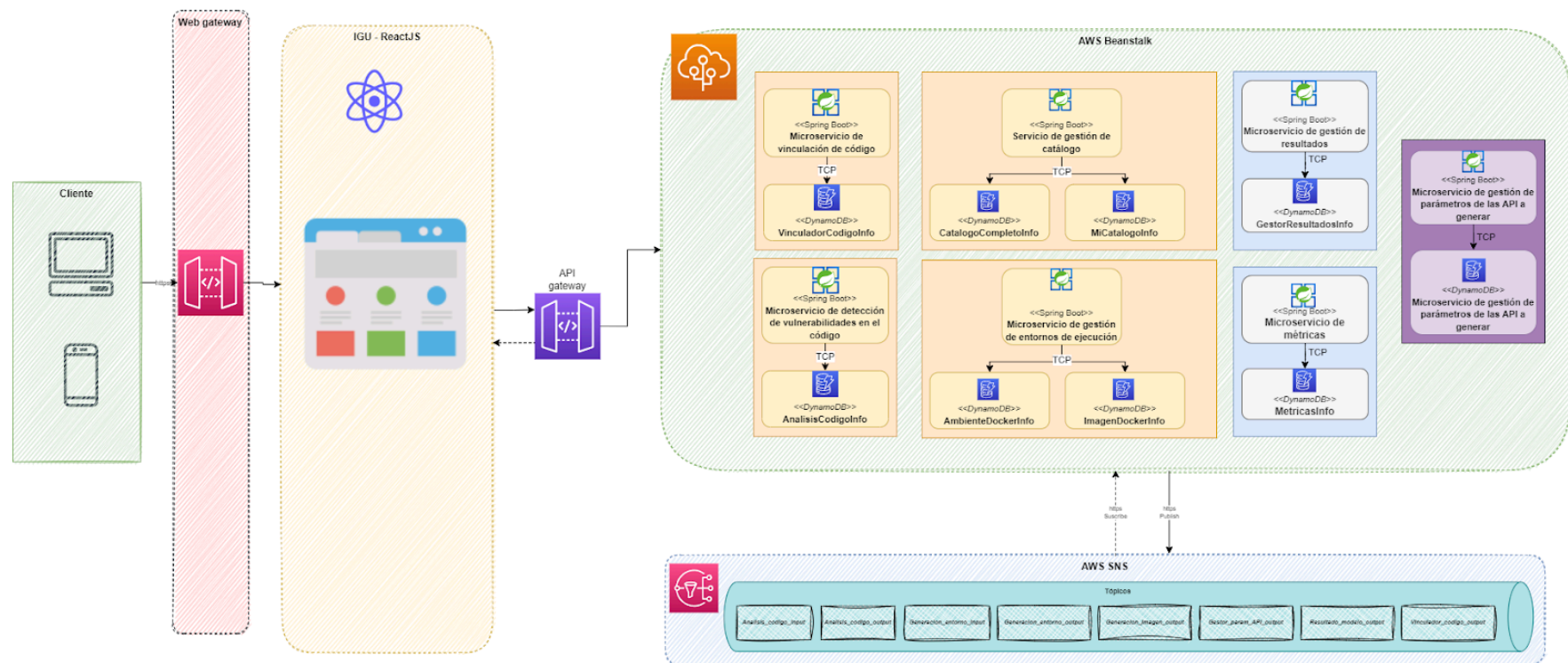
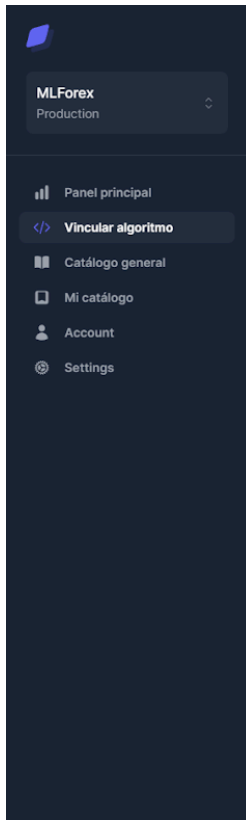


Figura 75. Arquitectura simplificada del prototipo implementado utilizando los servicios de AWS: Beanstalk, SNS, DynamoDB y API gateway. Se utilizan los lenguajes de programación Java y Javascript.



Vincular algoritmo

https://github.com/kgordillo-hub/5.RBFNN

Ingrese el nombre del algoritmo *

RBFNN

Seleccione la versión *

1

Agregue una descripción del algoritmo *

Modelo que utiliza redes neuronales y función radial para realizar la predicción

+ Agregar

Info. vinculaciones

Refrescar

ALGORITMO	VERSION	ID. TRANSACCION	VALIDACIÓN ESTRUCTURA OPENAPI	ANÁLISIS VULNERABILIDAD CÓDIGO	GENERACIÓN IMAGEN DOCKER
Sentiment_Analysys_Twitter	v1	7d2f92e0d0739dced91a77f26d311fc7	✓	✓	✓
Wavelets_SVM	v1	d9be518713fc0554633dbaa092cb561b	✓	✓	✓
BCVAR	v1	576ec976c9f9ab11f42bb0950fb6474b	✓	✓	✓
RBFNN	v1	7b33c13fffdeb0eacd2b3efe90e062ec	✓	✓	✓
LSTM_CNN	v1	b1208f73d7f1bb5b8876b9b07aeb9daa	✓	✓	✓

Rows per page: 1-0 of 5

Figura 76. Pantalla de vinculación de algoritmos a la plataforma. El usuario ingresa la información del algoritmo a vincular: enlace al repositorio, nombre del modelo, descripción y versión. Luego, al hacer clic en “Agregar” se dispara la validación del modelo.

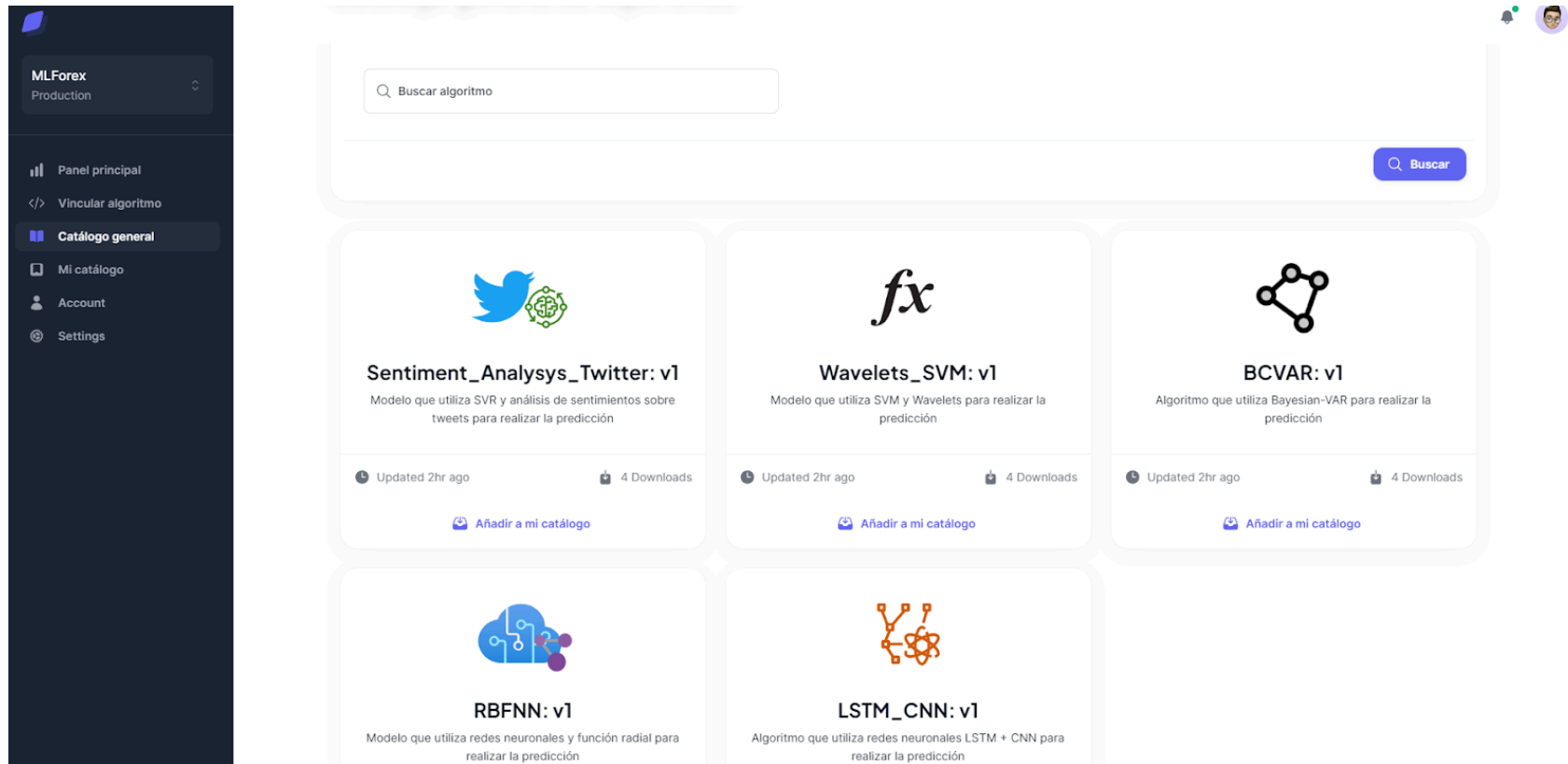


Figura 77. Pantalla del catálogo general de algoritmos vinculados a la plataforma. El usuario puede ver y escoger los modelos a usar.

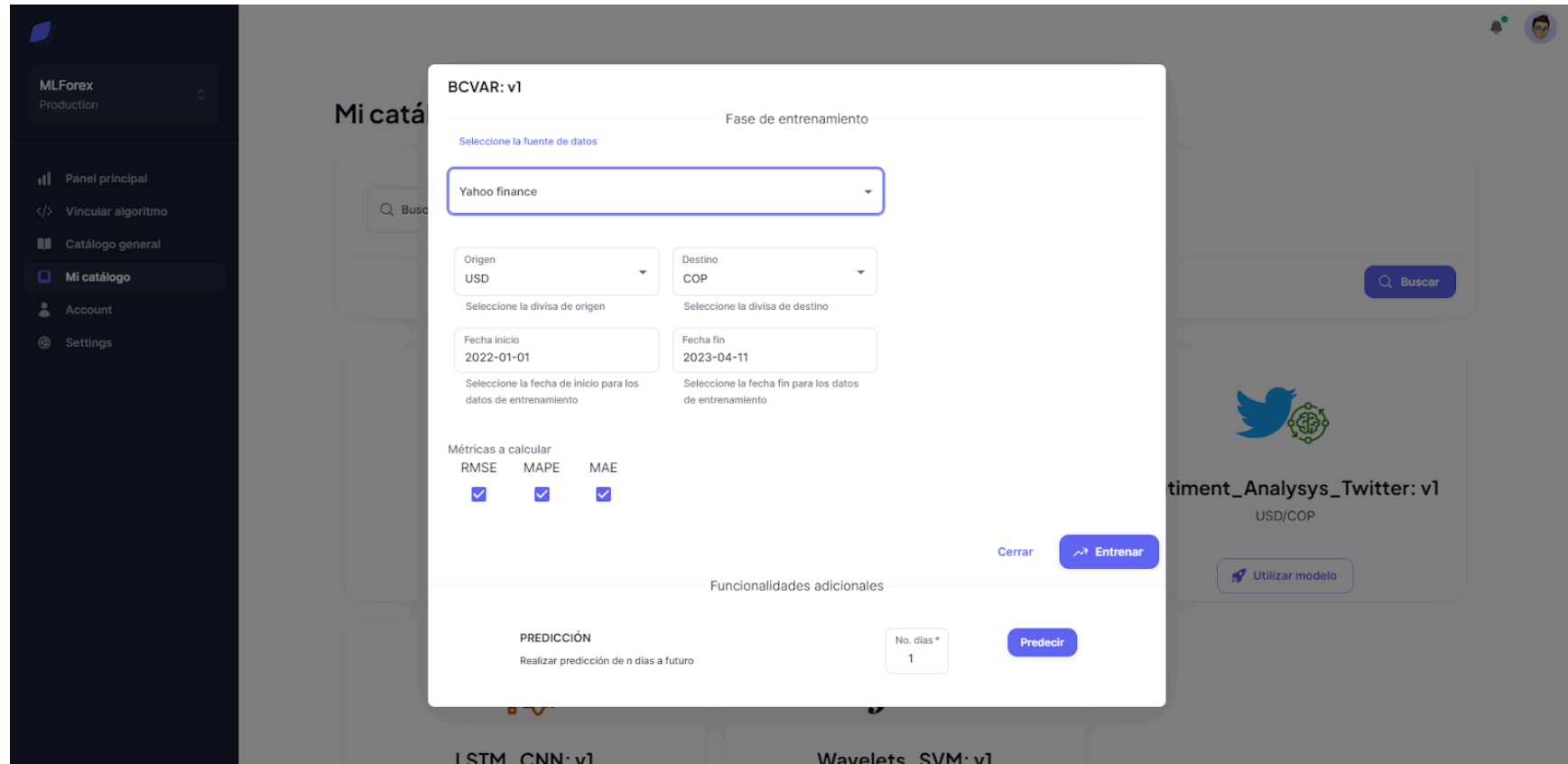


Figura 78. Pantalla del catálogo personal donde se puede hacer uso del algoritmo agregado, se tienen las opciones de iniciar el entrenamiento del algoritmos y de realizar una predicción una vez se ha entrenado.

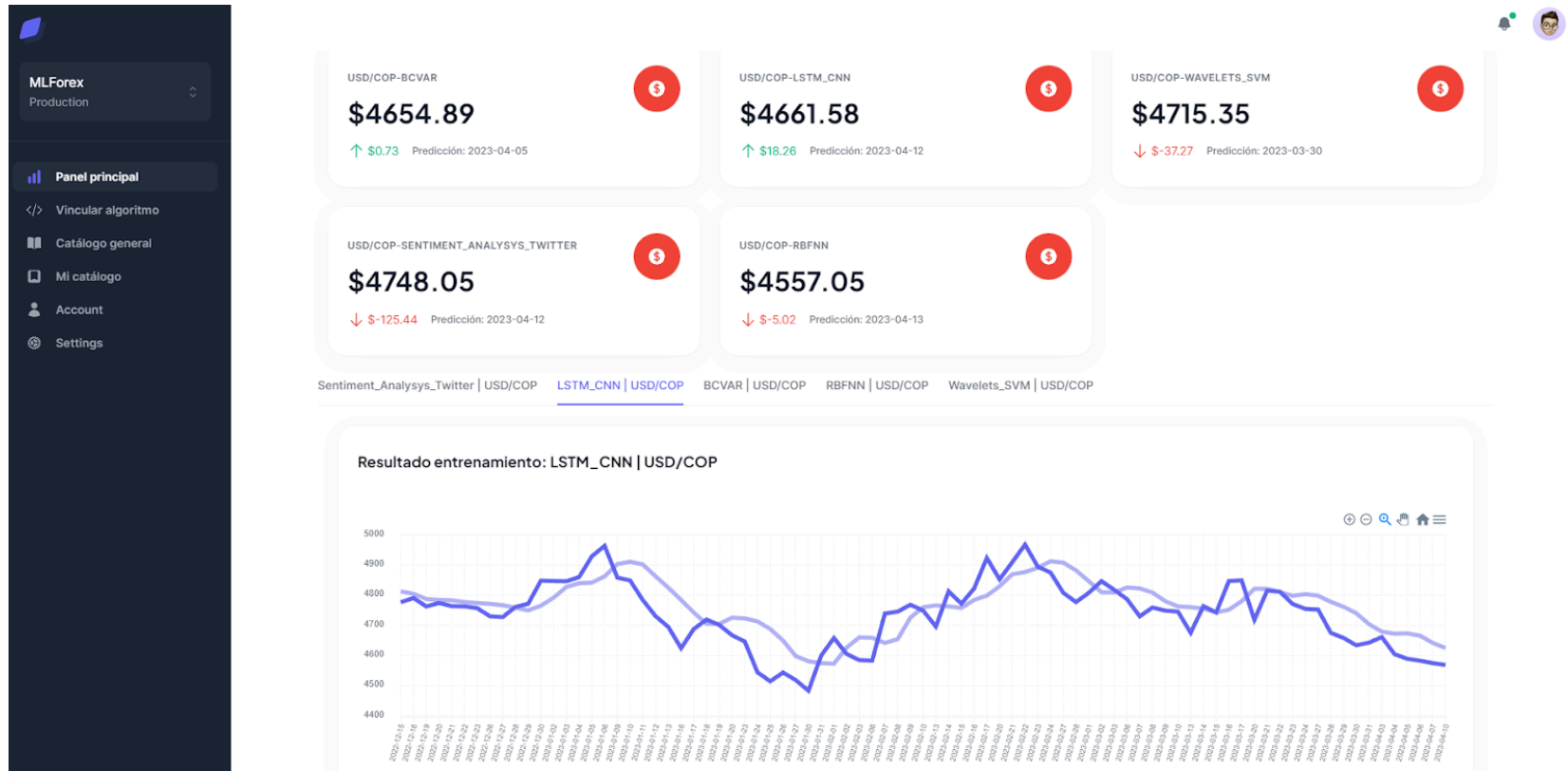


Figura 79. Pantalla principal que muestra la predicción del precio de cierta divisa para un día seleccionado y los resultados del entrenamiento para cada uno de los algoritmos utilizados en el catálogo personal.

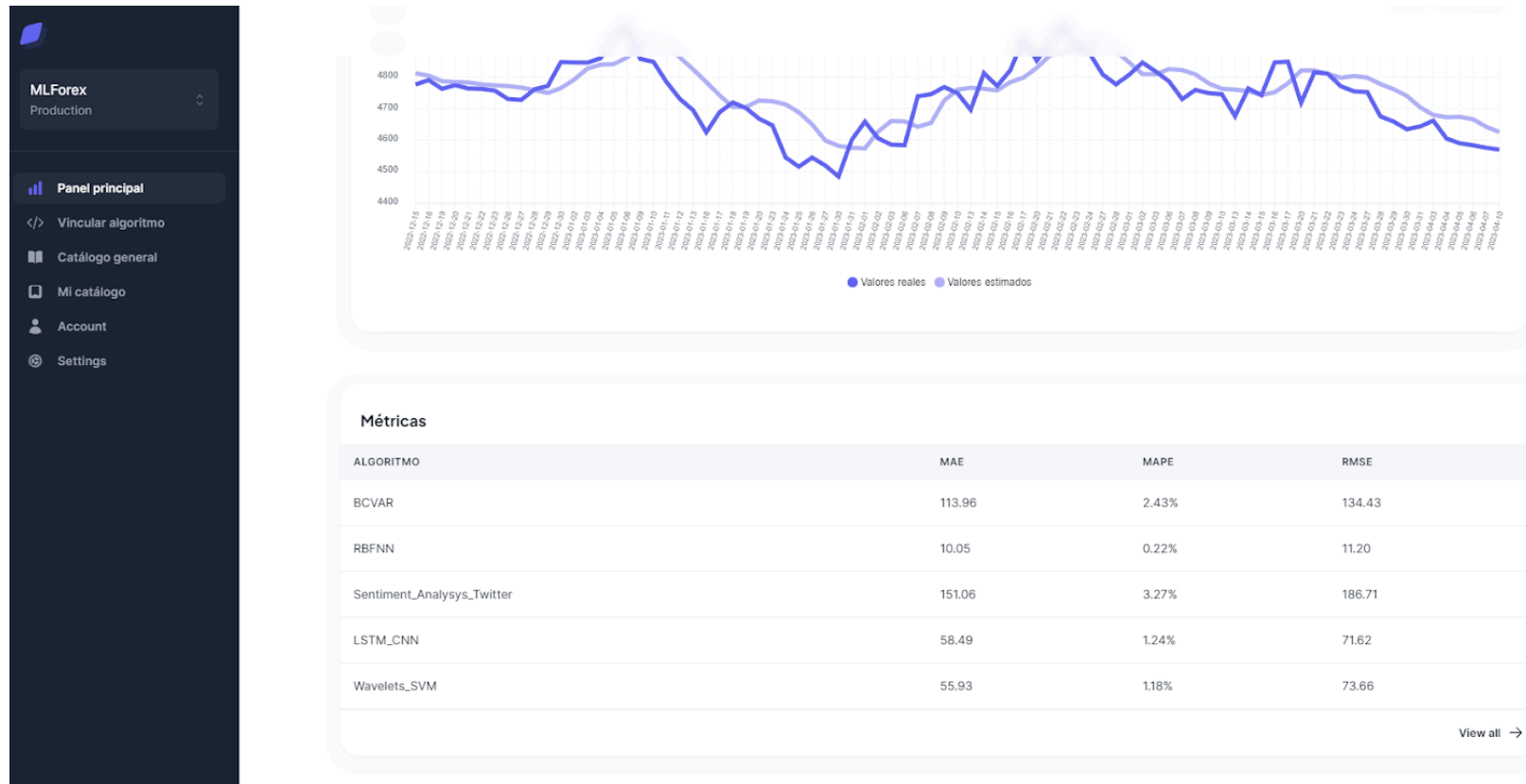


Figura 80. Pantalla principal que muestra las métricas de comparación: MAE, MAPE y RMSE entre los diferentes algoritmos entrenados.

7. CONCLUSIONES

- Durante la revisión de algoritmos, se detectó que varios trabajos utilizan técnicas similares para predecir el precio de diversas divisas. Por ejemplo, se emplean redes neuronales convolucionales o máquinas de vectores de soporte. La variación se encuentra en los parámetros utilizados para el entrenamiento de los algoritmos y en algunas ocasiones, los autores combinan varias técnicas, como los algoritmos evolutivos, para mejorar el rendimiento de los modelos. Se concluye que los trabajos publicados recientemente se centran en mejorar las técnicas existentes de Machine Learning para la realización de predicciones en el mercado de divisas, en lugar de desarrollar nuevas técnicas de predicción.
- Dentro de la revisión de la literatura se encontró que las métricas en común más utilizadas por los autores para medir el rendimiento de los diferentes algoritmos y realizar comparativas son: MAE (Mean Absolute Error), RMSE (Root Mean Square Error) y MAPE (Mean Absolute Percentage Error). Estas medidas de error en la predicción proporcionan una buena idea estadística de cuánto se alejan las estimaciones de los valores reales, lo que permite comparar el rendimiento de los algoritmos desarrollados o mejorados por otros autores.

Según los resultados obtenidos, se puede concluir que el algoritmo que utiliza funciones radiales para mejorar el entrenamiento de una red neuronal clásica es el que muestra el mejor rendimiento en términos de precisión. Por otro lado, se encontró que el algoritmo que combina análisis de sentimientos sobre la información de Twitter con vectores de soporte de regresión (SVR) obtuvo el peor desempeño. Si bien la incorporación de procesamiento de lenguaje natural al análisis puede enriquecer la información de entrada al modelo, se considera que la sustitución de la técnica SVR por una red neuronal podría mejorar el rendimiento de este algoritmo.

Otros modelos, aunque también utilizan elementos matemáticos complejos

como BCVAR, no presentaron resultados tan prometedores como el algoritmo RBF NN. Es posible que BCVAR esté optimizado para la predicción del precio de acciones en lugar de para el cálculo de precios de divisas, lo que podría explicar los resultados obtenidos.

- Se identificaron tres desafíos en el diseño de la plataforma. En primer lugar, la integración de múltiples algoritmos que utilizan diferentes tecnologías y librerías, lo cual se abordó mediante la virtualización en contenedores. En segundo lugar, debido a que muchos de estos algoritmos requieren una etapa de entrenamiento previa que puede durar varios minutos, se optó por desacoplar el sistema en pequeños servicios que se comunican de manera asíncrona y notifican sus estados mediante un gestor de eventos. Si bien este diseño desacoplado soluciona el problema de los altos tiempos de procesamiento, también introduce un pequeño retraso en las respuestas que percibe el usuario, lo cual no es crítico en este caso ya que se trata de un sistema de predicciones.

Otros aspectos que se tuvieron en cuenta durante el diseño del software incluyen el mantenimiento posterior del sistema. Una de las ventajas de tener un sistema modularizado en pequeños servicios es que el mantenimiento de cada uno de ellos se puede llevar a cabo de manera más sencilla y por diferentes equipos de desarrollo. Además, dado que cada servicio es independiente, si uno de ellos falla, el sistema completo no se verá afectado y seguirá funcionando con los componentes que sí estén funcionando correctamente.

Como recomendaciones para la implementación futura de este sistema, se sugiere lo siguiente: En primer lugar, desarrollar una capa de seguridad para proteger los servicios de posibles ataques. En segundo lugar, es necesario implementar un componente para controlar los roles y los mecanismos de acceso al catálogo de algoritmos. Por último, es importante aprovisionar máquinas con suficiente capacidad de cómputo y optimizadas para ejecutar algoritmos de aprendizaje automático para el despliegue del componente que gestiona el catálogo personalizado de algoritmos.

- Durante el montaje del prototipo, se enfrentó el desafío de desplegar los componentes de la plataforma en una infraestructura propia. Para solucionarlo,

se optó por alquilar servidores en la nube, concretamente en AWS (Amazon Web Services). Esta elección permitió descubrir una amplia gama de herramientas disponibles en AWS que facilitaron significativamente el proceso de despliegue e integración de los componentes. Entre estas herramientas, destaca AWS Elastic Beanstalk, que simplifica el proceso de despliegue al realizar todo el aprovisionamiento del entorno de ejecución, dejando al desarrollador la tarea de solo generar el código.

Otra herramienta que destaca es DynamoDB, una base de datos no relacional descentralizada que permite crear particiones de manera ágil y dinámica. En cuestión de minutos, se puede tener una base de datos completamente funcional, lo que ahorra mucho tiempo en el despliegue y la conexión de los componentes.

Finalmente, para el manejo del bus de eventos se utilizó el servicio Simple Notification Service (SNS) de AWS. Esta tecnología permite agregar tópicos y suscriptores de manera sencilla y además, libera a los desarrolladores de la responsabilidad de mantener los servidores que dan soporte a esta tecnología. El uso de SNS permitió una fácil integración con los demás servicios de AWS y un manejo eficiente de las notificaciones entre los diferentes componentes del sistema.

- Por último, de la revisión detallada de algoritmos en la sección 2.1.2, se ha encontrado que estos pueden ser combinados para generar nuevos algoritmos de predicción. Por ejemplo, se podría combinar el algoritmo de máquinas de vectores de soporte con el uso de Wavelets, con el algoritmo de análisis de sentimientos en Twitter al utilizar la matriz de salida de este último como entrada del primero. También se podrían combinar otros algoritmos, como el de redes neuronales basadas en funciones radiales y el de redes neuronales convolucionales, utilizando la función radial del primero para mejorar el rendimiento del segundo.

Estas combinaciones y mejoras en los algoritmos podrían generar una nueva línea de investigación en el futuro. En el capítulo siguiente, se discuten posibles aplicaciones y trabajos futuros.

8. TRABAJOS FUTUROS

- En este trabajo se ha desarrollado un prototipo de plataforma enfocado en la vinculación y ejecución de algoritmos de Machine Learning. Como trabajo futuro, se propone finalizar la plataforma siguiendo el diseño descrito en este documento, lo cual incluye la implementación de las funcionalidades de configuración y la capa de autenticación.
- Los algoritmos implementados tienen la capacidad de complementarse según las necesidades de los usuarios. En el futuro, podría incluirse un módulo que permita la combinación de algoritmos compatibles y la introducción de nuevos parámetros de medición.
- La plataforma tiene la capacidad de ser extendida para incluir más fuentes de datos según las necesidades, lo que significa que podría ser utilizada para realizar predicciones sobre otras series de tiempo, como el precio de acciones en los mercados bursátiles. En futuras iteraciones, se podría agregar un módulo para incorporar fuentes de datos adicionales a las descritas en este documento.
- Se encontró que, durante el proceso de despliegue en la nube, existen herramientas que facilitan la gestión de los algoritmos de Machine Learning, como Amazon SageMaker o Azure Machine Learning. Para futuros trabajos, se podría considerar la posibilidad de integrar estos proveedores como una alternativa a la virtualización de los entornos de ejecución.

REFERENCIAS

1. Islam, S. Hossain, E. Rahman, A. Hossain, S.M. Andersson, K. (2020). A review on Recent Advancements in FOREX Currency Prediction. Algorithms, MDPI.
2. Raimundo, M. S., & Okamoto, J. (2018, March). SVR-wavelet adaptive model for forecasting financial time series. In Proceedings of the 2018 International Conference on Information and Computer Technologies (ICICT) (pp. 111-114). DeKalb, IL, USA.
3. Das, S. R., Mishra, D., & Rout, M. (2020). A hybridized ELM-Jaya forecasting model for currency exchange prediction. Journal of King Saud University-Computer and Information Sciences, 32(2), 345-366.
4. Pradeepkumar, D. & Ravi, V. (2017). Forecasting financial time series volatility using particle swarm optimization trained quantile regression neural network. Applied Soft Computing, 58, 35-52.
5. Sidehabi, S., Indrabayu, and Tandungan, S. (2016). Statistical and Machine Learning approach in forex prediction based on empirical data. In 2016 International Conference on Computational Intelligence and Cybernetics, pp. 63-68. doi: 10.1109/CyberneticsCom.2016.7892568.
6. Thu, T.N.T. & Xuan, V.D. (2018). Using support vector machine in FoRex predicting. In 2018 IEEE International Conference on Innovative Research and Development (ICIRD) (pp. 1-5). Bangkok, Thailand. doi: 10.1109/ICIRD.2018.8376303.
7. Borgli, R.J., Stensland, H.K., Halvorsen, P., & Riegler, M.A. (2019). Saga: An Open Source Platform for Training Machine Learning Models and Community-driven Sharing of Techniques. 2019 International Conference on Content-Based Multimedia Indexing (CBMI), pp. 1-4. doi: 10.1109/CBMI.2019.8877455.

8. Mohammadi, W. A. Z. I. R. (2019). Currency Exchange Rate Forecasting Using Machine Learning Techniques. Graduate School of Applied Sciences. Near East University.
9. Hamed Naderi Semiromi, Stefan Lessmann, Wiebke Peters. (2020). News will tell: Forecasting foreign exchange rates based on news story events in the economy calendar. *The North American Journal of Economics and Finance*, 52, 101181. <https://doi.org/10.1016/j.najef.2020.101181>.
10. Chen, W., Xu, H., Jia, L., & Gao, Y. (2021). Machine learning model for Bitcoin exchange rate prediction using economic and technology determinants. *International Journal of Forecasting*, 37(1), 28-43. <https://doi.org/10.1016/j.ijforecast.2020.02.008>.
11. Sung, N., Kim, M., Jo, H., Yang, Y., Kim, J., Lausen, L., ... & Kim, S. (2017). Nsmf: A machine learning platform that enables you to focus on your models. *arXiv preprint arXiv:1712.05902*.
12. Zhao, S., Talasila, M., Jacobson, G., Borcea, C., Aftab, S. A., & Murray, J. F. (2018). Packaging and sharing machine learning models via the acumos ai open platform. In 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA) (pp. 841-846). IEEE.
13. IBM. (s.f). ¿Qué es Machine Learning? <https://www.ibm.com/ar-es/analytics/machine-learning>. Consultado el 16 de junio de 2021.
14. Pradeepkumar, D., & Ravi, V. (2018). Soft computing hybrids for FOREX rate prediction: A comprehensive review. *Computers & Operations Research*, 99, 262-284. <https://doi.org/10.1016/j.cor.2018.05.020>
15. No author. (n.d.). What are Mean Squared Error and Root Mean Squared Error? Vernier. Consultado el 16 de junio de 2021., de <https://www.vernier.com/til/1014>.
16. Swamidass, P.M. (2000). MEAN ABSOLUTE PERCENTAGE ERROR (MAPE). In *Encyclopedia of Production and Manufacturing Management*. Springer, Boston, MA. https://doi.org/10.1007/1-4020-0612-8_580. (traducción propia de la cita textual)

17. Koop, G., Korobilis, D., & Pettenuzzo, D. (2019). Bayesian compressed vector autoregressions. *Journal of Econometrics*, 210(1), 135-154. <https://doi.org/10.1016/j.jeconom.2018.11.009>.
18. P. Taveeapiradeecharoen, K. Chamnongthai and N. Aunsri, "Bayesian Compressed Vector Autoregression for Financial Time-Series Analysis and Forecasting," in *IEEE Access*, vol. 7, pp. 16777-16786, 2019, doi: 10.1109/ACCESS.2019.2895022.
19. Livieris, I.E., Pintelas, E., & Pintelas, P. (2020). A CNN–LSTM model for gold price time-series forecasting. *Neural Computing and Applications*, 32(12), 17351-17360. <https://doi.org/10.1007/s00521-020-04867-x>
20. Fan, H., Jiang, M., Xu, L., Zhu, H., Cheng, J., & Jiang, J. (2020). Comparison of Long Short Term Memory Networks and the Hydrological Model in Runoff Simulation. *Water*, 12(1), 175. <https://doi.org/10.3390/w12010175>
21. Kordonis, J., Symeonidis, S., & Arampatzis, A. (2016). Stock Price Forecasting via Sentiment Analysis on Twitter. In *Proceedings of the 20th Pan-Hellenic Conference on Informatics (PCI '16)* (pp. 1-6). Association for Computing Machinery. <https://doi.org/10.1145/3003733.3003787>
22. Yu, S., & Ou, J. (2009). Forecasting Model of Agricultural Products Prices in Wholesale Markets Based on Combined BP Neural Network-Time Series Model. In *International Conference on Information Management, Innovation Management and Industrial Engineering* (pp. 558-561). IEEE. <https://doi.org/10.1109/ICIII.2009.140>.
23. Botchkarev, A. (2019). Performance Metrics (Error Measures) in Machine Learning Regression, Forecasting and Prognostics: Properties and Typology. *Interdisciplinary Journal of Information, Knowledge, and Management*, 14, 45-79. <https://doi.org/10.28945/4184>
24. penAPI Initiative. (n.d.). Introduction. Recuperado Febrero 23, 2023, de <https://oai.github.io/Documentation/introduction.html>
25. Bansal, M., Goyal, A., & Choudhary, A. (2022). Stock Market Prediction with High Accuracy using Machine Learning Techniques. *Procedia Computer Science*, 215, 247-265. <https://doi.org/10.1016/j.procs.2022.12.028>.

26. Biswas, M., Shome, A., Islam, M. A., Nova, A. J., & Ahmed, S. (2021). Predicting Stock Market Price: A Logical Strategy using Deep Learning. In 2021 IEEE 11th IEEE Symposium on Computer Applications & Industrial Electronics (ISCAIE) (pp. 218-223). Penang, Malaysia: IEEE. doi: 10.1109/ISCAIE51753.2021.9431817.
27. Bittencourt, M. V. L., & Mosquera Agudelo, P. A. (2021). The impacts of the exchange rate volatility on Colombian trade with its main trade partners. *EconoQuantum*, 18(2), 57-81.
28. Villamizar Villegas, M. (2016). Identifying the effects of simultaneous monetary policy shocks. *Contemporary Economic Policy*, 34(2), 268-296. <https://doi.org/10.1111/coep.12144>
29. Amaya, C. S. H., & López, A. E. G. (2015). Funcionamiento del mercado forex, impacto en Colombia. *Innovando en la U*, (7), 133-147.

ANEXOS

Enlaces al código de los algoritmos de Machine Learning que se exponen en la sección 2.1.2

A continuación, se presenta la tabla que relaciona el nombre del algoritmo con el enlace al código escrito en Python utilizando Jupyter Notebook. Todos los algoritmos realizan la predicción sobre el precio de una divisa.

Nombre	Descripción	Enlace
SVR-Wavelets	Algoritmo que utiliza Wavelets para descomponer la función y vectores de regresión para hacer la predicción.	https://drive.google.com/drive/folders/1LUhNpwbWYVTjsxs5alEhIJRvd_uOEsOy?usp=sharing
BCVAR	Algoritmo que utiliza inferencia Bayesiana y vectores auto-regresivos para realizar la predicción.	https://drive.google.com/drive/folders/1NinZUD7uvfxV6LEcwHBq7gJGusNHGZry?usp=sharing
LSTM-CNN	Algoritmo que utiliza redes neuronales LSTM (Long Short Term Memory) junto con redes CNN (Convolutional Neural Network) para realizar la predicción.	https://drive.google.com/drive/folders/1jG7wIkSmShCq3fYRLstMfJs671hNlaN9?usp=sharing
Sentiment Analysis - Twitter	Algoritmo que utiliza análisis de sentimientos sobre los "Tweets" que se descargan de Twitter para luego realizar la predicción.	https://drive.google.com/drive/folders/1TktwxNaCyHzrUvtu2tKU1eHY3xVVLyCg?usp=sharing
RBF NN	Algoritmo que utiliza funciones radiales sobre una red neuronal para mejorar la predicción hecha.	https://drive.google.com/drive/folders/1t5jW0CZrgfTTf50Oi4v493jdTYyA0_2I?usp=sharing

Enlaces al código del prototipo realizado

A continuación, se presenta la tabla con los enlaces al código desarrollado para el montaje del prototipo de la sección 6. Los servicios web fueron escritos en Java y la interfaz de usuario en ReactJS.

Nombre	Descripción	Enlace
Analizador Código	Componente encargado de analizar el código en busca de vulnerabilidades.	https://github.com/kgordillo-hub/AnalizadorCodigo
Generador Entornos	Componente encargado de crear la imagen Docker, aprovisionar el ambiente de ejecución y de correrla en el ambiente local.	https://github.com/kgordillo-hub/GeneradorEntornos
Gestor Parametros API	Componente que valida y guarda la estructura de la especificación del servicio.	https://github.com/kgordillo-hub/GestorParametrosAPI
Metricas	Componente que calcula el MSE, MAPE y RMSE de los predicciones arrojadas por los algoritmos.	https://github.com/kgordillo-hub/Metricas
Gestor catálogo	Componente para gestionar tanto el catálogo general como el privado de algoritmos a usar.	https://github.com/kgordillo-hub/GestorCatalogo
Gestión resultados	Componente que gestiona los resultados arrojados por los algoritmos.	https://github.com/kgordillo-hub/GestionResultados
Vinculador Código	Componente que permite la vinculación de nuevos algoritmos a la plataforma.	https://github.com/kgordillo-hub/VinculadorCodigo
Frontend	Programa que contiene toda la parte visual de la plataforma.	https://github.com/kgordillo-hub/material-kit-react
Wavelets+SVR	Algoritmo de ML como servicio descrito en sección 2.1.2.1	https://github.com/kgordillo-hub/1.Wavelets_SVM
BCVAR	Algoritmo de ML como servicio descrito en sección 2.1.2.2	https://github.com/kgordillo-hub/2.BCVAR
LSTM+CNN	Algoritmo de ML como servicio descrito en sección 2.1.2.3	https://github.com/kgordillo-hub/3.LSTM_CNN
SA + SVR	Algoritmo de ML como	https://github.com/kgordillo

	servicio descrito en sección 2.1.2.4	o-hub/4.SA_Twitter
RBF + NN	Algoritmo de ML como servicio descrito en sección 2.1.2.5	https://github.com/kgordill o-hub/5.RBFNN